Manual Book

Biz-A

Histori perubahan **6**
Pendahuluan **9**
Istilah-istilah 9
Bab 1. Cara Kerja Biz-A **10**
Blok Diagram Biz-A 10
FlowChart Proses Login 11
Bab 2. Struktur File **12**
Bab 3. Main **13**
menu.json 13
Bab 4. File Template **14**
Definisi 14
Standard Konvensi Penamaan FIle Template 14
Tempat Penyimpanan 14
Lokal 14
Database Biz-A 14
Cara Upload Template ke Database 15
Install biz-a-cli 15
Upload App Template ke Database 15
Hapus App Template dari Database 16
Format Template Form 17
model 17
Format: 17
Penjelasan 17
Contoh: 18
tableName 18
functions 18
fields 19
Type Fields Yang Tersedia 19
Bawaan dari Formly 19
Input 20
money 22
date 23

bool 24
select 25
lookup 26
input-file 29
button 30
detail-grid 31
one-record-label 38
read-only-table 42
repeat 43
accordion 47
drag-drop-list 49
label 51
img 52
horizontal-stepper 54
chart 56
text 57
selection-list 58
swiper 59
report 60
Validator 61
Built-in Validator 61
Custom Validator (Synchronous) 62
Custom Validator (Asynchronous) 63
Variable 65
Html 66
Format Template List 67
caption 67
columns 67
Penjelasan 68
filter 68
distinct 69
functions 69
buttons 69
dbIndex 69
badge 70
Main Form template 71
Bab 5. Functions **71**
Cara Memanggil Fungsi 71
Fungsi-fungsi Reserved yang bisa digunakan: 71
onInit 71
onDestroy 72

useLibrary 72
beforeLoadData 73
afterLoadData 73
onPipe 73
Cara Memanggil Fungsi yang Lain 74
Event Button Click 74
Cara Mencetak 75
Event 76
Cara menggunakan Event 76
Cara Menampilkan Custom Form Dialog 76
Cara Ekspor Data ke File Excel 77
Cara Format Angka Menjadi Currency 78
Bab 6. Daftar Service yang Tersedia 78
dataService 78
Method sendModel 78
Method deleteModel 79
Method deleteModelNonId 80
Method deleteModelMD 80
Method deleteModelNonId 81
Method getModel 82
Method queryData 83
Cara pengisian key data 84
Cara Menggunakan Filter 86
Cara Mengubah apiConfig 87
Method json2fina 88
Method previewTemplateFina 89
Method getCurrentDate() 90
Method httpPost() 91
Method executeBlock() 92
Method updateImg() 92
Method genId() 93
executeCmd 93
Property http 94
callback 94
appService 96
sendMail 96
getCurrentDateTime 96
getLocation 97
logout() 97
helper 97
formatDate2API 97

loadingService 98
start() 98
updateProgress 98
stop 98
storageService 100
getCurrentUser() 100
getLoginInfo() 100
apiSvc 101
loadScript() 101
proxy(method:string, url:string, config:IAppConfig, body:any=null, options:any=null) 101
printService 103
print 103
authService 104
logout 104
getLogoutEvent 104
isLoggedIn 104
getExpiration 104
config 104
api 105
app 105
mapService 106
L 106
init 106
tile 106
marker 106
db 108
scriptService 109
load 109
rx 110
bottomSheetService 111
notificationService 113
Bab 7. Cara Setup Biz-A 114
Setting CDM 114
Setting FINA API 114
Setting Biz-A Server 114
Testing Tahap 1 115
Setting Menu 115
Setting File Template 115
Cara Membuat Form 116
Langkah Pertama 116
Langkah Kedua 116

Langkah Ketiga 116
Langkah Keempat 117
Langkah Kelima 118
Cara Membuat List 119
Langkah Pertama 119
Langkah Kedua 119
Langkah Ketiga 119
Langkah Keempat 119
Langkah Kelima 119
Setting Biz-A Client 119
Deployment 120
Debugger 125
Bab 8. Proxy Lokal & FINA Hub 125
Proxy Lokal 125
FINA Hub 126
WatchDog FINA Hub 128
Bab 9. Aturan Pembuatan Database 130
Wajib 130
Tabel 130
Generator 130
Trigger 130
Disarankan 130
Bab 10. Hak Akses 131
Pendahuluan 131
Bab 11. Management by Exception & Callback 132
Bab 12. Struktur Parameter 132
IApiConfig 132
Bab 13. Contoh Code 132
Contoh Membuat Field menghilang dalam kondisi tertentu sesuai isi modelnya 132
Contoh di detail-grid untuk hitung otomatis saat ada perubahan di qty ataupun Harga 133
Contoh Kirim Email di BIZA, setelah simpan ke database. 135
Contoh Untuk readonly field tipe money, select, & date 136
Contoh SQL membuat table, generator dan trigger sekaligus bisa menggunakan berikut. 137
Petunjuk Khusus 137
Contoh Cara Menyimpan File ke tabel 140

Histori perubahan

Tanggal	Keterangan	Oleh
11 Sep 2023	Drag-drop-list bisa object Penambahan ExecuteCmd di DataService	MAS
16 Sep 2023	Penambahan fungsi onPipe di List	MAS
19 Sep 2023	Detail-grid ada penambahan fitur: ● Bisa menampilkan tombol ● Tipe money & number akan rata kanan ● Penambahan props.rowStyleFn ● Penambahan property columns.readonly ● Untuk type: boolean bisa set icon ● Penambahan fungsi addColumn	MAS
20 Sep 2023	Penambahan type chart Perbaikan bug kalau loncat dari form ke form, form control-nya tidak ter-reset	MAS
29 Sep 2023	Detail-grid ada penambahan fitur addBtn & deleteBtn	MAS
9 Oct 2023	Lookup ada penambahan fitur: infinite scroll & searchKey	MAS
12 Okt 2023	One-record-label & read-only-table ada penambahan fitur index bisa indented.	MAS
19 Okt 2023	read-only-table ada penambahan fungsi setData	MAS
21 Okt 2023	detail-grid ada penambahan: 1. Ada property read-only di columns 2. Ada property onCalc Lookup: property filter bisa array dalam array	MAS
24 Okt 2023	Menghilangkan fungsi headerHTML & footerHTML. Gunakan template:'' untuk menampilkan HTML di form	MAS
16 Nov 2023	Ada penambahan parameter di function onCalc detail-grid	MAS
1 Dec 2023	Default value dari date adalah tanggal hari ini. Detail-grid akan load nilai dari model bila array pertama & key pertama not null	MAS
12 Dec 2023	Perbaiki keterangan config yang sebelumnya dibaca dari file menjadi dari	SCY

	Database Config	
13 Dec 2023	Ada penambahan fungsi previewTemplateFina di DataService	MAS
5 Jan 2024	Penambahan penjelasan filter di queryData	MAS
17 Jan 2024	Penambahan keterangan versi biz-a-cli jika ingin menggunakan Proxy	SCY
23 Jan 2024	Penambahan this.f() untuk memanggil fungsi	MAS
25 Jan 2024	Penambahan style di detail-grid & list	MAS
1 Feb 2024	Penambahan fungsi loadOptions di lookup	MAS
8 Feb 2024	Perbaikan date agar bisa diketik	MAS
9 Feb 2024	Penambahan fungsi validator	MAS
17 Feb 2024	Penambahan tipe select di detail-grid	MAS
22 Feb 2024	Penambahan event beforeEdit di detail-grid	MAS
13 Mar 2024	Ada perbaikan di button, label dan penambahan komponen text. Pemindahan getLoginInfo dari storageService ke config service.	MAS
16 Mar 2024	Penambahan mapService Mendokumentasikan daftar variable di Form	MAS
20 Mar 2024	Penambahan db service untuk indexedDB	MAS
21 Mar 2024	Penambahan event afterEdit di detail-grid Penambahan setting searchDisplay di lookup option.	MAS
23 Mar 2024	Penambahan scriptService Penambahan komponen selection-list	MAS
29 Mar 2024	Penambahan badge di button Penambahan rx service. Penambahan icon di input.	MAS
3 Apr 2024	Penambahan reserved function onDestroy	MAS
5 Apr 2024	Penambahan optional argumen “port” (default 212) pada uploadBizA Penambahan tabel “App” di database BizA Penambahan perintah uploadApp	AA
8 Apr 2024	Penambahan bottomSheetService Tombol add & delete di repeat sudah bisa pakai logo Penambahan notificationService	MAS
12 Apr 2024	Penambahan fungsi callback di Dataservice.	MAS

20 May 2024	Ubah informasi manual book tabel APPS menjadi SYS$APPS	SCY
30 May 2024	Penambahan komponen swiper	MAS
30 Jul 2024	Perubahan perintah uploadApp menjadi BizA Add Perubahan perintah deleteApp menjadi BizA Remove	AA
22 Aug 2024	Penambahan type report	MAS
25 Sep 2024	Hub CLI bisa melakukan koneksi ke FINA API HTTPS	AA
13 Des 2024	Penambahan fungsi beforeLoadData & afterLoadData di List saat pagination & filter.	MAS
28 Feb 2025	Penambahan cara send post via http ke API server selain FINA API	MAS
3 Mar 2025	Penambahan fitur `editOnEnter` untuk detail-grid	SCY
13 Juni 2025	Deployment dan setting page	AA
16 Sep 2025	Menambahkan contoh-contoh kode untuk melatih AI	SRF

=== DIGEST START ===

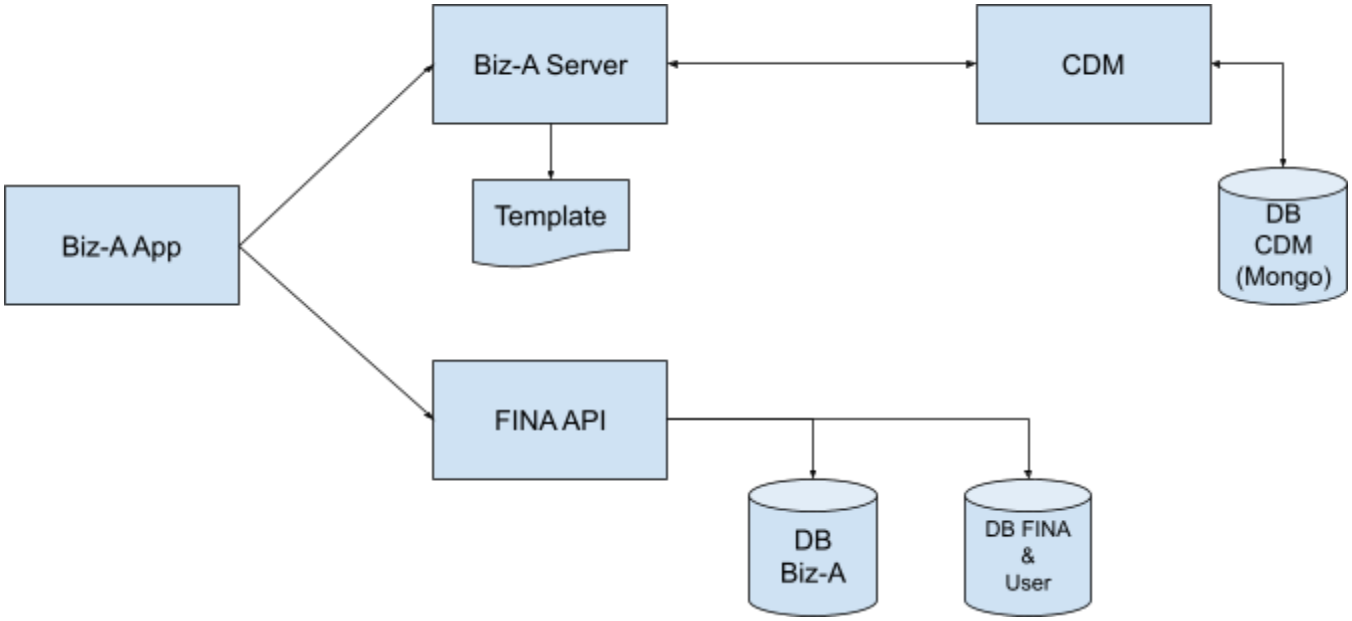
Pendahuluan

Istilah-istilah

1. Form: adalah user interface yang digunakan untuk isi, edit atau lihat sebuah record data Master yang bisa jadi mempunyai beberapa child atau grandchild.
2. List: adalah user interface yang menampilkan daftar data-data

Bab 1. Cara Kerja Biz-A

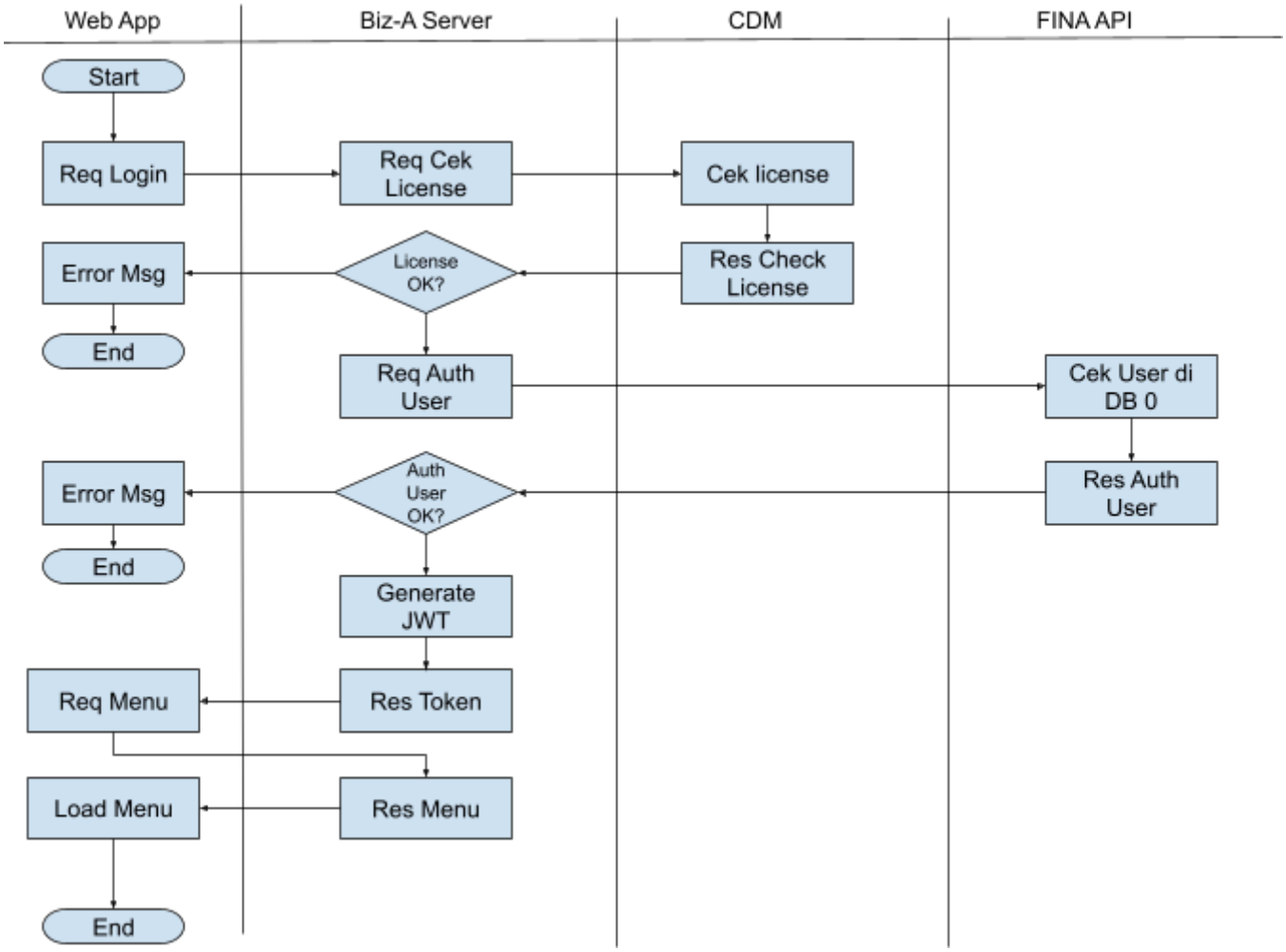
Blok Diagram Biz-A



Keterangan:

1. Biz-A App adalah aplikasi web yang dibuat menggunakan Angular. File ini akan di-host di https://biz-a.id.
2. Biz-A server adalah aplikasi server yang akan melayani request dari Biz-A App.
3. CDM adalah Customer Database Management-nya Imamatek. Fungsi CDM adalah mengontrol concurrent user agar selalu sesuai dengan lisensi yang masih aktif.
4. FINA API adalah program FINA yang berbasis windows, baik FINA yang bermuka maupun hanya webservice saja.
5. Database User akan selalu disimpan di database FINA di index 1. Jadi username & password pengguna Biz-A akan selalu disimpan di database FINA.
6. Database Biz-A disimpan di 1 wilayah LAN dengan FINA API. Database ini akan menyimpan segala data yang dibutuhkan oleh aplikasi yang dikembangkan oleh masing-masing perusahaan. Selain itu, Database Biz-A juga menyimpan segala customize program/data dari masing-masing company. File-file ini disebut dengan template.

FlowChart Proses Login



Penjelasan:

1. User membuka laman biz-a.id, lalu mengisi nama company, user & password.
2. Saat user klik tombol login, Web App akan kirim request login ke Biz-A Server. Format yang dikirim: {companyname, username, password}
3. Biz-A Server akan baca nama company & password CDM dari Database Config.
    a. Bila config file tidak ditemukan, akan return: 501, “Company does not exist”.
    b. Apabila license masih valid maka App Server akan request ke FINA API untuk mengecek apakah username & password-nya valid.
    c. Bila license tidak valid, akan response ke Web App dengan error: 401, “Sign in information is not valid”.
4. Bila license valid, App Server akan request ke CDM untuk mengecek lisensi perusahaan tersebut.
    a. Kalau username valid, akan diambil userID-nya dan akan dikembalikan ke web app bersama dengan token yang baru di-generate. Format data:
    {

idToken: token JWT,
expiresIn: xxx dalam detik,
userId:integer,
apiUrl: string → url ke FINA API
}

b. Bila tidak valid, akan response ke Web App dengan kode 401: “Username or password is not valid”.

5. Token dari Biz-A Server akan disimpan oleh Web app dan akan digunakan saat request ke App Server maupun ke FINA API.
6. Saat Web App request ke FINA API, FINA API akan baca token tersebut dan memvalidasi apakah token masih valid. Kalau masih valid, request dilanjutkan.

Bab 2. Struktur File

- \key
  - private.key → berisi private key RS256 512 bit
  - public.key → berisi pasangan public key di atas
- ~~\[nama-perusahaan]~~
  - ~~config.json, yang berisi:~~

~~{~~

~~companypassword:string,~~
~~isEncrypted: true/false, → kalau true: di hash dengan MD5~~
~~expiresin: xxx (dalam detik),~~
~~API_URL: string, → berisi IP dan port FINA API,~~
~~dbindex: number → diisi dengan index database di API yang digunakan,~~
~~smtp:{~~
~~host:xx.xx.xx.xx,~~
~~port:465,~~
~~secure: true,~~
~~auth:{~~
~~user: a@b.com~~
~~pass: password nya user~~
~~}~~
~~}~~

~~}~~Saat ini sudah dipindahkan ke Database Config

Bab 3. Main

menu.json

Contoh Isi:

```
[
	{“caption”: “menu 1”, “link”:[“./main”]},
	{“caption”: “menu 2”, “subMenu”:[
		{“caption”: “menu 2.1”, “link”:[“./form”, “[formName]”, null]},
		{“caption”: “menu 2.2”, “link”:[“./form”, “[formName]”, null]},
	]}

]
```

Penjelasan:

1. Selalu dibuka dengan **[** dan ditutup dengan **]**
2. Setiap baris terdiri dari 4 attribute:
    a. “caption”: diisi dengan nama menu yang akan ditampilkan.
    b. “link”:
        i. Hanya diisi kalau tidak ada sub menu.
        ii. Bila tidak punya sub menu, isi dengan:
            1. [“./main”] → masuk ke main form
            2. [“./form”, “[formname]”, null] → masuk ke file **formname.js**.
            3. [“./list”, “[listname]”] → masuk ke file **listname.js**.
    c. “subMenu”:
        i. Hanya perlu dipasang bila ada sub menu..
        ii. Bila ada sub Menu, diisi dengan baris seperti no: 2.
3. Semua menu bisa indent dengan jumlah yang tidak dibatasi.

Simpan file ini 1 folder dengan file-file template lainnya.

Bab 4. File Template

Definisi

Template di Biz-A merupakan program javascript atau json yang berfungsi untuk mengatur perilaku Biz-A. Template adalah File yang berekstensi ***.js*** yang berisi segala customize program dan juga file yang berekstensi ****.json*** yang berisi pengaturan menu di Biz-A

Standard Konvensi Penamaan FIle Template

Untuk standard konvensi penamaan file template terdiri menjadi 3 bagian :

1) **Menu :** Untuk penamaan template menu bisa langsung dibuatkan nama ***"menu.json"*****.** Khusus untuk Template menu file berekstensi ****.json.***
2) **Form :** Untuk penamaan template form, kata pertama adalah ***"nama tabel"*** dan kata kedua adalah kata "***form"*** di bagian. Contohnya : ***"karyawanform.js"*****.** Dengan tujuan, agar kita tahu bahwa file tersebut untuk form. Untuk template dari form berekstensi ****.js*****.**
3) **List :** Untuk penamaan template list, kata pertama adalah ***"nama tabel"*** dan kata kedua adalah kata ***"list/daftar"***. Contohnya : ***"karyawanlist.js"***. Dengan tujuan, agar kita tahu bahwa file tersebut untuk list. Untuk template dari list sama dengan form, yaitu berekstensi ****.js*****.**

Tempat Penyimpanan

Lokal

Source code Biz-A di folder lokal komputer programmer harus ditempatkan dalam struktur sebagai berikut:
`\biz-a-template\[nama company]\`
[nama company] diganti dengan nama perusahaan atau nama fitur.

Database Biz-A

Source code yang sudah dibuat oleh programmer, agar bisa digunakan oleh Biz-A, harus di-upload ke database.

Semua template Biz-A untuk company tertentu akan disimpan di dalam database firebird.
Pastikan struktur tabel berikut ini sudah ada di database tersebut:

```
SET TERM ^ ;
CREATE TABLE TEMPLATE(
  ID INTEGER NOT NULL,
  FILENAME VARCHAR(256) NOT NULL,
```

```
  TEMPLATE BLOB SUB_TYPE 0,
  PRIMARY KEY (ID),
  UNIQUE (FILENAME)
)^
CREATE GENERATOR TEMPLATE_GEN^
CREATE TRIGGER TEMPLATE_BI FOR TEMPLATE ACTIVE
BEFORE INSERT POSITION 0
AS BEGIN
  new.ID=GEN_ID(TEMPLATE_GEN,1);
END^
SET TERM ; ^
```

Cara Upload Template ke Database

Install biz-a-cli

Caranya:

1. Download & install node.js
    a. Versi node yang digunakan: 20.16.0
    b. Versi npm yang digunakan: 10.8.1.
2. Buka cmd
3. Jalankan: **npm install -g biz-a-cli@latest**
4. Cek apakah **biz-a-cli** sudah ter-install dengan baik atau belum dengan cara mengetik: **biza**
5. Apabila muncul cara penggunaan biza berarti biz-a-cli sudah ter-install dengan baik

Upload App Template ke Database

1. Bila menggunakan local/public IP, ketik:
   BizA add -s [IP of FINA API] -p [default: 212] -i [dbindex] -d [full or relative path to app folder]

   Contoh pakai IP tanpa path:
   BizA add -s http://192.168.1.1 -i 2

   Contoh pakai localhost:
   BizA add -s http://localhost -i 2

   Contoh dengan path:
   BizA add -s http://192.168.1.1 -i 2 -d "c:\biza\imamatek"

   Contoh dengan port non default:
   BizA add -s http://192.168.1.1 -p 1205 -i 2 -d "c:\biza\imamatek"

2. Bila menggunakan BizA Hub, ketik:
   BizA add -s [BizA Hub Server] -i [dbindex] --sub [subdomain of BizA Hub Server] -d [full or relative path to app folder]
   Contoh :
   `biza add -s https://server.biz-a.id -i 2 --sub imm -d "c:\biza\imm"`

3. Bila menggunakan Domain / Sub Domain, ketik:
   `BizA add -s [Domain] -i [dbindex] -d [full or relative path to app folder]`
   Contoh :
   `biza add -s https://finaapi.imamatek.com -i 2 -d "c:\biza\imm"`

4. Tunggu hingga proses selesai.
5. Setiap kali file-file template mengalami perubahan, ulangi langkah-langkah di atas.
6. Link yang ada di menu, agar bisa dibedakan untuk App yang mana, maka di tambahkan **namaApp-formname**. Delimiter menggunakan **Hypen Symbol (-)**. Pastikan nama App **tidak** menggunakan Hypen (-) bila app tersebut akan dipakai sebagai bagian dari multi App

Hapus App Template dari Database

1. Bila menggunakan public IP, ketik:
   BizA remove -s [IP of FINA API] -p [default: 212] -i [dbindex] -n [appname | multiple appname]
   Contoh menghapus semua app :
   `biza remove -s http://192.168.1.1 -i 2`

   Contoh menghapus 1 app :
   `biza remove -s http://192.168.1.1 -p 1205 -i 2 -n "imamatek"`

   Contoh menghapus lebih dari 1 app:
   `biza remove -s http://192.168.1.1 -p 1205 -i 2 -n "imamatek, spos"`

2. Bila menggunakan BizA Hub, ketik:
   BizA remove -s [BizA Hub Server] -i [dbindex] --sub [subdomain of BizA Hub Server] -n [appname | multiple appname]
   Contoh :
   `biza remove -s https://server.biz-a.id -i 2 -sub imm -n "imamatek"`

3. Bila menggunakan Domain / Sub Domain, ketik:
   BizA remove -s [Domain] -i [dbindex] -n [appname | multiple appname]
   Contoh **:**
   `biza remove -s https://finaapi.imamatek.com -i 2 -n "imamatek"`

4. Tunggu hingga proses selesai.

Format Template Form

Semua form yang dibuat harus mempunyai 1 file template dengan ekstensi **js** di dalam folder yang sama.. Isi file ini harus mengikuti format sebagai berikut:

```
get = function(){
      return {
            model:{},
            tablename:string,
            fields:[],
            functions:{},

      }
}
```

model

Digunakan di Form.
Berisi object yang akan di-map ke GUI web untuk menampung data dari form tersebut.

Format:

```
{
      [property]:[default value],
      [detail_table]:[
            {
                  [property]:[default value]
            }
      ]
}
```

Penjelasan

1. Model harus mempunyai key **id** dan harus ditempatkan di urutan pertama.
2. Nama key harus sama dengan nama field di tabel, namun dengan huruf kecil semua.
3. Untuk tabel detail, tambahkan key dengan nama sesuai dengan nama tabel detail-nya, value-nya adalah array dengan sebuah object.
4. Model detail sama seperti model master dengan penambahan key ke-2 harus bernama nama tabel master + **_id**. Misal: nama table master-nya adalah cuti maka key name-nya harus cuti_id.
5. Kalau model tidak diperlukan, kosongkan saja. Model yang kosong akan menyebabkan Biz-A tidak akan query data awal ke FINA API. Kondisi ini berguna untuk Form yang tidak mempunyai tabel di database Biz-A.

Contoh:

```
model:{
      id: null, //harus didahului dengan id (huruf kecil)
      tanggal_mohon : null,
      id_karyawan:null,
      tanggal_mulai_cuti: null,
      tanggal_akhir_cuti: null,
      jumlah_hari_cuti:0,
      alasan_cuti:null,
      detail_table:[
            {
                  id:null,
                  tableName_id:null,
                  detailField1:null,
            }
      ]
},
fields:{}
```

tableName

Digunakan di Form untuk menentukan nama tabel yang akan digunakan untuk menyimpan data model. Bila diisi maka Biz-A akan mencoba load secara otomatis data dari tabel. Kosongkan isi tableName apabila tidak mau load data secara otomatis.

functions

Key ini digunakan untuk membuat fungsi-fungsi yang digunakan di form.
Format:

```
functions: {
      [nama function1]: function ([parameter func]){
                        [...code]
      },
      [nama function2]: function ([parameter func]){
                        [...code]
      }
}
```

Penjelasan lebih rinci mengenai cara membuat function akan dibahas di bab selanjutnya

fields

Digunakan di Form.
Fungsi: Ini adalah untuk menampilkan komponen apa saja yang ada di dalam form tersebut.
Lengkapnya ada di Formly Field Config

Format data:

```
fields:[
      Komponen1,
      Komponen2,
      ….
]
```

Type Fields Yang Tersedia

Bawaan dari Formly

Semua komponen default bawaan dari Formly Material dapat digunakan. Daftar lengkapnya dapat dilihat di sini.

Apabila ingin set attribute tertentu, silakan gunakan cara sebagai berikut:

```
props:{
  attributes:{
    style:”background:green;”,
    …
  }
}
```

Input

Fungsi: komponen isian bebas alfanumerik.
Format:

```
{
   key: “xx”, //diisi dengan nama field tabel
   type: "input"|”money”|”integer”,
   props: {
     label?: string
     placeholder?: string
     required?: boolean
     description?: string,
     addonLeft?: AddonParam,
     addonRight?: AddonParam,
     hintStart?:string,
     hintEnd?:string

   }
}
```

Penjelasan props:

1. Label: diisi dengan caption label di kiri edit box.
2. Placeholder: diisi dengan tulisan ini akan muncul di dalam edit box saat kosong.
3. Required: diisi true kalau harus diisi. False kalau boleh kosong. Default= false.
4. Description: diisi dengan deskripsi yg akan muncul di bawah
5. Type: Selain diisi dengan input, type juga bisa diisi dengan berikut:
    a. color
    b. datetime-local
    c. email
    d. month
    e. password
    f. time
    g. week
6. AddonParam adalah object untuk menampilkan icon atau text di depan atau belakang input. Strukturnya adalah sebagai berikut:

    ```
    {
      icon?:string,
      text?:string,
      onClick:(control)=>{

      }
    }
    ```

    a. Isi icon atau text.
    b. Icon diisi dengan nama icon
    c. Text diisi dengan nama tulisan yang akan ditampilkan

    d. onClick adalah event handler saat icon/text tersebut diklik. Parameter yang dikirim adalah kontrol dari input.
7. addonLeft diisi apabila ingin menampilkan icon/text di depan input.
8. addonRight diisi bila ingin menampilkan icon/text di belakang input.
9. hintStart diisi bila ingin menampilkan kalimat penjelasan. Fungsinya sama seperti description.
10. hintEnd diisi bila ingin menampilkan hint yang nempel di belakang

money

Fungsi: Komponen input khusus untuk uang. Format yang ditampilkan akan ditambah: Mata uang, pemisah ribuan dan pemisah desimal.
Saat edit, ketiga karakter tersebut akan hilang.

```
{
   key: “xx”,
   type: "money",
   props: {
     label: "xxx",
     placeholder: "xxx",
     required: "true"
   }
}
```

date

Fungsi: Komponen input khusus untuk tanggal.

Format:

```
{
   key: “xx”,
   type: "date",
   props: {
     label: "xxx",
     placeholder: "xxx",
     required: "true",
     today: “true”,
   },
   modelOptions:{
     updateOn:’blur’
   }
}
```

Penjelasan:

1. Format display-nya adalah DD MMM YYYY.
2. Saat edit, formatnya adalah D/M/YYYY
3. Bila ingin set nilainya baik dari model maupun dari form Control, format-nya adalah: YYYY-MM-DD.
4. Contoh:
   a. `this.model.tanggal = ‘2013-03-13’;`
   b. `this.form.get(‘tanggal’).setValue(‘2013-03-13’);`
5. **today**, bila diset true, nilai default-nya adalah hari ini. Kalau kosong: nilai default: null.
6. **updateOn** perlu di-set dengan ‘blur’ agar nilai tanggal baru di-set saat kontrol kehilangan fokus, bukan setiap user ketik.

bool

Fungsi: menampilkan combo box untuk tipe field boolean.
Format:

```
{
   key: “xx”,
   type: "bool",
   props: {
     label: "xxx",
     required: "true",
     displayTrue: “betul”,
     displayFalse: “salah”
   }
}
```

Keterangan:

1. displayTrue : untuk mengganti tampilan pilihan untuk value: true. Kalau tidak diisi, default-nya adalah ‘true’.
2. displayFalse: untuk mengganti tampilan pilihan untuk value: false. Kalau tidak diisi default-nya adalah ‘false’;

select

Fungsi: Komponen untuk memilih dari drop down yang statis.
Format:

```
{
  key: "xx",
  type: "select",
  props: {
    label: "xxx",
    placeholder: "xxx",
    required: true,
    options:[ //berisi daftar yang akan ditampilkan
      { value: "1", label: "A"},
      { value: "2", label: "B"},
      { value: "3", label: "C"}
    ]
  }
}
```

lookup

Fungsi: Komponen dengan drop down yang lookup ke tabel tertentu
Format:

```
{
  key: "xx",
  type: "lookup",
  props: {
    label: "xxx",
    placeholder: "xxx",
    required: "true"
    lookupOptions:{ //berisi setting table lookup
      table:"xxx",//diisi dengan nama tabel lookup
      key:"xxx",//diisi dengan nama field ID lookup
      display:"xxx",//diisi dengan nama field yang tampil di list
      dbIndex:"x",//optional bila dbIndex beda dgn default
      filter:string|FilterOption,//optional bila ingin filter data,
                                 //diisi dengan kondisi filter
      order:[{column:1, dir:'asc'|'desc'], //bila tidak diisi,
                //order by default adalah 2,
                //diisi index kolom atau nama field yg akan di-urut
      foreignKey: string,
      defaultDisplay:string,
      searchKey: array of string,
      searchDisplay: string,
    },
    readonly:boolean,
    style:string,
  }
}
```

Penjelasan:

1. Properti lookupOptions:
    a. **table** : nama tabel lookup
    b. **key**: primary key dari table lookup
    c. **display**: nama field yang mau ditampilkan.
    d. **dbIndex**: index database yang akan di-query
    e. **filter** bisa diisi dengan 2 tipe:
        i. String: Langsung nama_field = value. Contoh: `filter:"field1='a'"`
        ii. Bila kondisi filter lebih dari 1, gunakan format filter seperti di queryData.
    f. **order** Cara setting-nya sama dengan order di queryData.
    g. Untuk lookup ke stored procedure:
        i. Isi table dengan query yang dibungkus dengan tanda kurung buka & tutup.
        ii. Gunakan **$key** sebagai pengganti key.

iii. Display tidak perlu diisi lagi.
iv. Contoh: `table:"(select quantity from GetItemBal('$key', current_date, -1))"`

h. **foreignKey**: optional. Digunakan untuk mengisi nama key yang digunakan untuk lookup ke sebuah stored procedure, seperti ItemUnit. Untuk seperti itu, maka di table-nya diisi dengan: 'ItemUnit('$ForeignKey').
i. **defaultDisplay**: optional. Diisi untuk menggantikan tulisan 'Loading…' saat data lookup belum / gagal di-load.
j. **searchKey** : optional. Bila diisi, maka di panel options akan muncul edit box untuk search data di list options. Diisi dengan daftar nama field yang akan di-search.
k. **searchDisplay**: optional. Drop down window saat search akan menampilkan field sesuai dengan nilai searchDisplay. Bila tidak diisi, default sama dengan display.

2. Lookup akan melakukan infinite scroll.
   a. Secara default lookup akan load 10 record pertama. Saat panel options di-scroll maka akan load 10 berikutnya lagi.
   b. Cara mengubah jumlah record yang di-load setiap query adalah: `lu.lookup.queryLength = 100;`
   c. Di mana lu adalah instance dari lookup yang bisa diambil dengan cara: `let lu = this.fields[x]._componentRefs[0].instance;` x adalah index dari component lookup.
3. `readonly` : true berarti tidak bisa diganti.
4. `style`: cara isinya sama dengan props.attributes.style
5. Method
   a. loadOptions: berfungsi untuk memaksa lookup load options-nya tanpa perlu membuka drop down list. Cara penggunaannya lihat di contoh di bawah.

Contoh menampilkan lebih dari 1 field.
Bila ingin menampilkan lebih dari 1 field, pisahkan antar field dengan **|| ':' ||** dan dibungkus dengan tanda kurung.
Contoh:
display : "(fieldA || ':' || fieldB)"

Apabila ingin mengubah filter secara run-time, set ulang lookupOptions-nya. Nanti data lookup-nya akan di-load ulang.

Contoh:
```
this.fields[2].props.lookupOptions = newLookupOptions;
```

Contoh penggunaan loadOptions
```
onInit: function(){
  let lookup = this.fields[0]._componentRefs[0].instance.lookup;
  lookup.loadOptions()
}
```

Penjelasan:

1. fields[0] : ganti 0 dengan index fields lookup yang akan di-load.
2. Contoh ini akan melakukan loadOptions saat onInit. Bila ingin load di waktu yang berbeda, silakan lakukan di dalam pipe event tersebut

input-file

Fungsi: Sebagai komponen untuk input file.
Format:

```
{
      key:'nama_key',
      type:'input-file',
      props:{
        label:'text saat belum ada file yang di-upload’,
        filetype:'.xls, .xlsx',
      }
}
```

Keterangan:

1. **label** adalah tulisan yang ditampilkan saat belum ada file yang di-upload.
2. **filetype** diisi dengan extension apa saja yang bisa dipilih. Bila ada lebih dari 1, pisahkan dengan tanda koma.

button

Fungsi: sebagai komponen yang bisa diklik dan mengerjakan sesuatu.
Format:

```
{
  type: "button",
  props: {
    btnType   : string,
    text      : "xxx", //tulisan yang muncul di tombol
    color     : primary|accent|warn,
    variant   : 'basic(default)|raised|flat|stroked|icon|fab|mini-fab',
    icon      : string,
    disabled  : boolean
    onClick   : function($event, id){
                  //short code
                }
    badge:BadgeParam,
  }
}
```

Keterangan:

1. btnType:
    a. submit: untuk mengirim data ke server
    b. button: tombol biasa tanpa fungsi khusus
    c. reset: untuk mengembalikan isi form ke nilai awal
    d. Bila kosong, default-nya adalah **button**.
2. Props onClick mempunyai 2 parameter, yaitu:
    a. $event: berisi instance HTML button yang dikirim
    b. id : merupakan id unik dari button tersebut. Id unik ini bisa digunakan untuk mencari tahu button mana yang di-klik di FormArray
3. **variant** : string (optional). Bila tidak diisi, default = 'basic'. Contoh tampilan tombol bisa dilihat di sini.
4. icon: string (optional). Bisa digunakan untuk semua varian tombol. Daftar icon yang bisa digunakan ada di sini.
5. disabled: boolean (optional) untuk membuat tombol enable atau disable.
6. Short code adalah kode typescript yang akan dijalankan oleh browser saat tombol ditekan. Untuk lebih rinci, silakan lihat di bagian **Short Code**.
7. Badge: diisi dengan BadgeParam.

detail-grid

Fungsi: sebagai komponen yang digunakan untuk menampilkan data detail dari tabel Master-Detail.
Format:

```
{
      key:'id'
},
{
      type:'detail-grid',
      props:{
            tableName : 'nama_table_master',
            detailKey : 'nama_table_detail',
            watchKey  : string,
            dbIndex   : number,
            columns: [
                  { key: 'id'},
                  {
                        key   : 'xxx',
                        label : 'yyy',
                        type  : 'money'|
                                'number' |
                                'date'|
                                'boolean'|
                                'lookup_combo'|
                                'select'
                              ,
                        lookup : {
                              table  : 'aaa',
                              key    : 'bbb',
                              display: 'ccc',
                              dbIndex: 'x',
                              filter : 'xx=yy',
                              order  : [{column:1, dir:'asc'}]
                        },
                        options:[
                          {label:string, value:string},...
                        ],
                        arg1:string,
                        arg2:string,
                        readonly:boolean,
                        useIcon:boolean,
                        style:object,
                  },
                  {
```

```
                key:'id',
                label:'Action',
                buttons:[
                  {
                    caption: string, //optional
                    icon:string,
                    color:string,
                   style:object,
                    onClick: function(id, rowIndex){
                    }
                  }
                ]
              }
          ],
          rowStyleFn:function(row,idx){
          },
          addBtn:{
            show:boolean,
            icon:string (optional),
            label: string (optional),
          },
          deleteBtn:{
            show:boolean,
            icon:string (optional),
            label:string,
            key:string
          },
          onCalc:[
            {
              watchKey:string[],
              function:function(rowControls, value, key){
              }
            }
          ],
          editOnEnter: boolean (Default: False)
      }
}
```

Penjelasan:

1. Pastikan ada field key id baik di master maupun di detail.
2. props:
    b. **tableName**: diisi dengan nama table master
    c. **detailKey**: diisi dengan nama table detail. Key ini harus sama dengan key detail di model yang bertipe array of object.
    d. **Columns** array of object mempunyai mempunyai properti sebagai berikut:

i. **key**: akan menggunakan key dari model detail yang mana
ii. **label**: akan menentukan caption dari column header. Kalau tidak diisi, kolom tersebut akan hidden.
iii. **type**: cara menampilkan field tersebut. Pilihannya adalah:
    1. **date**: format tanggal. Secara default tampilannya adalah 'DD MMM YYYY'. Apabila mau diganti, isi formatnya di arg1.
    2. **money** : format mata uang. Default format adalah Rp. Kalau mau diubah, isi arg1 dengan locale setting dan arg2 untuk mata uangnya. Misal untuk Amerika serikat, isi **arg1** dengan **en-US** dan **arg2** dengan **USD**.
    3. **number**: format angka dengan pemisah ribuan dan tanpa desimal. Default adalah setting angka di Indonesia. Kalau mau diubah, isi arg1 dengan kode locale-nya. Misal untuk Amerika serikat, isi: **en-US**.
    4. **boolean** : true / false. Tampilan true/false bisa diganti dengan mengisi **arg1** untuk nilai true dan **arg2** untuk nilai false
    5. **lookup_combo** : field yang lookup ke tabel yang lain dan cara memilihnya menggunakan combo box.
    6. Selain itu berarti tidak di-format.
iv. **lookup** hanya digunakan bila **type**-nya adalah: **lookup_combo** atau **lookup_form**. Tipe datanya sama dengan lookupOptions-nya komponen lookup.
v. **buttons**: berfungsi untuk menampilkan tombol di detail-grid
    1. key akan digunakan untuk mengirim id saat tombol di-klik
    2. caption: untuk menampilkan tulisan di tombol (optional)
    3. color : lihat color button di list.
    4. style: optional, object untuk set style button.
    5. onClick: diisi dengan function dengan parameter id & rowIndex. Fungsi ini akan dipanggil saat tombol di-klik.
vi. **readonly**: boolean. Bila di-set true, kolom yang bersangkutan tidak bisa di-edit. Default false.
vii. **useIcon**: boolean (default false). Hanya berfungsi untuk type boolean. Bila useIcon true, maka arg1 akan menjadi icon display saat true dan arg2 untuk false.
viii. **style**: object (optional). Untuk set style kolom yang bersangkutan.
ix. **options**: array (optional). Hanya berfungsi untuk tipe **select**. Cara mengisinya sama seperti komponen select.

e. rowStyleFn: adalah fungsi yang kembaliannya berupa string maupun object untuk menentukan styling per baris.
    i. Parameter yang dikirim adalah row & row index
    ii. Contoh untuk styling alternate color row:

```
rowStyleFn:function(row, i){
  return {
    'background-color': i%2 ? '#ff0000' : '#ffffff'
  }
}
```

f. addBtn
    i. Bila show = true, maka tombol tersebut akan tampil di bagian kiri atas detail-grid.
    ii. Kalau tombol tersebut di-klik, akan memanggil fungsi newRow dengan object kosong.
    iii. icon bila di-set akan mengganti tombol add tersebut. Default: ‘add’.
g. deleteBtn
    i. Bila show = true, maka di detail grid akan muncul 1 kolom dengan nama kolom sesuai dengan title, dengan tombol delete.
    ii. icon: menentukan icon dari tombol. Default: ‘delete’.
    iii. key: adalah field value yang menentukan baris detail mana yang akan dihapus.
    iv. label: menentukan nama kolom untuk tombol delete tersebut. Apabila kolom sudah ada, tombol akan ditambahkan di kolom tersebut.
h. **watchKey**: adalah optional key yang akan dipantau oleh detail-grid. Bila key tersebut berubah, detail-grid otomatis akan loadData. Bila tidak di-assign, default watchKey-nya adalah ‘id’.
i. **dbIndex**: adalah optional key yang kalau di-set, akan menentukan database yang akan digunakan untuk query data.

6. Method:
    a. addColumn untuk menambah kolom secara run-time. Parameter-nya sama seperti elemen props.columns.
    b. newRow, lihat penjelasan di bagian cara penggunaan no: 2.
    c. loadData, lihat penjelasan di bagian cara penggunaan no: 3.
    d. assignData: adalah method untuk assign data langsung dari array.
        i. Bentuk object-nya harus: {data:[object of data]}
        ii. Untuk menghapus isi grid, kirim array kosong: `assignData({data:[]})`
        iii. Untuk assign model dalam jumlah yang banyak sekaligus:

```
this.grid.assignData({data:[
  {a:1, b:'y},
  {a:2, b:'x'}
]})
```

    e. setEditMode(true|false): berfungsi untuk mengubah state grid menjadi edit mode.
7. Runtime property:
    a. currentRow
        i. Merupakan Angular signal.
        ii. Untuk mengubah baris yang ter-highlight di detail-grid
        iii. Contoh:
            1. Untuk baca: grid.currentRow()
            2. Untuk tulis: grid.currentRow.set(3)
    b. currentCol
        i. Merupakan Angular signal.
        ii. Untuk mengubah kolom yang ter-highlight di detail-grid
        iii. Contoh penggunaan sama seperti currentRow
    c. isEditing

        i. Merupakan Angular signal.
        ii. Untuk mengubah edit mode dari cell tersebut

8. Event:
    a. **selectedRowChange**: adalah event yang akan terpicu saat baris berubah. Lihat contoh code di no: 7.
    b. **beforeEdit**: adalah event yang akan terpicu saat setiap cell masuk ke edit mode dengan cara menekan F2 atau double click di cell tertentu. Parameternya adalah row yang sedang di-edit. Parameter yang dikirim adalah object {row, col}
    c. **afterEdit**: adalah event yang akan terpicu saat setiap cell berhasil masuk ke edit mode. Event ini akan terjadi setelah beforeEdit dan setelah komponen edit-nya muncul. Parameter yang dikirim adalah komponen sesuai dengan **type** cell tersebut.
9. Bila model detail dengan key pertamanya truthy, maka data di model akan langsung ditampilkan di grid.
10. **editOnEnter** bisa diset jika dibutuhkan dan bernilai true. Jika true maka, pemilihan field di dalam detail grid bisa langsung diakses. Hal ini akan mempermudah penggunaan detail grid pada smartphone ataupun tablet.

Cara Penggunaan detail-grid di dalam template:

1. Dalam onInit, simpan variabel grid-nya dengan cara:
    - `this['grid'] = this.fields[index_field_grid]._componentRefs[0].instance.grid;`
    - Ganti **index_field_grid** dengan urutan grid tersebut di dalam **fields** (mulai dari 0).
    - Setelah itu, grid bisa diakses dengan cara: **this.grid**
    - Variable **grid** bisa diganti dengan apa saja.
2. Untuk menambah record baru:
    - this.grid.**newRow**(data, refresh);
    - **data** adalah object yang berisi record yang mau ditambahkan. Tidak harus sama dengan model detail. Hanya key-key yang cocok saja yang akan di-assign.
    - Parameter **refresh** adalah optional. Kalau tidak diisi, default = true. Fungsinya untuk menentukan apakah setelah record baru ditambah perlu refresh grid atau tidak.
3. Untuk mengisi data dari database:
    - this.grid.loadData(option)
    - Parameter option adalah optional.
    - Kalau option tidak diisi: detail-grid akan load langsung dari model detail sesuai setting.
    - Kalau mau load dari tabel lain, kirim option dalam bentuk: {block:**execute_block**}
    - Ganti **execute_block** dengan sql dalam bentuk **EXECUTE BLOCK**.
4. Untuk mengambil (dan menyimpan) data yang diisi ke server, gunakan:
    - let data = this.form.value
    - Di dalam data akan berisi detail model yang berupa array. Semua data detail-grid ada di dalam array tersebut.
5. Untuk menangkap event saat field di detail-grid berubah:
    - Gunakan event **valueChanges** dari FormControl-nya Angular.
    - Contoh: this.form.get('model_detail').valueChanges.subscribe();

6. Tabel detail harus mempunyai field **tableName_ID** di mana tableName adalah nama tabel master detail tersebut. Field ini harus Foreign Key ke ID table Master tersebut.
7. Untuk mengetahui saat selected row berubah, subscribe di event **selectedRowChange**.
    - contoh:

```
this.grid = this.fields[index_field_grid]._componentRefs[0].instance.grid;
this.grid.selectedRowChange.subscribe(row=>{
  console.log(row, 'current selected row');
}
```

8. Untuk menambah kolom baru:

```
this.grid.addColumn({
  key:'a',
  label:'abc'
})
```

9. Contoh penggunaan onCalc.

```
fields:[
  {
     type:'detail-grid',
     props:{
       columns:[
        {key:'a', label:'a'},{key:'b', label:'b'},{key:'c', label:'c'},
       ],
       onCalc:[
         {
           watchKey:['a', 'b'],
           function:function(rowCtls, value, key){
             rowCtls.get('c').setValue(
               rowCtls.get('a').value * rowCtls.get('b').value
             )
           }
         }
       ]
     }
   }
]
```

   Penjelasan:
   a. watchKey adalah key yang akan dipantau. Bila key-key tersebut nilainya berubah, maka `function` akan dipanggil.
   b. Function akan menerima parameter:
      i. rowCtls, yaitu formGroup dari baris detail-grid yang berubah.
      ii. value: nilai yang berubah
      iii. key: nama key yang berubah
   c. Contoh di atas akan menghitung c = a x b setiap kali nilai a atau b berubah.

d. Pastikan menggunakan addRow atau assignData saat akan menambahkan record baru.

10. Contoh menetapkan lebar kolom

```
fields:[
  {
    type:'detail-grid',
    props:{
      columns:[
        {key:'a', label:'a', style:{'flex':'0 0 30%'}},
        {key:'b', label:'b', style:{'flex':'0 0 10%'}},
        {key:'c', label:'c'},
      ],
    }
  }
]
```

one-record-label

Fungsi: sebagai komponen yang menampilkan data 1 record dalam bentuk tabel ke bawah.
Format:

```
{
      type:'one-record-label',
      props:{
            queryParam: {
                  filter:[]
                  Columns:[{data, title}, ...],
            },
            watchKey:string | Array,
            filters:[
                  {
                        index: number | string,
                        valueIndex: number,
                        value:string,
                        replaceKey:string,
                        watchKeyIndex:integer,
                  }
            ],
            dbIndex:number
      }
}
```

Penjelasan:

1. Penjelasan rinci untuk **queryParam** dapat dilihat di penjelasan param dari fungsi DataService.queryData.
2. **watchKey** menentukan key apa yang kalau berubah akan memicu loading data. **watchKey** juga bisa berupa Array bila ada lebih dari 1 key yang harus dipantau.
3. filters berfungsi menentukan aturan untuk mengganti nilai filter saat load data.
    a. **index**:
        i. menentukan index filter queryParam yang akan di-update.
        ii. Bila filter hanya 1 tingkat, gunakan number.
        iii. Bila filter mempunyai indent, gunakan string untuk menandakan index filter dengan titik sebagai pemisah. Misal: '1.0' berarti index kedua sub index pertama.
    b. **valueIndex**: diisi dengan 1 kalau akan mengubah parameter filter **value1** dan diisi dengan 2 kalau mau mengubah **value2**.
    c. **value**: parameter optional yang digunakan bila nilai yang akan diubah berupa sebuah expression. Misal sebuah select statement (select id from a where d_id=**_A_**).
    d. **replaceKey**: parameter optional yang perlu ada bila **value** diisi. Fungsinya untuk menentukan parameter yang akan diganti. Dalam contoh di atas, parameter-nya adalah '_A_'.

e. **watchKeyIndex**: parameter optional yang diisi bila watchKey berupa array. Isinya berupa index dari **watchKey** yang mau diganti. Isi 0 bila nilai watchKey pertama yang akan menggantikan filter tersebut.

4. dbIndex: untuk menentukan index database yang akan di-query

Contoh 1 watchKey dengan filter langsung dari nilai watchKey:

```
{
    type:'one-record-label',
    props:{
        queryParam:{
            start:0,
            length:1,
            order:[{column:0, dir:'asc'}],
            filter:[
                {
                    junction:'',
                    column:'CUTI_MOHON.ID',
                    operator:'='}],
           columns:[
               {data:'CUTI_MOHON.ID_KARYAWAN.ID.EMPLOYEES.NAMA_KARYAWAN',
                title: 'Nama'},
               {data:'CUTI_MOHON.TANGGAL_MOHON', title: 'Tanggal Mohon'},
               {data:'CUTI_MOHON.TANGGAL_MULAI_CUTI', title: 'Cuti Mulai'},
               {data:'CUTI_MOHON.TANGGAL_AKHIR_CUTI', title: 'Sampai'},
               {data:'CUTI_MOHON.JUMLAH_HARI_CUTI', title: 'Jumlah Cuti'},
               {data:'CUTI_MOHON.ALASAN_CUTI', title: 'Alasan Cuti'}
           ]
       },
       watchKey:'cuti_id',
       filters:[{ index:0, valueIndex:1 }],
   }
},
```

Contoh 1 watchKey dengan nilai filter ambil dari select statement.

```
{
  type:'one-record-label',
  props:{
    queryParam:{
      length:1,
      filter:[{ junction:'', column:'A.ID', operator:'='}],
      columns:[
```

```
        {data:'CUTI.TANGGAL', title:'Tanggal'},
        {data:'CUTI.KARYAWAN_ID.ID.KARYAWAN.NAMA', title:'Nama'}
      ]
    },
    watchKey:'id',
    filters:[
      {
        index:0,
        valueIndex:1,
        value:'=(SELECT ID FROM KARYAWAN WHERE KARYAWAN_ID=_A_)',
        replaceKey:'_A_',
    ]
  }
}
```

Contoh 2 watchKey dengan 2 filter:

```
{
  type:'one-record-label',
  props:{
    queryParam:{
      length:1,
      filter:[
        { junction:'', column:'A.ID', operator:'='}
        { junction:'and', column:'CUTI.TIPE_ID', operator='='}
      ],
      columns:[
        {data:'CUTI.TANGGAL', title:'Tanggal'},
        {data:'CUTI.KARYAWAN_ID.ID.KARYAWAN.NAMA', title:'Nama'}
      ]
    },
    watchKey:['karyawan_id', 'tipecuti_id'],
    filters:[
      {
        index:0,
        valueIndex:1,
        value:'=(SELECT ID FROM KARYAWAN WHERE KARYAWAN_ID=_A_)',
        replaceKey:'_A_',
        watchKeyIndex:0
      },
      {
        index:1,
        valueIndex:1,
        watchKeyIndex:1
      }
    ]
```

```
    }
}
```

read-only-table

Fungsi: Untuk menampilkan data multi record dalam bentuk tabel
Format:

```
{
      key:string,
      type:'read-only-table',
      props:{
            queryParam: {},
            watchKey:string | Array,
            filters:[
                  {
                        index: number,
                        valueIndex: number,
                        value:string,
                        replaceKey:string,
                        watchKeyIndex: number,
                  }
            ],
            dbIndex:number,
            nextControlId:string,
            prevControlId:string,
            onSelect:function(){}
      }
}
```

Penjelasan:

1. Cara setting queryParam, watchKey & filters di read-only-table sama dengan one-record-label.
2. Kolom-kolom di read-only-table bisa mempunyai key **type**. Cara penggunaannya sama seperti di detail-grid.
3. `key` hanya perlu diset apabila control ini digunakan untuk select record.
4. `nextControlId` adalah id dari element selanjutnya saat user menekan tombol Tab.
5. `prevControlId` adalah id dari element sebelum tabel saat user menekan tombol Shift+Tab.
6. Bila control read-only-table digunakan untuk memilih data, hasil yang dipilih akan muncul di dalam form.value dalam bentuk array.
7. `onSelect` adalah function yang akan dipanggil apabila detail grid di-klik.
8. Untuk memicu refresh, panggil perintah:
   ```
   this.fields[idxField]._componentRefs[0].instance.refresh();
   ```
   Ganti **idxField** dengan index yang sesuai.
9. Untuk load data dari array:
   ```
   let data = [{a:1, b:3}];
   this.fields[idxField]._componentRefs[0].instance.table.setData(data);
   ```

repeat

Fungsi: tipe ini digunakan untuk menampilkan kontrol yang sifatnya berulang.

Format:

```
{
      key:'bla',
      type:'repeat',
      props:{
            addText         :'',
            addBtn          : ButtonParams,
            deleteText      :'',
             deleteBtn      : ButtonParams,
            label           :'',
            description     :'',
            repeatClassName:string,
            backgroundColor:'',
            btnDeleteWidth :number (optional, default = 1),
            distance        :number (optional, default = 2),
            throttle        :number (optional, default = 500),
            onScroll        :[function(page)] (optional)
      },
      fieldArray:{
            fieldGroup:[
                  {object field yg akan diulang}
            ]
      }
}
```

Penjelasan:

1. key diisi dengan property model bertipe array yang akan menampung object dari record-record-nya.
2. props:
    a. **addText** adalah text yang akan ditampilkan di tombol untuk menambah record baru. Kalau diisi maka tombol untuk menambah record baru akan muncul.
    b. **addBtn**: ButtonParams adalah parameter optional. Bila diisi, efeknya sama seperti kalau isi addText. Value-nya sama dengan button.props.
    c. **deleteText** adalah text yang akan ditampilkan di tombol hapus per record. Kalau diisi maka setiap record akan muncul tombol hapus.

    d. **deleteBtn**:ButtonParams adalah parameter optional. Cara isinya sama dengan button.props kecuali event onClick. Bila **deleteBtn** sudah digunakan, property **deleteText** sudah tidak diperlukan lagi.
    e. **label** adalah tulisan yang akan muncul di setiap header record.
    f. **description** adalah tulisan yang akan muncul di bawah header record.
    g. ~~**backgroundColor** diisi dengan warna latar belakang dari panel yang akan di-repeat~~.
    h. ~~**btnDeleteWidth** diisi dengan lebar kolom untuk tombol Delete. Nilainya antara 1 - 12. Default 1. Apabila di-set, lebar tombol tidak otomatis membesar. Lebar tombol tergantung dari panjang **deleteText**.~~
    i. repeatClassName: string, property optional untuk set className dari div repeat. Beberapa class yang bisa digunakan, misalnya: Class background color dari bootstrap.
3. fieldArray adalah key wajib. Di dalamnya harus ada key fieldGroup di mana di dalam fieldGroup ini diisi dengan field-field yang akan ditampilkan secara berulang.
4. Komponen ini mempunyai kemampuan untuk melakukan infinite scroll. Fungsi ini akan bermanfaat bila data yang akan di-load banyak, maka bisa di-load secara bertahap. Setelah scroll bar mencapai titik tertentu, baru akan load data berikutnya lagi. Cara setting-nya adalah sebagai berikut:
    a. **distance**: diisi dengan berapa jarak maksimal untuk load data berikutnya. Bila diisi 1, maka saat scroll sudah mencapai 90% data, akan load ulang. Kalau 2 = 80%.
    b. **throttle**: diisi dengan berapa lama delay mouse scroll sebelum memicu load data ulang. Angka yang diisi dalam satuan mili detik.
    c. **onScroll** adalah fungsi yang akan dipakai untuk load data berikutnya.
5. Cara me-reset infinite scroll: `this.fields[x]._componentRefs[0].instance.reset();`
    a. Ganti x dengan index field repeat yang akan di-reset.
    b. Pastikan saat infinite scroll di-reset, array list data-nya juga di-reset.

Contoh infinite scroll untuk load item di FINA:

```
fields:[
  {
    key:'items',
    type:'repeat',
    props:{
      label:'Item:',
      onScroll:(page)=>{
        this.f().scroll(page)
      }
    },
    fieldArray:{
      fieldGroup:[
        {
```

```
          className:'col-2',
          key:'itemno',
          type:'input',
          props:{label:'Item No:'}
        },
        {
          className:'col-8',
          key:'itemdesc',
          type:'input',
          props:{label:'Description:'}
        }
      ]
    }
  },
],
functions:{
  onInit:()=>{
    this.model.items = [];
    this.f().addItem(0)
  },
  addItem:(start)=>{
    let param = {
      length : 20,
      start : start,
      columns:[
        {data:'ITEM.ITEMNO', key:'itemno'},
        {data:'ITEM.ITEMDESCRIPTION', key:'itemdesc'}
      ],
      dbIndex:1
    }
    this.dataService
      .queryData(param, this.config.api, true, true)
      .subscribe(data=>{
        this.model.items = this.model.items.concat(data)
        this.model = structuredClone(this.model)
      })
  },
  scroll:(page)=>{
    this.f().addItem(page-1)
  },
```

```
}
```

accordion

Fungsi:Tipe ini digunakan untuk membuka / menutup panel saat user klik.
Format:

```
{
  type:'accordion',
  props:{
    multi:true|false,
  },
  fieldGroup:[
    {
      props:{
        title:string,
        description:string,
      },
      fieldGroup:[
        {object field yg akan ditampilkan di accordion}
      ]
    }
  ]
}
```

Contoh menampilkan daftar item dalam bentuk accordion.

```
fields: [
  {
    key: 'items',
    type: 'repeat',
    props: {
      label: 'List Item',
      description: 'Daftar Item',
    },
    fieldArray:{
      fieldGroup: [
        {
          key:'item',
          type:'accordion',
          props:{ multi:false },
          fieldGroup:[
            {
              props:{
                title:'',
                description:'',
```

```
            },
            fieldGroup:[
              {key:'itemno'},
              {key:'itemdesc'},
              {
                key:'qty',
                type:'input',
                props:{
                  label:'qty'
                }
              }
            ]
          }
        ],
        expressions:{
          'fieldGroup[0].props.title' : 'field.formControl.value.itemno',
          'fieldGroup[0].props.description' :'field.formControl.value.itemdesc',
        }
      },
    ]
  }
}
```

Bentuk model-nya adalah sebagai berikut:

```
this.model.items = [
  {
    item:{
      itemno:’a’,
      itemdesc:’item a’,
      qty:1,
    },
  },
  {
    item:{
      itemno:’b’,
      itemdesc:’item b’,
      Qty:2,
    }
  }
]
```

drag-drop-list

Fungsi: Untuk menampilkan list yang isinya bisa di-drag dan drop.
Format:

```
{
      key: 'bla',
      type: 'drag-drop-list',
      props:{
            listClassName:'col-3',
            displayKey:string
      }
}
```

Cara pakai:

1. Property **listClassName** digunakan untuk menentukan lebar kolom. Bisa diisi dengan **col-1** sampai dengan **col-12**.
2. Untuk update isi list dilakukan dengan cara update this.model.key dengan format:

```
[
      {
            id:'xx',
            caption:'xxx',
            data:[string|object],
      }
]
```

Di mana:

1. id adalah identifier dari list tersebut.
2. caption adalah label yang ditampilkan di atas masing-masing list.
3. data adalah isi di masing-masing list. Bisa berupa string maupun object.
4. Bila data berupa object, maka property **displayKey** wajib diisi. displayKey adalah key object yang akan ditampilkan ke list.

Contoh code

```
this.model = {
      bla:[
            { id:'1', caption:'A', data:['A1', 'A2', 'A3'] },
           { id:'2', caption:'B', data:['B1', 'B2', 'B3'] }
          ]
}
```

Key **bla** adalah key yang di set di fields.

label

Fungsi: Menampilkan nilai dari sebuah property model dalam bentuk label.
Format:

```
{
      key: 'bla',
      type: 'label',
      props:{
            type?:string,
            arg1?:string,
            arg2?:string,
            onClick?:function($event,id){}
            badge?:BadgeParam,
      }
}
```

Penjelasan:

1. Label berfungsi menampilkan nilai dari sebuah key
2. Bila ingin mem-format nilai yang ditampilkan, gunakan optional props: **type, arg1, arg2.** Cara isinya sama seperti di detail-grid.
3. Event onClick akan dipicu saat label tersebut diklik oleh user. Parameter yang akan dikirim adalah:
    a. $event : yang berisi element dari label yang diklik
    b. Id : yaitu ID dari label tersebut
4. Label juga bisa menampilkan optional badge.

img

Fungsi: Menampilkan gambar.
Format:

```
{
      key: 'bla',
      type: 'img',
      props:{
            url:string,
            width:number,
            height:number,
            dbIndex:number,
            table:string,
            imgCol:string,
            keyCol:string,
            formatCol:string (optional),
            imgClassName:string (optional),
            attributes?:{
                  loading:'lazy'(default)|'eager',

            }
      }
}
```

Keterangan:

1. Width & height adalah lebar & tinggi dari gambar tersebut.
2. Key:
    a. Kosongkan bila url-nya berupa data base64 atau hard code url
    b. Isi dengan nama field yang menyimpan url
3. Bila akan set url secara dinamis: `this.fields[index_field].props.url = 'https://xyz.com/img.jpg'` di mana index_field adalah index dari control img tersebut.
4. Apabila image disimpan di dalam url tertentu, simpan url tersebut dalam kolom tertentu lalu isi key dengan nama field yang menyimpan url tersebut.
5. Apabila image akan load dari database, properti yang harus diisi adalah:
    a. dbIndex
    b. table
    c. imgCol
    d. keyCol
    e. formatCol. Bila tidak ada field yang menyimpan format image, asumsinya format image-nya adalah jpeg.
6. Property imgClassName digunakan untuk set class dari img. Misal: img-fluid-nya bootstrap.

Contoh setting untuk load logo dari tabel Item di FINA:

```
{
  key:’itemno’,
  type:’img’,
  props:{
    width:100,
    height:100,
    dbIndex:’1’,
    table:’item’,
    keyCol:’itemno’,
    imgCol:’logo’,
    formatCol:’format_logo’
  }
}
```

horizontal-stepper

Fungsi: Untuk menampilkan control dalam bentuk multi layar dengan tombol next & previous di bawahnya.

Format:

```
{
  type:'horizontal-stepper',
  props:{
    prevBtnCaption:string,
    nextBtnCaption:string,
    lastBtnCaption:string,
    lastBtnClick:function($event){},
    nextBtnClick:function($event){},
    prevBtnClick:function($event){}
  },
  fieldGroup:[
    {
      props:{
        label:string //caption untuk halaman 1
      },
      fieldGroup:[
        {type:'input',..},
        …
      ]
    },
    {
      props:{label:string}, //label untuk halaman 2
      fieldGroup:[
        //isi dengan field untuk halaman 2
      ]
    },
    … dst-nya
  ]
}
```

Keterangan:

1. **prevBtnCaption** diisi dengan caption tombol untuk kembali ke halaman sebelumnya. Tombol hanya muncul kalau berada di page ke-2 dan seterusnya dan key ini ada nilainya.
2. **nextBtnCaption** diisi dengan caption tombol untuk maju ke halaman berikutnya. Tombol ini hanya akan menyala apabila semua field required di halaman tersebut sudah terisi dan key ini ada nilainya.
3. **lastBtnCaption** diisi dengan caption tombol di halaman terakhir. Tombol ini akan menyala bila berada di halaman terakhir dan key ini ada nilainya.
4. **lastBtnClick** diisi dengan function yang akan dipanggil saat tombol terakhir ditekan.

5. **nextBtnClick** diisi dengan function yang akan dipanggil saat tombol next ditekan.
6. **prevBtnClick** diisi dengan function yang akan dipanggil saat tombol previous ditekan.

chart

Fungsi: Untuk menampilkan chart.
Format:

```
{
  type:'chart',
  props:{
    styles:{
        width:'400px'
    },
    config:{
        type:'bar'|'line'|etc,
        data:{
            labels:['a','b','c'],
            datasets:[{
                label:'abjad',
                data:[1,2,3]
            }]
        }
    }

  }
}
```

Penjelasan:

1. styles berfungsi untuk mengirim styling ke object tersebut. Format bisa dalam bentuk string maupun object. Key-nya adalah nama style dari css.
2. Config adalah format konfigurasi grafik yang digunakan. Library yang digunakan adalah chart.js. Ada banyak konfigurasi yang bisa dibuat. Untuk informasi yang lebih rinci, silakan lihat langsung ke website resmi chart.js.
3. Bila ingin update data secara dinamis, lakukan hal berikut ini:
    a. Di onInit:
    ```
    this.config = {} //isi config awal
    this.fields[x].props.config = {...this.config} //sesuaikan x
    ```

    b. Di event untuk update data:
    ```
    //update this.config
    this.fields[x].props.config = {...config}
    ```

    c. Pastikan menggunakan spread operator saat assign config.

text

Fungsi: untuk menampilkan text.
Alternatif lain untuk menambahkan html di template adalah sebagai berikut:
{template:' bla-bla-bla '}

Namun cara di atas tidak bisa di-set style-nya.
Dengan menggunakan text, format-nya bisa diset sebagai berikut:

```
{
  type:'text',
  props:{
    text:string,
    attributes:{
      style:string,
    }
  }
}
```

selection-list

Fungsi: Menampilkan list dengan check box. Menggunakan komponen ini.
Format:

```
{
  key:’’,
  type: ‘selection-list’,
  props: {
    multiple: boolean,
    togglePosition: 'before'|’after’,
    listOptions: [{label, value}]
  }
}
```

Penjelasan:

1. multiple:boolean. Bila true berarti bisa memilih lebih dari 1.
2. togglePosition:menentukan posisi checkbox. Default:’after’.
3. listOptions: merupakan isi dari list.
    a. Bentuknya array dari object
    b. Isi object:
        i. label: nilai yang akan ditampilkan di list
        ii. value: nilai yang akan di-assign ke key bila list terpilih. Value bisa berupa string atau object apa saja.
4. Item list yang terpilih akan masuk sebagai value-nya key.

swiper

Fungsi: Untuk menampilkan sejumlah gambar/video secara bergantian.

Format:

```
{
  type:'swiper',
  props:{
    images:[
      {img:string},
      {iframe:string},
    ],
    params:{
      navigation:boolean,
      pagination:boolean,
      scrollbar:boolean,
    }
  }
}
```

Penjelasan:

1. Images adalah array of object yang isinya bisa:
    a. {img} di mana img diisi dengan url dari gambar yang ingin ditampilkan
    b. {iframe} di mana iframe diisi dengan url iframe youtube yang akan ditampilkan.
        i. Pastikan url youtube ditambahkan query param `enablejsapi=1`
        ii. Contoh:

```
{iframe:'https://www.youtube.com/embed/bla?si=OjWS&enablejsapi=1'}
```

2. Params adalah parameter swiper
3. Cara menampilkan gambar sebelum / sesudahnya bisa menggunakan swipe atau navigator (bila dinyalakan).
4. Khusus untuk youtube video, harus menggunakan navigator karena swipe di dalam video tidak berfungsi.
5. Default lebar adalah 100% dan height-nya memenuhi aspect rasio 16:9. Bila ingin diubah, silakan ubah style-nya.

report

Fungsi: Untuk menampilkan report dengan konsep band.

Format:

```
{
  type:'report',
  props:{
    structure : ReportObject
  }
}
```

Penjelasan:

1. Requirement rinci mengenai cara membuat struktur report, silakan ikuti link ReportObject di atas.

Validator

Validator berfungsi untuk melakukan pengecekan terhadap nilai yang diinput di dalam komponen. Ada 3 jenis validator yang bisa digunakan.

Built-in Validator

Built-in Validator yang sudah tersedia adalah:

1. required: boolean, akan invalid saat tidak diisi.
2. min: number, untuk validasi nilai minimal
3. max: number, untuk validasi nilai maksimal
4. minLength: number, untuk validasi panjang string minimal
5. maxLength: number, untuk validasi panjang string maksimal
6. pattern: regular expression yang harus diikuti.

Semua built-in tersebut, bila ingin digunakan, bisa diletakan di dalam props.
Contoh:

```
{
  key:’nama’,
  type:’input’,
  props:{
    label:’Nama Karyawan’,
    minLength:3, //panjang minimal 3 karakter
    maxLength:20, //panjang maksimal 20 karakter
  }
},
{
  key:’usia’,
  type:’input’,
  props:{
    label:’usia karyawan',
    min:17, //usia minimal
    max:99, //usia maksimal
  }
}
```

Custom Validator (Synchronous)

Cara kedua untuk memasang validator adalah dengan membuat sendiri fungsi yang akan mengecek keabsahan nilai yang diisi.
Format pembuatan custom validator adalah sebagai berikut:

```
{
  key:'ip',
  type:'input',
  props:{
    label:'IP Komputer',
    required:true
  },
  validators:{
    cekip:{
      expression:function(control){
        return boolean_value;
      },
      message:function(error, field){
        return `"${field.formControl.value} bukan merupakan format alamat
IP`
      }
    }
  }
}
```

Cara penggunaan:

1. Object **validators** harus mempunyai minimal 1 key. Key ini merupakan nama validator. Boleh bebas apa saja. Bisa lebih dari 1
2. Isi object masing-masing validator harus mempunyai key:
    a. expression
        i. Value-nya adalah sebuah function expression dengan parameter control yang akan dicek
        ii. Return dari function expression ini harus berupa boolean. True berarti lolos validasi
    b. message
        i. Value-nya adalah sebuah string atau function expression dengan parameter error & field
        ii. Return dari function expression ini harus berupa string
    c. Kedua function expression tersebut dapat memanggil fungsi yang ada di functions.

Custom Validator (Asynchronous)

Ada kalanya pengecekan nilai perlu melakukan proses yang cukup panjang, misalnya: query data ke FINA API.
Dalam kasus tersebut, validasi bisa dilakukan secara asynchronous agar tampilan Biz-A tetap lancar (tidak membeku).

Formatnya adalah sebagai berikut:

```
{
  key:'nama',
  type:'input',
  props:{
    label:'Nama Karyawan',
    required:true
  },
  asyncValidators:{
    cekNama:{
      expression:function(control){
        return promise | Observable
      },
      message:function(error, field){
        return `"${field.formControl.value} bukan merupakan format alamat
IP`
      }
    }
  }
}
```

Penjelasannya sama seperti validators di atas. Kecuali kembalian dari expression harus berupa Promise atau Observable.

Contoh berikut untuk mengecek apakah user sudah ada atau tidak.

```
{
  key:'user',
  type:'input',
  props:{ label:'Nama User'},
  modelOptions:{ updateOn:'blur'},
  asyncValidators:{
    cekUnik: {
      expression: function(c){
        if (!c.value) return this.of(true)
        let param = {
          filter:[{junction:'', column:'USERS.USERNAME', operator:'=',
                  value1:`'${c.value.toUpperCase()}'`}],
```

```
          columns:[{data:'USERS.USERID', key:'userid'}],
          dbIndex:1
        }
        return this.dataService.queryData(param, this.config.api, true,
true).pipe(
          this.map(data=>data.length == 0)
        )
      },
      message: function(error, field){
        return `User "${field.formControl.value}" sudah ada`
      },
    },
  }
},
```

Built-in Custom Validator

Dalam Biz-A sudah tersedia built-in Custom Validator untuk email.
Cara menggunakannya adalah:

```
{
  key:'email',
  type:'input',
  props:{
    type:'email',
    label:'Alamat Email',
  },
  validators:{
    validation:['email']
  }
}
```

Variable

Berikut ini adalah daftar variabel yang ada di Form. Semua variabel dapat diakses dengan `this`.

1. model
2. form
3. fields
4. id : berisi query parameter yang dikirim /form/namaform/:id
5. apiConfig : sama dengan config.api
6. appConfig : sama dengan config.app
7. data : untuk menampung data global.
8. debug : untuk menampilkan debug windows di bagian bawah form.
9. numParser : adalah NumberParser dengan locale id.
10. NumberParser : adalah library parser. Fungsi untuk parsing angka. Lihat cara penggunaannya di sini.
11. XLSX : adalah library XLSX.
12. dayjs : adalah library dayjs yang sudah di-extend dengan
    a. customParseFormat
    b. weekOfYear.
    c. utc
    d. timezone
13. signal: adalah fungsi signal dari Angular.
14. io: merupakan socket.io client.

Html

Untuk menampilkan data html, gunakan key template.
Contoh:

```
fields:[
  {
    template:`
      <p>Hello world</p>
    `
  }
]
```

HTML di template akan di-sanitize. Jadi kalau ada html yang sifatnya script/embedded, dsb-nya akan dibuang.

Format Template List

Semua list yang dibuat harus mempunyai 1 file template dengan ekstensi **js** di dalam folder yang sama..
Isi file ini harus mengikuti format sebagai berikut:

```
get = function(){
      return {
            caption:string=’’,
            columns:[],
            filter:[],
            functions:{},
            dbIndex:number,
            distinct:true|false,
            buttons:[]
      }
}
```

caption

Berfungsi untuk menampilkan caption di sisi kiri atas list. Default = kosong.

columns

Digunakan untuk List.
Formatnya adalah sebagai berikut:

```
columns:[
      {
            “title”:”Caption kolom Header”,
            “data”:”NAMA_TABEL.NAMA_FIELD”,
            “type”:”string | date | money | boolean”,
            “style”:object,
      },
      {
            “title”:”Caption 2”,
            “data”:”NAMA_TBL.ID_FK.LOOKUP_ID.LOOKUP_TBL.LOOKUP_FIELD”
      },
      {
            “title”:””,
            “data”:”NAM_TBL.ID”,
            “button”:[
                  {
                        “caption”:”xxx”,
                        “icon”:”home”,
                        “color”:“primary”|”warn”|”accent”,
                        “link”:[“../../main”],
                        “onClick”:”[namaFungsiEventHandler]”
```

```
                    }
                ]
            }
]
```

Penjelasan

1. **title** adalah kata-kata yang akan muncul di header table.
2. **data** adalah field yang akan digunakan untuk query ke database. Cara setting-nya sama seperti di **queryData**. Bila ada 2 kolom dengan data yang sama, pastikan title-nya berbeda biar tidak bentrok.
3. **type** sama dengan **type** di detail-grid.
4. **style** cara pakainya sama seperti di detail-grid.
    a. Berbeda dengan detail-grid, untuk menetapkan lebar kolom, gunakan {width:'xx%'}.
5. Khusus untuk field yang lookup ke database lain selain database Biz-A, gunakan tipe lookup-combo. Cara setting-nya lihat di penjelasan detail-grid**.**
6. Untuk kolom button:
    a. Kosongkan title. Title yang kosong juga sekaligus akan menghilangkan kemampuan sorting (perurutan) di kolom tersebut.
    b. Isi data dengan primary key dari tabel tersebut. Informasi ini akan digunakan untuk edit atau delete.
    c. Isi icon dengan gambar yang sesuai. Daftar lengkap nama icon yang bisa digunakan dapat dilihat di sini.
    d. color untuk menentukan theme. Bisa memilih 'primary' (default), 'warn' atau 'accent'.
    e. Link diisi dengan nama form yang akan dipanggil saat tombol ditekan.
    f. Alternatif lain dari **link** adalah **onClick**, di mana value-nya bisa diisi dengan:
        i. Nama fungsi yang terdapat di file functions.js.
        ii. Atau `function(){}` (harus 3 baris)

filter

Key ini hanya digunakan untuk list.
Bila diisi, akan menjadi default filter dari list saat dibuka.
Format array ini sama seperti filter di fungsi **queryData**.

Untuk value1 & value2, bisa diisi dengan variabel.
Contoh value yang isinya akan diganti dengan variable adalah:
value1:"${userId}"

Atau bisa juga dalam bentuk select query:
value1: "(select id from table1 where key=${userId}"

Variabel-variabel yang bisa digunakan adalah sebagai berikut:

1. userId

distinct

Digunakan bila memerlukan distinct di list.
Default value: false.

Cara penggunaannya:

```
columns:[...],
distinct:true,
```

functions

Penjelasannya sama seperti functions di Form.
Fungsi-fungsi default yang tersedia di list adalah:

1. useLibrary
2. beforeLoadData
3. afterLoadData
4. onPipe

buttons

Key ini berfungsi untuk menampilkan tombol di bagian atas list.
Formatnya sama dengan format button di columns.

dbIndex

Berfungsi untuk menentukan indeks database yang digunakan. Apabila tidak digunakan, Biz-A akan menggunakan dbIndex default.

badge

Badge adalah icon kecil yang muncul di sudut sebuah komponen seperti gambar di bawah ini:



Badge dapat muncul di komponen-komponen di bawah ini:

1. Button
2. Label

Cara setting-nya BadgeParam:

```
props:{
  badge:{
    content : string|number|null|undefined,
    color?  : 'primary'|'accent'|'warn',
    disabled? : boolean,
    hidden?   : boolean,
    overlap?  : boolean,
    position? : 'above before' | 'above after' | 'below before' | 'below after',
    size?     : 'small' | 'medium' | 'large'
  }
}
```

Penjelasan:

1. Tambahkan setting di atas di masing-masing props komponen yang akan ditambahkan badge.
2. Content adalah isi dari badge. Badge hanya muncul bila content ada isi.
3. Color: warna dari badge. Default: kosong.
4. Disabled: bila true, badge akan tampil dengan warna abu-abu. Default: false.
5. Hidden: bila true, badge akan hilang walau ada content. Default false.
6. Overlap: bila true, badge akan sedikit menimpa sudut komponen. Default true.
7. Position:
    a. Above before = kiri atas
    b. Above after = kanan atas (default)
    c. Below before = kiri bawah
    d. Below after = kanan bawah
8. Size: kecil, menengah atau besar. Default: medium.

Bila mau update badge secara runtime:

```
this.fields[x].props.badge = {content:'2'};
this.fields[x].props = structuredClone(this.fields[x].props);
```

Main Form template

Main Form (halaman utama) bisa ditambahkan template.
Caranya adalah:

- Buat file **main.js,** buat kode yang diinginkan dan simpan pada masing-masing folder pada template biz-A.
- Template main.js bersifat optional:
    - Jika ada file main.js, maka template di dalamnya akan dijalankan.
    - Jika tidak ada file main.js, maka tidak akan muncul error "Template Not Found." alias berjalan normal.
    - Tidak boleh ada key “subMenu” untuk menggunakan fitur ini.

Bab 5. Functions

Cara Memanggil Fungsi

Cara memanggil fungsi ada 3 cara:

1. this.doFunction(‘nama_fungsi’, parameter 1, … parameter n) (hanya untuk backward compatible. Jangan digunakan untuk program baru)
2. this.functions.nama_fungsi(parameter-1, … parameter-n)
3. this.f().nama_fungsi(parameter-1, … parameter-n)

Ketiga cara tersebut akan menghasilkan efek yang sama.
Perbedaannya adalah, untuk cara 1, **this** akan mengacu ke form / list pemanggil.
Sedangkan cara ke-2 & 3, **this**-nya undefined.
Bila ingin menggunakan cara ke-3 dan memerlukan **this**, silakan kirim **this** sebagai salah satu parameter dari fungsi tersebut.

Fungsi-fungsi Reserved yang bisa digunakan:

1. onInit(): fungsi ini akan dipanggil segera setelah form berhasil ditampilkan.
2. useLibrary(): function ini digunakan untuk memuat library external.
3. beforeLoadData(): fungsi ini akan dipanggil sebelum form atau list melakukan loading data.
4. afterLoadData(): fungsi ini akan dipanggil setelah form atau list melakukan loading data.
5. onPipe(): fungsi ini akan dipanggil oleh list setelah berhasil loading data.

onInit

onInit adalah fungsi reserved yang, kalau disediakan, akan dipanggil oleh form saat pertama kali di-load.

Parameter yang dikirim adalah id dari form tersebut dengan tipe string. Bila dalam mode insert, nilai id adalah ‘null’ (string). Bila mau dicek harus menggunakan !! (2 tanda seru)

```
onInit: function(id:string){
      //code yang akan dijalankan saat inisialisasi form
      if (!!id){
            //lakukan sesuatu bila id = ‘null’
      }
}
```

Beberapa hal yang bisa dilakukan di fungsi onInit, antara lain:

1. Mengaktifkan debug window
   `this.debug = true;`
   Parameter `this.debug` nilai default-nya adalah false.

2. Mengisi Atribut Tag HTML Khusus Secara runtime
3. Mengisi initial value dalam mode insert.

onDestroy

onDestroy adalah fungsi reserved yang, kalau disediakan, akan dipanggil oleh form pada saat user akan loncat ke form yang lain.
Gunakan form ini apabila ada sesuatu yang harus di-destroy atau dibersihkan. Misal: timer.

Contoh:

```
onDestroy: function(event){
  this.timer.stop();
}
```

useLibrary

Apabila ada fungsi-fungsi yang akan digunakan di lebih dari 1 template, agar code tersebut bisa digunakan ulang, code tersebut perlu ditarik ke sebuah library external.

File library external formatnya sama dengan file template. Tapi hanya property **functions**-nya yang diisi. Untuk penyeragaman penamaan, semua file external library namanya selalu dimulai dengan **lib_**. Misal: **lib_pph.js**.

Untuk menggunakan fungsi-fungsi yang ada di external library tersebut, di template tersebut harus memuati dulu external library tersebut dengan cara menambahkan nama library tersebut di fungsi useLibrary.

Contoh:

```
functions:{
      useLibrary: function(){
            return ['lib_pph', 'lib_abc'];
      }
}
```

Bila ada lebih dari 1 library yang akan dimuati, tinggal disebutkan nama file-nya (tanpa js) dengan dipisahkan tanda koma.

Setelah menambahkan fungsi useLibrary, fungsi-fungsi di dalam template tersebut sudah bisa menggunakan fungsi yang ada di library yang dimuati dengan cara yang sama seperti memanggil fungsi di template yang sama. Untuk lebih jelas, lihat **Cara memanggil fungsi lain**.

Hati-hati dalam menggunakan fungsi yang ada di library, khususnya di onInit karena bisa jadi saat onInit dijalankan, library belum tuntas di-load. Bila hal ini yang terjadi, silakan keluarkan fungsi dari library dan pindahkan ke onInit.

beforeLoadData

Dalam kondisi-kondisi tertentu, kita perlu melakukan sesuatu sebelum Biz-A load data. Misalnya: setting dbIndex. Untuk itu, kita bisa lakukan di dalam fungsi beforeLoadData.
Khusus untuk form, beforeLoadData akan mengirim parameter id.

Hati-hati untuk fungsi yang terdapat di external library karena bisa jadi saat tiba di beforeLoadData, library belum ter-load dengan baik.

afterLoadData

Fungsi ini akan dipanggil sesaat setelah Biz-A berhasil load data. Khusus untuk form, ada parameter id.
Hati-hati untuk fungsi yang terdapat di external library karena bisa jadi saat tiba di beforeLoadData, library belum ter-load dengan baik.

onPipe

Fungsi ini akan dipanggil oleh List Biz-A.
Apabila data hasil query ingin diolah lebih lanjut, misalnya: memendekan panjang data, menghitung calculated field, dan sebagainya.
Return-nya harus berupa observable.

Cara penggunaannya:
Tambahkan fungsi seperti di bawah:

```
onPipe: function(...data){
  return this.of(data.map( d=>({
    ...d,
    name : d.name.length>20 ? d.name.slice(0, 20).concat('...') : d.name
  })))
}
```

Code di atas berfungsi untuk memangkas kolom **name** menjadi maksimal 20 karakter dan ditambah dengan titik 3.

Cara Memanggil Fungsi yang Lain

Sebuah fungsi dapat memanggil fungsi yang lain baik fungsi yang berada di template yang sama maupun fungsi dari external library.

Cara memanggil fungsi yang lain adalah: **this.f().nama_fungsi(…parameter)**
Contoh:

```
1 functions:{
2    hitungA: function(b){
3         return b*2;
4    },
5    hitungC: function(d){
6         return this.f().hitungA(d);
7    }
8 }
```

Penjelasan:
Baris 2-4 adalah fungsi yang akan dipanggil.
Baris 6 adalah contoh fungsi **hitungC** yang sedang memanggil fungsi **hitungA**. Apabila parameter fungsi ada lebih dari 1, tinggal ditambahkan saja di sebelah kanan dan dipisahkan dengan koma.

Event Button Click

Dalam function.js juga bisa mendefinisikan fungsi-fungsi yang bisa dipanggil saat button click.

Formatnya:

```
btnSendClicked: function (){
     console.log('button send sudah di-klik');
}
```

Supaya fungsi btnSendClicked bisa terpanggil, maka kita harus sambungkan tombol btnSend dengan event ini, dengan cara menambahkan property **onClick** pada fieldType di template.js.
Contoh:

```
{
	type: "button",
	props: {
		text: "Kirim",
		btnType: "submit",
		onClick: function($event){
			this.f().btnSendClicked();
		}
	}
}
```

Penjelasan:

1. Deklarasi di atas adalah salah satu contoh definisi field di template.js dengan tipe button.
2. Baris **onClick: function($event){** yang ditutup dengan **}** merupakan code untuk event handler saat button tersebut di-klik.
3. Di antara {} bisa diisi dengan code apa pun. Dalam hal ini, kita memanggil fungsi **this.f()** yang berfungsi untuk memanggil fungsi yang ada di file function.js.
4. Parameter pertama dan seterusnya adalah optional apabila mau mengirim parameter yang lain ke fungsi.

Cara Mencetak

Contoh kode:

```
get = function(){
	return {
		fields:[
			{
				type:'button',
				props:{
					text:'Cetak',
					onClick:function(){
						this.f().onPrint();
					}
				}
			}
		],
		functions:{
			onPrint:function(){
				this.helper.print('<p>Hello World</p>');
			},
			onAfterPrint:function(){
				console.log('after print');
			}
	}
```

```
}
```

Event

Event merupakan kejadian yang terjadi karena dipicu oleh kejadian tertentu.
Event diperlukan di script template agar kita bisa melakukan sesuatu atas terjadinya kejadian tertentu.
Misal: Pada saat field A berubah, kita mau nilai field B dihitung ulang.
Tanpa event, untuk melakukan contoh di atas kita perlu secara berkala pakai timer untuk terus mengecek nilai Field A dan dibandingkan dengan nilai yang kita simpan. Kalau terjadi perbedaan, berarti nilai Field A telah berubah sehingga kita perlu melakukan sesuatu.

Cara menggunakan Event

Setiap field di model mempunyai event yang bernama valueChanges.

Contoh: Bila model mempunyai key: id dan saat id berubah, kita akan melakukan sesuatu, maka kode-nya sebagai berikut:

```
this.form.get('id')?.
ges.subscribe( value =>{
      console.log( value );
}
```

Value adalah nilai yang berubah.

Cara Menampilkan Custom Form Dialog

Dialog adalah sebuah form yang ditampilkan secara modal (user tidak bisa pindah ke layar yang lain sebelum form tersebut ditutup).
Contoh form dialog yang bisa digunakan adalah perintah **this.helper.print** seperti yang dijelaskan di atas. Hanya saja, form print tersebut hanya dibuat khusus untuk mencetak, tidak bisa digunakan untuk keperluan yang lain.

Custom Form Dialog ini selalu akan ada 2 tombol di bagian bawahnya, yaitu tombol OK & Cancel. Label-nya bisa diubah-ubah.

Cara membuat custom form dialog adalah sebagai berikut:

1. Buat sebuah config dengan tipe MatDialogConfig. Parameter apa saja yang bisa diisi di config ini bisa dilihat di sini.
2. Config ini perlu ada key **data**. Format data ini harus berupa object:
    a. caption: akan ditampilkan di bagian atas form
    b. captionOK: akan menggantikan caption tombol OK

    c. captionCancel: akan menggantikan caption tombol Cancel
    d. model: diisi dengan {}
    e. fields: diisi dengan array control apa saja yang akan ditampilkan di dalam custom form dialog. Cara pakainya sama seperti fields di form template.
3. Variable config dikirim ke **this.dialog.open** dengan 2 parameter:
    a. this.FormlyDialogComponent
    b. config
4. Tampung referensi no:3 di sebuah variable agar bisa assign observable **afterClosed**.
5. Observable **afterClosed** akan mengirim 1 parameter data yang berisi nilai dari model form dialog tersebut bila user klik tombol OK.

Berikut ini contoh code cara membuat custom form dialog:

```
const config = new this.MatDialogConfig();
config.width = `${window.innerWidth}px`;
config.height = `${window.innerHeight}px`;
config.data = {
  caption:'xxx',
  captionOK:'OK',
  captionCancel:'Batal',
  model:{},//bila ada default value, set di sini
  fields:[
    //isi dengan control yg mau ditampilkan di dialog
  ]
}
const ref = this.dialog.open(this.FormlyDialogComponent, config);
ref.afterClosed().subscribe((data)=>{
      //data berisi model dari form dialog
})
```

Cara Ekspor Data ke File Excel

```
1 const wb = this.XLSX.utils.book_new();
2 const ws = this.XLSX.utils.aoa_to_sheet(
3     [['A', 'B', 'C'],['A1','B1','C1'],['A2','B2','C2']]
4 );
5 XLSX.utils.book_append_sheet(wb, ws, "sheet1");
6 XLSX.writeFile(wb, `${this.listName}.xlsx`);
```

Penjelasan:

1. Dalam list & form terdapat library this.XLSX yang bisa digunakan untuk export-import ke dan dari file excel.
2. Baris 1 untuk create object workbook.
3. Baris 2 untuk create object worksheet. Fungsi aoa_to_sheet mengharapkan parameter array of array.
4. Baris 3 adalah contoh array of array yang dikirim. Array baris pertama akan dijadikan sebagai caption.
5. Baris 5 untuk membuat sheet baru dengan nama “sheet1”.
6. Baris 6 untuk simpan workbook tersebut ke dalam file.

Untuk import dari file excel, ganti baris 2 menjadi this.XLSX.utils.table_to_sheet(html_table);

Cara Format Angka Menjadi Currency

Fungsi: Untuk format angka sehingga mengandung pemisah ribuan dan desimal.
Format:
const parser = new this.NumberParser(locale:string, currency:string=’IDR’, isCurrency:boolean=true);

Penjelasan:

1. locale: diisi dengan nama negara. Misal untuk Indonesia adalah **id**.
2. currency: diisi dengan nama mata uang yang akan digunakan. Default: **IDR**.
3. isCurrency diisi false bila hanya ingin menampilkan angka dengan pemisah ribuan/desimal tanpa simbol mata uang. Default: true.
4. Untuk menggunakan parser, tinggal panggil fungsi **format** dengan parameter angka yang akan di-format.

```
const parser = new this.NumberParser('id');
console.log(`${parser.format(1000}`);
```

Maka akan muncul **Rp. 1.000,00**

Bab 6. Daftar Service yang Tersedia

dataService

Method sendModel

Fungsi: Untuk mengirim model ke API agar disimpan di database.
Parameter:

1. model:any, yaitu model yang akan disimpan ke database. Ambil dari **this.model**.

2. apiConfig: IApiConfig, ambil dari variable **this.config.api**.
3. tableName:string, yaitu nama tabel yang akan diisi.
4. useSpinner:boolean = true. Bila false tidak akan muncul spinner saat dijalankan.
5. raiseException:boolean = true. Bila false, kalau error akan tidak akan raise exception.

Return:

1. Kalau sukses: {success:'OK'}
2. Kalau gagal: {error:[error message-nya]} dan akan muncul exception

Penjelasan:

1. model bisa berupa object 1 tingkat atau pun master-detail.
2. Model bisa dalam bentuk array. Dalam hal ini, semua record akan disimpan sekaligus. Bila ada yang error, semuanya akan di-roll back.
3. Bila id = null akan insert record baru. Bila tidak null, akan edit berdasarkan id tersebut.
4. Untuk model dengan bentuk Master-Detail mode edit, detail yang lama akan dihapus semua dan detail yang baru akan di-insert semua.

Contoh code:

```
1   this.dataService.sendModel(this.model, this.config.api, 'permohonan_cuti')
2   .subscribe( data=>{
3         alert(`success`);
4         this.router.navigate(['../list', 'daftarcuti']);
      })
```

Baris 1 adalah cara memanggil sendModel. Salin saja 1 baris dan hanya perlu ganti parameter terakhir dengan nama tabel.
Baris 2 bertujuan untuk menunggu data respon dari API. Salin habis.
Baris 3 bertujuan untuk menampilkan alert di browser. Baris ini akan dijalankan setelah terima data tanpa exception.
Baris 4 bertujuan untuk mengalihkan ke mana setelah save berhasil. Cara mengisi routing-nya sama persis dengan yang di menu.json.

Method deleteModel

Tujuan: Untuk menghapus 1 record tertentu
Parameter:

1. id: number, yaitu berupa id dari record yang akan dihapus.
2. apiConfig: IApiConfig, ambil dari variable: **this.config.api**.
3. tablename: string, yaitu nama tabel yang record-nya akan dihapus.

Return:

1. Kalau sukses: {success:'OK'}
2. Kalau gagal: {error:[error message]} dan akan muncul exception.

Contoh code:

```
function(id){
1     if (confirm("Yakin mau hapus?")){
2         this.dataService.deleteModel(id, this.apiConfig, 'employees').pipe(
3             this.switchMap( ()=>this.loadData() )
          )
4          .subscribe(()=>{
5              alert('berhasil hapus data');
           })
      }
}
```

Baris 1 adalah kode konfirmasi.
Baris 2 adalah cara memanggil fungsi deleteModel dari dataService. Copy paste dan ganti parameter terakhir dengan nama tabel.
Baris 3 adalah proses *pipe* untuk reload data table. Copy paste bulat-bulat.
Baris 4 adalah proses untuk menunggu semua proses di atas selesai.
Baris 5 adalah untuk menampilkan alert di browser saat semua proses sudah berhasil dijalankan tanpa exception.

Method deleteModelNonId

Tujuan: Menghapus sebuah record berdasarkan field selain ID
Parameter:

1. tableName:string = nama tabel yang akan dihapus
2. Key: string = nama field yang menjadi key untuk dihapus
3. Value: any = nilai key yang akan digunakan untuk mencari record yang akan dihapus.
4. apiConfig: IApiConfig = ambil dari this.config.api.

Return

1. Observable dengan data:
    a. Kalau sukses: {success:'OK'}
    b. Kalau gagal: {error:[error message]} dan akan muncul exception.

Method deleteModelMD

Tujuan: Untuk menghapus 1 record master dengan detail-detail-nya.
Parameter:

1. id: number, yaitu berupa id dari record master yang akan dihapus.
2. apiConfig: IApiConfig, ambil dari variable: **this.config.api**.

3. master: nama tabel master
4. detail: nama tabel detail

Return:

1. Kalau sukses: {success:'OK'}
2. Kalau gagal: {error:[error message]} dan akan muncul exception.

Contoh code-nya sama seperti deleteModel. Tinggal sesuaikan method name & parameter-nya.

Method deleteModelNonId

Tujuan: Untuk menghapus record bukan dengan Id
Parameter:

1. Tablename: nama tabel yang akan dihapus
2. Key: nama field yang akan dihapus
3. Value: nilai key yang akan dihapus
4. apiConfig: isi dengan this.config.api

Method getModel

Tujuan: Mengambil data dari tabel berdasarkan id model yang dikirim.
Parameter:

1. Model:any
2. apiConfig: IApiConfig
3. tableName:string
4. detailTableName: string = ‘’ | DetailInfo

Return: Observable(response)

Penjelasan:

1. detailTableName dapat berupa string atau object DetailInfo.
2. DetailInfo = {tableName:string, masterKey:string}, digunakan untuk tabel yang tidak mengikuti konvensi Biz-A

Method queryData

Tujuan: Untuk query data ke API
Parameter:

1. param, adalah object yang berisi:
    a. **length** diisi dengan jumlah record yang akan diambil. Kalau mau ambil semua, isi dengan -1.
    b. **start** diisi dengan berapa banyak length yang akan diloncati. Kalau tidak ada yang diloncati, isi dengan 0 (nol)
    c. **order** diisi untuk mengurutkan data. Isinya adalah array object berikut.
        i. **column** adalah index kolom yang akan diurutkan. Mulai dari 1.
        ii. **dir** diisi dengan 'asc' atau 'desc'.
    d. **filter** diisi dengan format array of condition, di mana condition berisi: {junction, column, operator, value1, value2} di mana:
        i. **junction**: diisi dengan **and** atau **or**
        ii. **column**: diisi dengan nama field yang akan di-filter. Formatnya sama seperti columns|data
        iii. **operator**: diisi dengan **=**, **<>**, **>**, **<**, **>=**, **<=**, **between**, **like**, **in, exists, not exists, is, containing**.
        iv. **value1**: diisi dengan nilai yang akan difilter.
        v. **value2**: hanya diisi bila operator **between**.
        vi. **filter**: Array of filter
    e. **columns** merupakan array dari {data, title, alias, coalesce, valuenull, valuenotnull, fkaggregate, aggregatefield}.
        i. **title** adalah caption yang akan ditampilkan ke tabel.
        ii. **data** adalah field yang akan di-query.
        iii. **alias** (optional) diisi untuk menetapkan field alias. Alias ini diperlukan pada saat field-nya berupa **select** (lihat penjelasan poin viii).
        iv. **coalesce** (optional) diisi **true** atau **false**. Default **false**. Bila coalesce di-set true maka wajib ada key **valuenull,** sedangkan untuk **valuenotnull** optional. Bila **valuenotnull** tidak diisi maka nilai field yang akan digunakan. Key **coalesce** ini diperlukan untuk menghindari munculnya nilai null.
        v. **fkaggregate** (optional) diisi dengan nilai **max** atau **min**. Bila dipakai, maka wajib ada key: **aggregatefield**. Key ini akan digunakan sebagai field aggregate. Fungsi ini digunakan saat akan join data dengan menggunakan nilai aggregate sebagai nilai penghubungnya. Misal: join dengan tabel A dengan ID yang terbesar.
        vi. **key** diisi dengan nama key yang ingin di-map bila flag **map** dinyalakan.
        vii. Tabel pertama yang muncul akan digunakan sebagai tabel utama saat select.
        viii. Bila akan melakukan select di dalam select, bungkus select tersebut dengan tanda kurung buka & tutup dan set **alias**-nya. Contoh: `data:'(select count(*) from table)', alias:'countTable'`**.** Dalam kasus ini, nama alias-nya tidak boleh **count** karena count adalah reserved word di Firebird.
    f. **distinct**:boolean diset ke true apabila kita ingin query secara otomatis menghilangkan record-record yang berulang.

g. **dbIndex**: number, bila mau query dari database di luar default.
h. Semua parameter di atas sifatnya optional. Apabila tidak diisi, default value-nya adalah:

```
{
        start:0,
        length:-1,
        order:[],
        filter:[],
        columns:[],
        dbIndex: default dari setting
}
```

2. **apiConfig**: IApiConfig
3. **parse** : boolean. Default = false. Kalau diisi true, maka data response.data akan otomatis sudah di-JSON.parse-kan.
4. **Map**: boolean. Default = false. Kalau diisi true dan parse juga true, maka data response selain akan di-parse, juga akan di-map dengan key yang diisi di columns.
5. **raiseError**: boolean = true. Secara default bila return dari queryData ada error, akan raise exception. Bila tidak mau raise exception, set flag ini jadi false.

Return: Observable(response)
response :
Kalau berhasil:

```
{
        recordsTotal: number,
        data: String of (array of object)
}
```

Kalau ada kesalahan:

```
{
        error: [pesan error]
}
```

Dan akan muncul exception

Cara pengisian key **data**

1. Untuk format data langsung tanpa join, sandingkan Nama_tabel dan Nama_field dengan penghubung karakter **. (titik)**.
   Contoh:
   `table1.field1`
   Akan membentuk query:
   `SELECT field1 FROM TABLE1`

2. Format untuk foreign key yang perlu join ke tabel lain
   `TBL.ID_FK.LOOKUP_ID.LOOKUP_TBL.LOOKUP_FIELD`

Query yang akan terbentuk adalah:

```
Select A.LOOKUP_FIELD
FROM TBL
LEFT OUTER JOIN LOOKUP_TBL A ON A.LOOKUP_ID = TBL.ID_FK
```

3. Untuk foreign key join ke foreign key yang lain, gunakan format:

   ```
   TBL.ID_FK.LOOKUP_ID.TBL2.LOOKUP_FIELD.LOOKUP2_ID.TBL3.LOOKUP3
   ```

   Query yang akan terbentuk adalah:

   ```
   Select B.LOOKUP3
   FROM TBL
   LEFT OUTER JOIN TBL2 A ON A.LOOKUP_ID = NAMA_TBL.ID_FK
   LEFT OUTER JOIN TBL3 B ON B.LOOKUP2_ID = A.LOOKUP_FIELD
   ```

4. Untuk select dalam select, gunakan kurung buka di awal & kurung tutup. Dan gunakan alias.

   ```
   {data:'table1.ID}
   {data:'(select count(*) from tabel2 where tabel2.id=table1.id)', alias:'count'}
   ```

   Query yang akan terbentuk adalah:

   ```
   SELECT table1.ID, (select count(*) from tabel2 where tabel2.id=table1.id) count
   FROM table1
   ```

5. Untuk menghindari nilai null, gunakan coalesce

   ```
   {data:'table1.amount', coalesce:'true', valuenull:'0',
   valuenotnull:'1'}
   ```

   Query yang akan terbentuk adalah:

   ```
   Select coalesce(table1.amount, '0', table1.amount)
   FROM table1
   ```

6. Untuk join dengan aggregate field:

   ```
   {data:'table1.id.field1.table2.field2', fkaggregate:'max', aggregatefield:'id'}
   ```

   Query yang akan terbentuk adalah:

   ```
   Select table1.id, A.field2
   FROM table1
   LEFT OUTER JOIN table2 A ON A.id=(select max(id) from table2 where field1=table1.id)
   ```

7. Format untuk foreign key yang perlu join ke **Stored Procedure**

   ```
   TBL.ID_FK.OUTPUT_SP_LOOKUP_FIELD.STORED_PROCEDURE.OUTPUT_SP_FIELD
   ```

   Query yang akan terbentuk adalah:

   ```
   Select A.OUTPUT_SP_FIELD
   FROM TBL
   LEFT OUTER JOIN STORED_PROCEDURE A
   ON A.OUTPUT_SP_LOOKUP_FIELD = TBL.ID_FK
   ```

Contoh cara pakai:

```
const param = {
      start:0,
      length:-1,
      order:[{column:1, dir:'asc'}],
      filter:[
            {junction:'', column:'table.field1', operator:'=', value1:id}
      ],
      columns:[
            {data:'table.field1', title:'XXX'},
            {data:'table.field2', tible:'yyy'}
      ]
};
this.dataService.queryData(
      param,
      this.config.api
)
.subscribe( (res:any)=>{
      this.totalRows = res.recordsTotal;
      this.dataSource = JSON.parse(res.data);
})
```

Cara Menggunakan Filter

1. Junction:
    a. Diisi '' bila merupakan elemen pertama dari filter.
    b. Diisi dengan **and** atau **or** untuk elemen filter kedua dan seterusnya.
2. column:
    a. Bila berupa field, format-nya sama seperti columns|data.
    b. Bila berupa query:
        i. Pastikan return-nya hanya 1 record
        ii. Bungkus dengan kurung buka & kurung tutup.
        iii. Contoh: **(select count(*) from tbl_A where tbl_A.id=tbl_B.id)**
    c. Bila merupakan field dari stored procedure:
        i. Pastikan column-nya menggunakan alias
        ii. Panggil dengan format (storedprocedure).field
3. Operator, value1 & value2: lihat penjelasan di atas.
4. filter:

a. Digunakan bila ingin mengelompokan where clause menjadi beberapa bagian
b. Biasanya untuk memisahkan antara junction **or** dan **and**.
c. Jumlah tingkat filter tidak dibatasi. Jadi di dalam filter bisa ada filter lagi.

Cara Mengubah apiConfig

Ada kalanya queryData perlu mengarah ke FINA API di luar yang sudah di-setting di apiConfig. Untuk itu, data di apiConfig perlu di-override.
Struktur data apiConfig adalah sebagai berikut:

```
{
  dbIndex: number,
  url:string,
  methodClass:string,
  useProxy?:boolean,
  finaDbIndex?: number,
  subdomain?:string
}
```

Contoh cara override apiConfig ke IP FINA API yang lain:

```
let config = Object.assign(this.config.api, {url: 'http://ip-lain:212'};
queryData(param, config).subscribe(res=>console.log(res));
```

Method json2fina

Tujuan: Untuk inject data fina menggunakan format json.
Parameter:

1. Data Json yang akan di-inject. Format datanya bisa dilihat di fungsi importJson.
2. apiConfig
3. raiseException:boolean = false. Bila true, saat error akan raise exception.

Contoh payload pada transaksi CR:


```
{
   "_parameters": [
      {
         "id": -1,
         "dataModule": "TdmARPayment",
         "transType": "ARPMT",
         "userId": 0,
         "dbIndex": 3,
         "data": {
            "master": {
               "billtoid": 8660,
               "bankaccount": "112.01",
               "chequedate": "2024-06-28",
               "chequeamount": 11720448,
               "paymentdate": "2024-06-28",
               "chequeno": "cek1001",
               "cheque": 1,
               "description": "Coba Input Cust Recv dari Biz-A",
               "detail": [
                  {
                     "arinvoiceid": 110173,
                     "paymentamount": 9246752,
                     "disctakenamount": 10000,
                     "detail": [
                        {
                           "discaccount": "122.01",
                           "discount": 10000
                        }
                     ]
                  },
                  {
                     "arinvoiceid": 110175,
                     "paymentamount": 2473696,
```

```
                "disctakenamount": 20000,
                "detail": [
                  {
                    "discaccount": "122.01",
                    "discount": 20000
                  }
                ]
              }
            ]
          }
        }
      }
    ]
}
```

Return:
Observable <TJSONObject>
Kalau sukses: {result:[{id: transactionid}]}
Kalau gagal: {result:[{error: [Error Message]}]}

Method previewTemplateFina

Tujuan: Untuk preview template tertentu
Parameter:

1. Data Json sesuai dengan format berikut:

```
{
  dbIndex:number,
  id: number,
  table:string,
  keyfield:string,
  dictionary: array of {
    param:string,
    value:any
  }
}
```

2. apiConfig

Return: array of byte dari gambar dengan format jpeg.

Penjelasan:

1. Dictionary adalah array dengan isi param-value yang akan di-assign ke dictionary fast report template terkait.
2. Daftar dictionary untuk masing-masing form dapat dilihat di sini di bagian Template.

Method getCurrentDate()

Tujuan: Untuk mendapatkan tanggal dari browser.
Parameter: tidak ada
Return: tanggal hari ini dengan format MM/DD/YYYY.

Method httpPost()

Tujuan: Untuk kirim request POST dengan atau tanpa spinner yang muncul.
Parameter:

1. Url: string: berisi url yang dituju.
2. Body: Isi dari body yang akan di-POST.
3. withSpinner: boolean. True = muncul spinner, False = tidak muncul.
4. useProxy: boolean. Optional. Default = false. True = pengiriman melalui proxy. False = langsung dikirim ke url.
5. raiseError:boolean (optional. Default = true)
6. Params:[{param,value}]. Optional header param yang ingin dikirim. Default = null.

Return: Observable response dari request tersebut.

Metode ini bisa digunakan untuk memanggil fungsi-fungsi di FINA API yang belum di-expose oleh Biz-A. Contoh script untuk memanggil fungsi FINA API **calcDiscount** dari method **TFinaMethod**.

```
calcDisc : function(data, dbIndex){
  const url = this.dataService.getUrlApi(
    {
      url : this.config.api.url,
      dbIndex : dbIndex,
      methodClass : 'TFinaMethod'
    },
    'calcDiscount'
  );
  const param = this.dataService.json2Parameters([data])
  return this.dataService.httpPost(
     url, params, false, this.config.api.useProxy
  )
}
```

Method executeBlock()

Tujuan: Untuk mengeksekusi block sql yang dikirim langsung ke Firebird..
Parameter:

1. blockSQL:string
2. apiConfig: IApiConfig

Return: Observable hasil query dari execute block tersebut.

Method updateImg()

Tujuan: Untuk menyimpan file binary ke dalam sebuah field di database FINA API.
Parameter:

1. config: IApiConfig
2. body:any. Isi body ini mengikuti parameter di POST img di FINA API.
3. withSpinner:boolean = true,
4. raiseError:boolean = true

Return: Observable post dengan hasil:

1. Kalau berhasil: {success:'OK'}
2. Kalau gagal: {error:'error_message'}

Contoh cara update file jpg lalu simpan ke field logo di tabel Item di database FINA:

```
fields:[
  {key:'file', type:'input-file',props:{isi sendiri}},
  {type:'button', props:{
     text:'kirim',
     onClick:()=>{
       this.f().btnKirimClick();
     }
  }
],
functions:{
  btnKirimClick:()=>{
    const target = this.form.get('file').value;
    const reader = new FileReader();
    reader.readAsBinaryString(target);
    getBody = (f)=>({
      dbIndex:'1',
      tableName:'item',
      imgCol : 'logo',
      keyCol : 'ItemNo',
      formatCol : 'format_logo',
      keyValue  : 'ITEM A', //ini adalah item yang akan di-update
```

```
      formatValue : 'jpg',
      img: window.btoa(f.target.result)
    })
    this.fromEvent(reader, 'load').pipe(
      this.mergeMap(e=>this.dataService.updateImg(this.config.api, getBody(e)))
    ).subscribe(data=>{
      console.log(data);
    })

  }
}
```

Method genId()

Tujuan: Untuk mendapatkan nilai dari generator dan dinaikan 1.
Parameter:

1. generator_name:string → diisi dengan nama generator yang akan diambil nilainya.
2. Config: IApiConfig
3. dbIndex:number (optional) → Hanya perlu diisi bila generator-nya selain dbIndex default Biz-A.

Contoh code untuk mengambil generator APINV_GEN di database FINA:

```
this.dataService.genId('APINV_GEN', this.config.api, 1)
.subscribe(data => console.log(data))
```

executeCmd

Tujuan: Mengeksekusi sederetan perintah ke FINA API. Teknik ini digunakan apabila executeBlock memakan waktu terlalu lama sehingga time out.
Parameter:

1. Array of Command.

Format Command =

```
{
  arg:any,
  tablename:string,
  command:string
}
```

Cara pakai:

1. Untuk menjalankan perintah sendModel:
    a. Isi arg dengan model yang akan disimpan
    b. Isi tablename dengan nama tabel
2. Untuk menjalankan perintah di DataService yang lain:

a. Isi arg sesuai dengan perintah yang dibutuhkan. Tidak perlu mengirim apiConfig.
b. Kosongkan tablename
c. Isi command dengan nama perintah di DataService

Contoh:
Bila mau memanggil perintah deleteModelNonId, isi parameter-nya sebagai berikut:

```
[
  {
    arg:['nama_tabel', 'key', 'value'],
    command:'deleteModelNonId'
  }
]
```

Property http

Property http digunakan bila akan melakukan request selain POST.
Tipe property ini adalah HttpClient. Penjelasan lengkapnya bisa dilihat di sini.
Property http ini bisa digunakan untuk POST ke API server selain FINA API.
Berikut contohnya:

```
let body = {data:'bla', apiAddress:'http://localhost:8000'}
this.dataService.http.post(
    `${this.config.api.url}/TOrmMethod/[methodName]`,
    body,
    {
      params: this.dataService.getHeaderParams(this.config.api)
    }
).subscribe(data=>{
    console.log(data)
})
```

Penjelasan Parameter:

1. Url: harus dimulai dengan TOrmMethod. Selanjutnya baru bisa pakai nama sub method yang lain.
2. Body harus berupa object dan harus ada key dengan nama **apiAddress**. apiAddress ini yang akan berisi url dan port target API yang dituju.
3. Options: ikuti contoh default.

callback

Fungsi: Untuk memanggil CLI-Script.
Syarat: harus menggunakan hub.
Parameter:

1. Data:any, merupakan isi body yang akan diterima oleh CLI Script.
2. Scriptid: number, merupakan id dari script yang akan dipanggil.
3. Config: IApiConfig. Gunakan this.config.api

Return:

1. Bila setting benar akan membalikan observable http post.
2. Bila tidak menggunakan subdomain akan mengembalikan of(null).

appService

sendMail

Tujuan: Untuk mengirim email melalui server
Parameter:

1. appConfig
2. Data

Format data:

```
{}
      companyname: this.config.app.company,
      mailOptions:{
            from:'Biz-a <biz-a@imamatek.com>',
            to:'bla <bla@abc.com>',
            subject: 'isi dengan subject email',
            text: 'isi dengan text email',
            html: 'isi email dalam format html'
      },
      smtpConfig:{
            host:string,
            port:number,
            secure:boolean,
            auth:{
                  user:string,
                  pass:string
            }

}
```

.
Keterangan:

1. smtpConfig adalah parameter optional. Bila tidak ada, default akan menggunakan config yang ada di server.
    a. host adalah alamat smtp server yang akan digunakan
    b. port: adalah port smtp yang akan digunakan
    c. secure: true bila menggunakan SSL.
    d. auth:
        i. user: username yang akan digunakan untuk otentikasi ke smtp server.
        ii. pass: adalah password saat otentikasi ke smtp server

getCurrentDateTime

Tujuan: Mendapatkan tanggal dan jam saat ini dari server.
Parameter:

1. appConfig

Return: `{date:tanggal_jam_saat_ini}`
`Format tanggal dalam ISO format (YYYY-MM-DDTHH:mm:ss.sssZ)`

getLocation

Tujuan: Untuk mendapatkan posisi GPS dari browser saat itu.
Parameter: tidak ada
Return: GeolocationCoordinates.

logout()

Tujuan: Untuk logout dari session
Parameter: this.config.app
Return: tidak ada.

helper

formatDate2API

Tujuan: mengubah format date dari server ke format yang bisa diterima oleh API

loadingService

Fungsinya: Untuk menampilkan spinner saat Biz-A melakukan proses yang membutuhkan waktu cukup lama.

start()

Tujuan: Menampilkan modal form dengan spinner di tengah-tengah.
Parameter:

1. Message: string; optional. Untuk mengganti caption yang muncul. Default: **Loading…**
2. Mode: 'determinate'|'indeterminate'.
    a. determinate artinya: hanya 1 putaran dari 0 hingga 100. Biasanya digunakan untuk progress yang sudah tahu jumlahnya.
    b. indeterminate artinya akan berputar berulang-ulang sampai ditutup. Biasanya digunakan untuk progress yang tidak tahu kapan akan selesai.

Return: Referensi ke instance loading form-nya. Referensi ini akan digunakan saat meng-update nilai atau menutup form.

updateProgress

Tujuan: mengupdate progress bar dalam mode **determinate**.
Parameter

1. Ref: diisi dengan referensi yang didapat saat start().
2. Value: number (0 s/d 100), diisi dengan angka antara 0 hingga 100 untuk menunjukan progress.

Return: tidak ada.

stop

Tujuan: Menutup form progress.
Parameter: Referensi form yang akan ditutup.
Return: tidak ada.

Contoh kode untuk menampilkan progress::

```
let ref = this.loadingService.start('Injecting...', 'determinate');
let count = 1;
const MAX = 3;
this.range(1,MAX).pipe(
      this.concatMap( () => this.dataService.json2fina( obj, this.apiConfig, 3))
).subscribe({
      next: () => {
```

```
            count++;
            this.loadingService.updateProgress( loadingRef, count*100/MAX )
      },
      complete: ()=>{
            this.loadingService.updateProgress( loadingRef, 100);
            this.loadingService.stop(loadingRef);
            alert('selesai')
      }
})
```

storageService

Fungsi: Menyediakan fungsi-fungsi yang berhubungan dengan local storage browser

getCurrentUser()

Tujuan: Untuk mendapat user yang saat ini.
Parameter: tidak ada
Return: User ID yang bersangkutan atau -1 kalau tidak ada.

getLoginInfo()

Sudah tidak digunakan. Gunakan config.api.

apiSvc

Fungsi: Menyediakan fungsi-fungsi yang berhubungan dengan Web API pihak ke-3.

loadScript()

Tujuan: untuk load library API dari pihak ke-3. Ini untuk mengganti <script> di HTML.
Parameter:

1. src: string. Diisi dengan url dari library tersebut.
2. async:boolean. Diisi dengan true kalau mau di-load tanpa ditunggu atau false bila mau tunggu. Hati-hati bila false karena hal ini akan membuat Biz-A seolah-olah hang. Sedangkan kalau true, pastikan library sudah selesai loading sebelum digunakan.
3. defer:boolean. True kalau mau di-load secara paralel dan dieksekusi saat HTML sudah selesai di-parsing.

Return: tidak ada

Untuk yang ingin mendapatkan event saat script sudah berhasil di-load, silakan menggunakan scriptService.

proxy(`method:string, url:string, config:IAppConfig, body:any=null, options:any=null`)

Tujuan: untuk mengalihkan panggilan dari Biz-A client ke API server agar tidak terkena CORS error. Dengan menggunakan proxy, semua request akan ditujukan ke Biz-A server dulu, baru diteruskan ke API server.

Parameter:

1. Method:string (GET | POST)
2. url:string. Diisi dengan url server yang dituju
3. config:IAppConfig. Diisi dengan this.config.app.
4. Body (optional). Diisi dengan body yang akan dikirim ke API server
5. Options (optional). Diisi dengan object {responseType, headers}
    a. responseType:string
    b. headers adalah object yang akan mengatur proxy request headers yang akan dirimkan ke API Server.
    c. Bila ingin mengubah “User-Agent” yang akan dikirimkan ke API Server, maka set di header ***‘biza-proxy-user-agent’***.
        i. Bila header ini bernilai “” (string kosong), maka Proxy tidak mengirimkan header ‘User-Agent’ ke Server API
        ii. Bila header ini tidak diset, maka Proxy akan menggunakan ‘User-Agent’ yang dikirimkan oleh browser yang tempat BizA Client dibuka
        iii. Contoh :
        {

```
headers: {
    "Content-Type": "text/plain",
    "biza-proxy-user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64;
rv:136.0) Gecko/20100101 Firefox/136.0",
}
}
```

d. Bila ada headers yang tidak ingin dikirimkan ke API Server, maka set di header ***'biza-proxy-remove-header'***, isinya berupa case insenstive comma-separated string dengan delimiter ',' (koma). Contoh :

```
{
    headers: {
        "Content-Type": "text/plain",
        "biza-proxy-remove-header": "sec-ch-ua, sec-ch-ua-mobile",
    }
}
```

Return: apapun response dari API server yang di-request.

printService

print

Tujuan: untuk mencetak sebuah file HTML.
Contoh:

```
this.printService.print(html:string, onAfterPrintFn:Function);
```

Penjelasan:

1. Parameter pertama adalah data string yang akan di-print dalam format HTML.
2. Parameter kedua adalah function yang akan dipanggil setelah user klik tombol Print.

authService

logout

Tujuan: untuk logout dari Biz-A
Parameter:

1. this.appConfig

Return: Tidak ada

getLogoutEvent

Tujuan: Event ini akan terpicu saat Biz-A logout.
Parameter: tidak ada
Return: observable

Contoh:

```
this.authService.getLogoutEvent().subscribe(()=>console.log('logout'))
```

isLoggedIn

Tujuan: Untuk mengecek apakah user masih login atau tidak
Parameter: tidak ada
Return: Boolean

getExpiration

Tujuan: Untuk mendapatkan waktu expired.
Parameter: tidak ada
Return: object dayjs.

config

api

Tujuan: Mengambil config untuk API Server

```
dbIndex: number,
url:string,
methodClass?:string,
useProxy?:boolean,
finaDbIndex?: string,
subdomain?:string,
defaultForm?:string,
```

app

Tujuan: Mengambil config untuk Biz-A server

```
company: string,
url:string
```

mapService

Tujuan: Untuk menampilkan peta & overlay
Service ini menggunakan library Leaflet.

Fungsi-Fungsi yang tersedia:

L

L merupakan object dari Leaflet
Contoh penggunaan: this.mapService.L

init

Parameter Input:

1. divId: string, merupakan id dari div yang akan menampilkan peta
2. coord:[lat, lng], merupakan array latitude & longitude sebagai koordinat awal yang akan menjadi titik tengah peta.
3. zoom:number, merupakan tingkat zoom yang akan digunakan. Nilainya antara 0 sampai 18. 18 adalah zoom yang paling dekat.

Return: Object map

Fungsi: Untuk inisiasi Library Leaflet dengan melakukan setting default icon serta memanggil fungsi map.

Contoh pemakaian: this.mapService.init('map', [0, 0])

tile

Berfungsi untuk load peta default menggunakan openstreetmap.
Cara penggunaan: this.mapService.tile()

marker

Berfungsi untuk membuat tanda di peta.
Parameter input:

1. koordinat: [latitide, longitude]
2. Options: marker options.

Contoh aplikasi di Biz-A:

```
fields:[
  {
    type:'text',
    id:'map',
    props:{attributes:{ style:'height:300px’}}
  }
],
functions:{
  onInit:()=>{
    const setMarker = (coord)=>{
      this.marker.setLatLng(coord, {
        draggable:'true' //set marker agar bisa digeser
      }).bindPopup(coord).update()
    }
    const initMap = async()=>{
      let curLocation = [0, 0]
      let map = await this.mapService.init('map', curLocation,3)
      this.mapService.tile.addTo(map)
      this.marker = this.mapService.marker(curLocation, {draggable:'true'})
      map.addLayer(this.marker)

      //set peta ke lokasi GPS
      map.locate({setView:true, maxZoom:18, enableHighAccuracy:true})

      map.on('locationfound', (e)=>{
        setMarker(e.latlng)
        map.setZoom(18)
      })

      this.marker.on('dragend', (e)=>{
        let newPos = this.marker.getLatLng()
        setMarker(newPos)
      })
    }
    initMap()
  }
}
```

db

Tujuan: Untuk akses indexedDB menggunakan dexie.
Contoh cara pakai:

```
onInit:function(){
  const doDb = (async()=>{
    let Dexie = await this.db.load() //inisialisasi dexie
    let db = new Dexie('DB') //create database dengan nama DB
    db.version(1).stores({ //buat table friends
      friends:`
      id,
      name,
      age`
    })
    db.friends.bulkPut([ //isi dgn 3 record
      {id:1, name:'a', age:31},
      {id:3, name:'b', age:29},
      {id:5, name:'c', age:27},
    ]).then(()=>{
      //cari record umur antara 25 dan 30
      return db.friends.where('age').between(25,30).toArray()
    }).then(friends=>{
      console.log('found:')
      friends.map(f=>console.log(f.name))
      return db.friends.orderBy('age').toArray() //order berdasarkan umur
    }).then(friends=>{
      console.log('order by age')
      friends.map(f=>console.log(f))
    }).catch(e=>console.log('error:', e))
  }).bind(this)

  doDb()
},
```

scriptService

Tujuan: Untuk load external script/library secara runtime hanya 1 kali saja.

Fungsi yang tersedia:

load

Parameter:

```
{
  name:string,
  src:string,
}
```

Penjelasan:

1. name adalah nama library
2. Src adalah url dari library tersebut.

Return: observable yang akan selesai pada saat script sudah berhasil di-load.

Contoh penggunaan:

```
let script = {
  name:'ocr',
  src:'https://cdnjs.cloudflare.com/ajax/libs/tesseract.js/4.1.1/tesseract.min.js'
}
this.scriptService.load(script).pipe(
  this.tap(()=>console.log('loading...'))
).subscribe(()=>{
  console.log('loaded')
  this.f().getVideo();
})
```

rx

rx service terdapat di Form & List.
Selain mengakses fungsi atau operator Rxjs yang sudah ada, seperti this.mergeMap dll, fungsi-fungsi yang lain juga bisa diakses dengan cara:
this.rx.get.xxx
Di mana xxx adalah nama fungsi atau operator rxjs yang ingin digunakan.
Daftar lengkap RxJs bisa dilihat di sini.

Property lain yang bisa diakses adalah:

1. rx.webSocket

bottomSheetService

Service ini berguna untuk menampilkan form yang muncul dari bawah. Form ini umumnya digunakan untuk aplikasi yang dibuka dari mobile.

Fungsi-fungsi yang tersedia:

1. open(config:MatBottomSheetConfig).
    a. Penjelasan tentang MatBottomSheetConfig dapat dilihat di sini.
    b. Di dalam config ada property data yang bertipe object. Key-nya adalah:
        i. fields: adalah isi form yang akan ditampilkan di bottom sheet. Cara isinya sama dengan fields di form.
        ii. Model:mengirim model dan data awal untuk form bottom sheet.
    c. Return dari open adalah MatBottomSheetRef.
2. Variable-variable yang bisa diakses di dalam form:
    a. this.data yang merupakan instance dari config.data
    b. this.form
    c. this.bottomSheet: MatBottomSheetRef, merupakan instance bottom sheet yang sedang terbuka.
    d. Agar bisa menggunakan variable **this**, pastikan deklarasinya menggunakan **function**, bukan **arrow function**.

Contoh:
Button di dalam bottom sheet yang kalau di-klik akan menutup bottom sheet-nya.

```
const config = {
  data:{
    model:{},
    fields:[
      {
        type:'button',
        props:{
          text:'Tutup Bottom Sheet',
          onClick:function(){ //tidak boleh arrow function
            this.bottomSheet.dismiss(this.data.model);
          }
        }
      }
    ]
  }
}
let ref = this.bottomSheetService.open(config)
ref._ref.closed.subscribe(data=>{
```

```
  console.log('data yg dikirim', data);
}
```

notificationService

Service ini berfungsi untuk menampilkan pesan menggunakan MatSnackBar.
Fungsi-fungsi yang tersedia adalah:

1. showSuccess
    a. Parameter: message yang akan ditampilkan.
    b. Durasi: 3 detik hilang sendiri
2. showError
    a. Parameter: message yang akan ditampilkan
    b. Durasi: 5 detik hilang sendiri
3. showNotif
    a. Bila ingin membuat notification sendiri, silakan gunakan showNotif.
    b. Parameter:
        i. message:string
        ii. action:string
        iii. Config: `MatSnackBarConfig<any>`
    c. Return: `MatSnackBarRef<TextOnlySnackBar>`

Contoh:
Untuk menampilkan kesalahan:

```
this.notificationService.showError('Form belum lengkap')
```

Bab 7. Cara Setup Biz-A

Setting CDM

Biz-A akan menggunakan sign-in yang sama dengan yang digunakan di FINA OF Web Server. Apabila belum ada, silakan dibuat dulu menggunakan FINA.
Catat nama company & password-nya.

Setting FINA API

Berikut urutan setting yang harus dilakukan untuk mempersiapkan FINA API:

1. Setup FINA API. Cara setup FINA API bisa dilihat di dokumen ini.
2. Untuk memastikan FINA API sudah berjalan dengan baik, coba request ke: `http://IP_PUBLIC:212/fina/rest/TMfgMethod/sql_data_to_json/select * from company/1` . Bila dapat response, berarti FINA API-nya sudah siap digunakan.
3. Buat 1 database FINA yang akan digunakan sebagai tempat penyimpanan user. Jadikan database ini sebagai database nomor 1.
4. Buat 1 database firebird kosong dengan menggunakan Flamerobin. Jadikan database ini sebagai database nomor 2.
5. Buat tabel-tabel yang diperlukan oleh Biz-A secara manual melalui Flamerobin.
6. Pastikan regional setting untuk FINA API sudah menggunakan decimal separator dengan karakter titik (.) dan tanggal menggunakan format M/dd/yyyy. Khusus untuk penggunaan FINA API sebagai windows service, cek regional setting dan mengubahnya (bila perlu) melalui menu: Control Panel | Regional and Language | Administrative | Welcome Screen and new user accounts | Copy Settings.

Setting Biz-A Server

Cara setting Biz-A Server:

1. Config yang diinginkan pada https://admin.biz-a.id.
2. Setting file tersebut sesuai dengan penjelasan di Bab 2.
3. Bila menggunakan FINA Hub:
    a. API_URL: Tambahkan nama subdomain di query parameter “subdomain” dan path /hub di belakang. Contoh: “API_URL”: “~~https://biz-a.herokuapp.com/hub/fina/rest/TOrmMethod/%22list%22?subdomain=imamatek~~”

    “https://server.biz-a.id/hub/fina/rest/TOrmMethod/%22list%22?subdomain=imamatek”

    b. useProxy harus false

Testing Tahap 1

Pastikan setting tahap 1 ini sudah ter-set dengan baik dengan cara masuk ke website **biz-a.id**, lalu login dengan company name, username & password yang tepat.
Bila berhasil masuk, berarti setting tahap 1 sudah selesai dan sudah bisa lanjut ke setting tahap berikutnya.

Setting Menu

Silakan ikuti petunjuk cara setting menu di bab 3.
Lalu sign out dan sign in lagi untuk melihat apakah menu yang dibuat sudah muncul sesuai dengan yang dibuat.
Setiap form atau list yang dibuat, pastikan namanya dicatat dengan baik karena nama form atau list ini yang akan dipakai untuk membuat file javascript (js) yang akan dijelaskan setelah ini.

Setting File Template

Untuk setiap menu item yang dibuat, harus dibuat 1 file template js di dalam folder company/[companyname].
Sebagai contoh, bila menu.json adalah sebagai berikut:

```
[
    {"menuName":"", "caption":"Home", "link":["./main"], "subMenu":[]},
    {"menuName":"", "caption":"Master Data", "link":[], "subMenu":[
      {"menuName":"", "caption":"Karyawan", "link":[], "subMenu":[
        {"menuName":"", "caption": "Karyawan Baru", "link":["./form", "employee", null], "subMenu":[]},
        {"menuName":"", "caption": "Daftar Karyawan", "link":["./list", "employees"], "subMenu":[]}
      ]}
    ]},
    {"menuName":"", "caption":"Data", "link":[], "subMenu":[
      {"menuName":"", "caption":"Cuti", "link":[], "subMenu":[
        {"menuName":"", "caption":"Permohonan Cuti Baru", "link":["./form", "permohonancuti", null],
"subMenu":[]},
        {"menuName":"", "caption":"Daftar Cuti", "link":["./list", "daftarcuti"], "subMenu":[]}
      ]}
    ]}
]
```

Maka harus ada 4 file template js yang disediakan, yaitu:

1. employee.js
2. employees.js
3. permohonancuti.js
4. daftarcuti.js

Tips: Untuk mempermudah mengidentifikasi isi file, bedakan nama template file antara form dan list. Misal: untuk form selalu diakhiri dengan form. Mis: karyawanform.js. Lalu untuk list diakhiri dengan kata list, misal: karyawanlist.js.

Untuk upload file-file template ini, silakan gunakan npm uploadbiza seperti yang dijelaskan di Bab 4 bagian upload template.

Cara Membuat Form

Langkah Pertama

Buat file template js sesuai dengan petunjuk pembuatan menu di atas

Langkah Kedua

Lalu isi form tersebut dimulai dari membuat kerangkanya dulu, yaitu:

```
get = function(){
      return {
            model:{},
            fields:[],
            functions:{}
      }
}
```

Langkah Ketiga

Isi array **fields** yang akan ditampilkan di form sesuai dengan petunjuk di Bab 4.
Tips: Tambahkan fields 1 per 1. Setiap kali menambah 1 field, cek dulu di biz-a.id. Hal ini akan mempermudah mencari letak kesalahan yang terjadi.

Contoh untuk menambahkan input nama dan button di form:

```
get = function(){
  return {
    model:{},
    fields:[
      {
        type: 'input',
        key: 'nama'
      },
```

```
      {
        type: “button”,
        props: {
          text : “Save”,
          onClick  : function($event, id){
          }
        }
      }
    ],
    functions:{}
  }
}
```

Langkah Keempat

Isi object **model** dari form tersebut.
Setelah model dibentuk, buatkan tabel dengan nama yang sama persis dengan nama form ini, lalu isi field-nya sama persis dengan property di model ini.
Semua property yang ada di model harus ada di tabel. Tapi di tabel boleh ada field yang tidak ada di model.
Isi juga key: tableName bila model ingin otomatis di-load dari tabel saat mode edit.

```
get = function(){
  return {
    model:{
      nama:null
    },
    tableName:’karyawan’,
    fields:[
      {
        type: ‘input’,
        key: ‘nama’
      },
      {
        type: “button”,
        props: {
          text : “Save”,
          onClick  : function($event, id){
          }
        }
      }
    ],
    functions:{}
  }
}
```

Tips: Nyalakan debug mode untuk melihat apakah data yang diisi di GUI sudah masuk ke model atau tidak. Cara menyalakan debug mode bisa dilihat di Bab 5 bagian **onInit**.

Langkah Kelima

Isi object **functions** dengan fungsi-fungsi yang akan dikerjakan di form tersebut.
Beberapa fungsi yang biasanya perlu dibuat antara lain:

1. Fungsi untuk menyimpan model ke database
2. Fungsi untuk mengirim email notifikasi.
3. Fungsi untuk pindah ke menu yang lain.

Contoh fungsi untuk menyimpan model ke tabel ‘Karyawan’:

```
get = function(){
  return {
    model:{
      nama:null
    },
    fields:[
      {
        type: ‘input’,
        key: ‘nama’
      },
      {
        type: “button”,
        props: {
          text : “Save”,
          onClick  : function($event, id){
            this.f().btnSaveClicked();
          }
        }
      }
    ],
    functions:{
      this.dataService.sendModel(
        this.model,
        this.config.api,
        ‘karyawan’
      ).subscribe(()=>{
        alert(`Karyawan ${this.model.nama} berhasil disimpan`)
      })
    }
  }
}
```

Tips: sama seperti **fields**, lakukan penambahan fungsi secara bertahap. Setiap kali ada penambahan, cek perilakunya di biz-a.id. Script di file js ini akan otomatis di-reload saat kembali masuk ke menu tersebut. Jadi apabila mau refresh script di form A, tinggal ke menu yang lain, lalu balik lagi ke form A. Tidak perlu sign-out dan sign-in lagi.

Cara Membuat List

Langkah Pertama

Buat file template js sesuai dengan menu yang sudah dibuat.

Langkah Kedua

Isi file template tersebut dengan format sebagai berikut:

```
get = function(){
      return {
            columns:[],
            filter:[],
            functions:{}
      }
}
```

Langkah Ketiga

Isi **columns** sesuai dengan petunjuk di Bab 4 bagian **Columns**.
Tips: Tambahkan kolom 1 per 1 dan lakukan pengecekan melalui biz-a.id.

Langkah Keempat

Apabila diperlukan filter-filter default, silakan isi di array **filter**. Cara membuat filter bisa dilihat di Bab 6 bagian **queryData**.

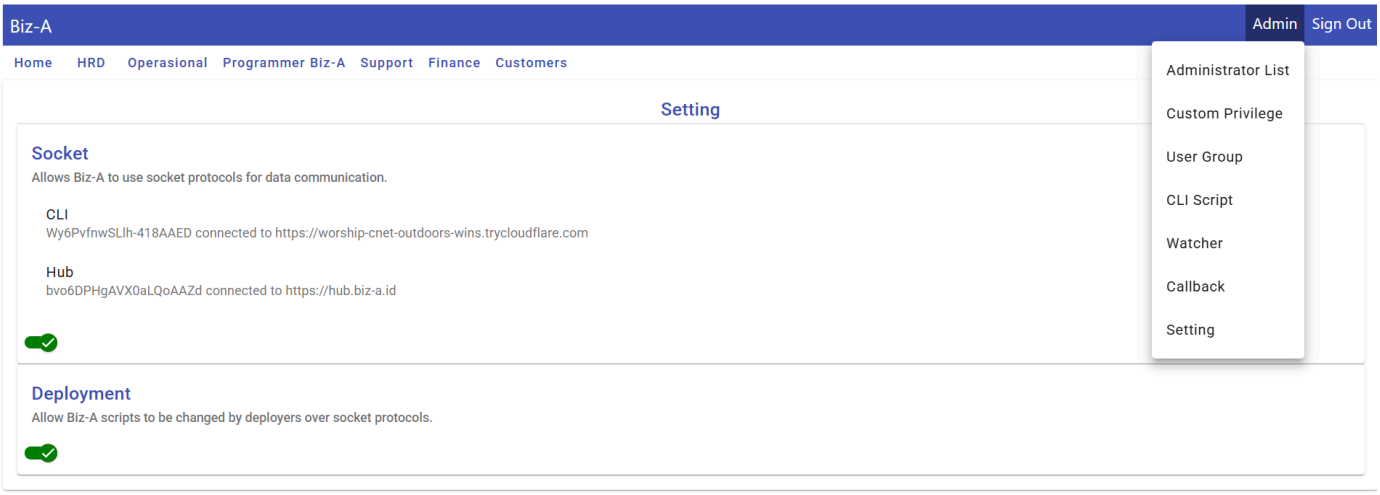
Langkah Kelima

Tambahkan fungsi-fungsi yang diperlukan di object **functions**.

Setting Biz-A Client

1. Login ke Biz-A Client dengan hak Akses **Administrator**

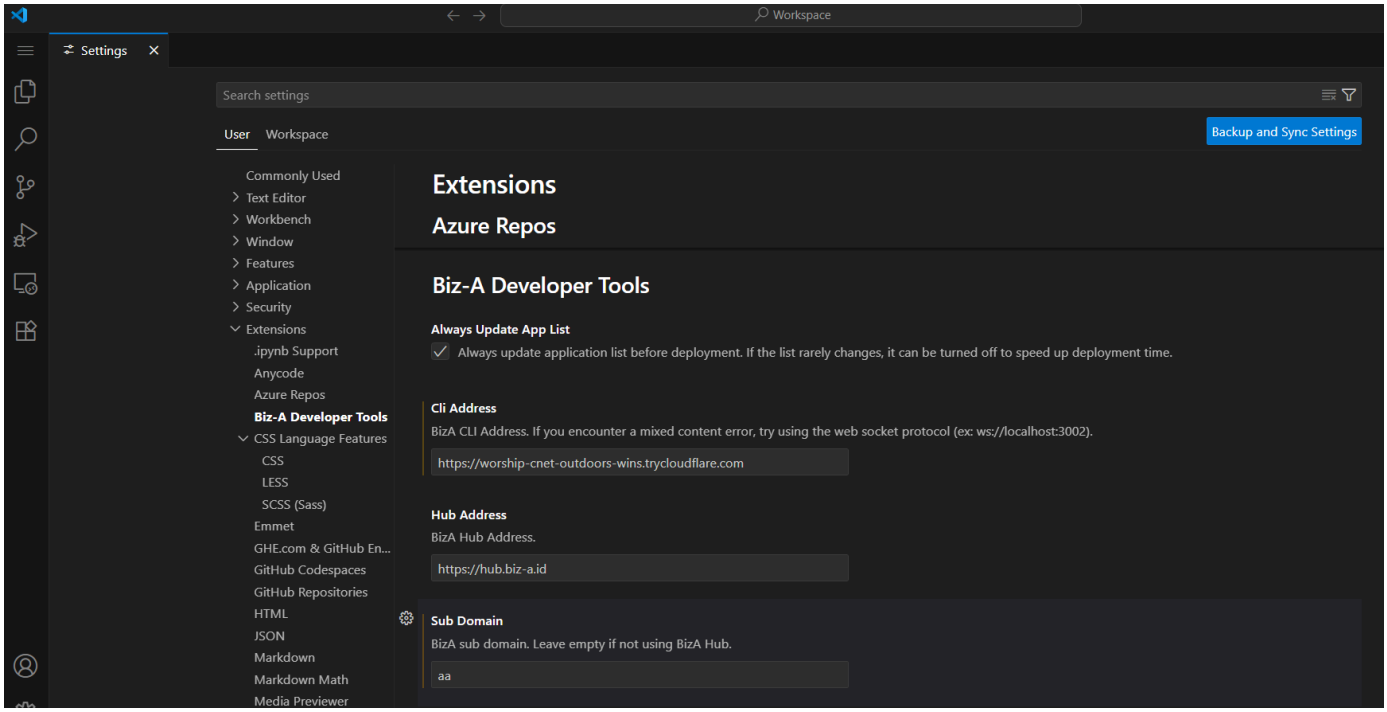
2. Pilih menu “Admin” - “Setting



- Socket
    - Bila dinonaktikan, maka koneksi ke CLI dan Hub akan diputus.
    - Bila diaktifkan, maka koneksi CLI dan Hub akan disambung.
    - CLI bila terkoneksi akan ada Public URL dari CLI yang terkoneksi. Contoh : pada gambar di atas Public URL CLI adalah *https://worship-cnet-outdoors-wins.trycloudflare.com*
    - Nilai default untuk opsi ini adalah **aktif**.
- Deployment
    - Nilai default untuk opsi ini adalah **non-aktif**.
    - Feature ini hannya bisa aktif, apabila opsi *Socket* aktif dan terkoneksi dengan baik.

Deployment

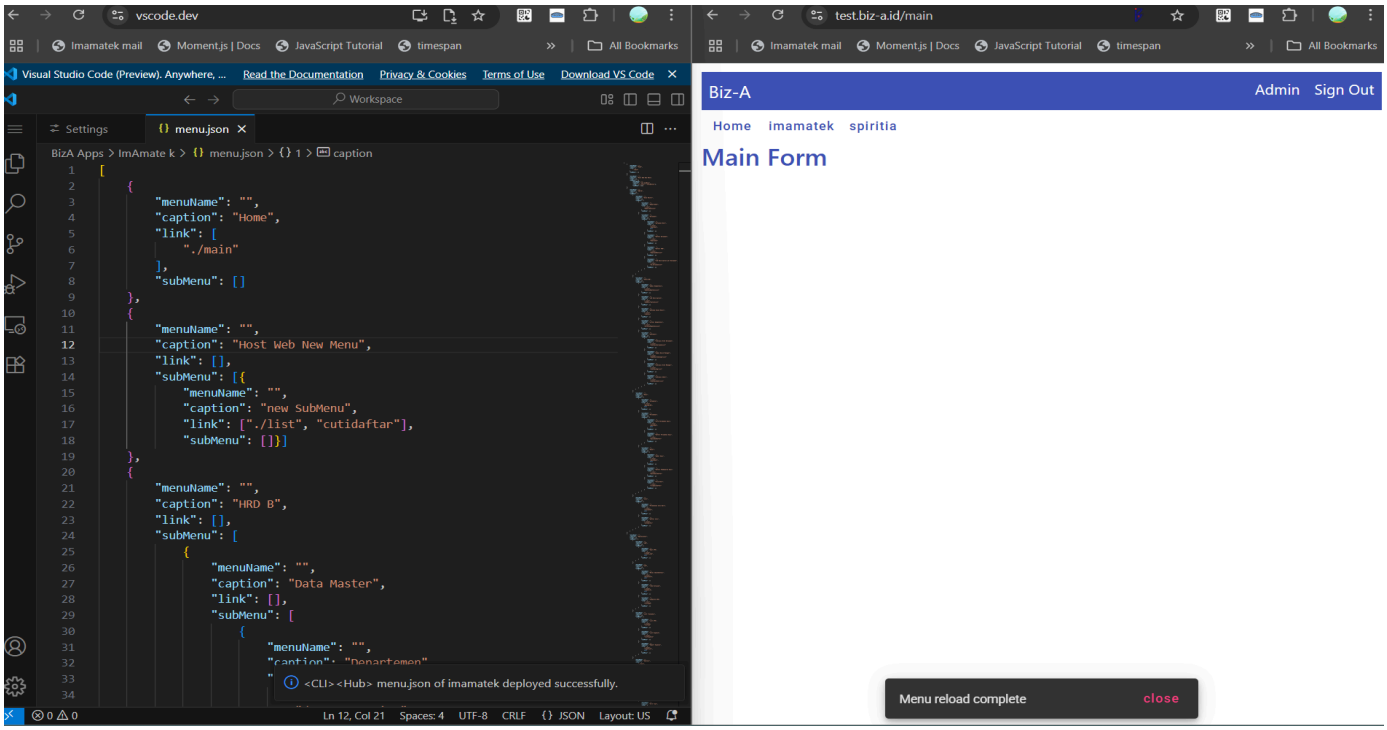
1. Setup Biz-A dengan minimal 1 buah App. (Pastikan sudah bekerja dengan baik).
2. Gunakan Editor VsCode (desktop or web).
3. Install Biz-A DevTools extension untuk VsCode.
4. Aktifkan feature deployment pada Biz-A Client. Simpan Public URL CLI yang tertera pada form setting tersebut.
5. Pada VsCode, buka folder app Biz-A template yang sesuai dengan App yang ada di Biz-A Client. Pastikan memilih “Open Folder”, bukan “Open File” ketika pertama kali membuka sebuah file script. Hal ini diperlukan agar workspace di vsCode bisa aktif.
6. Pada VsCode, masuk ke bagian setting untuk extension Biz-A DevTools.
    a. Isi Public URL CLI pada opsi “Cli Address”
    b. Isi subDomain yang dipakai pada opsi “subDomain”



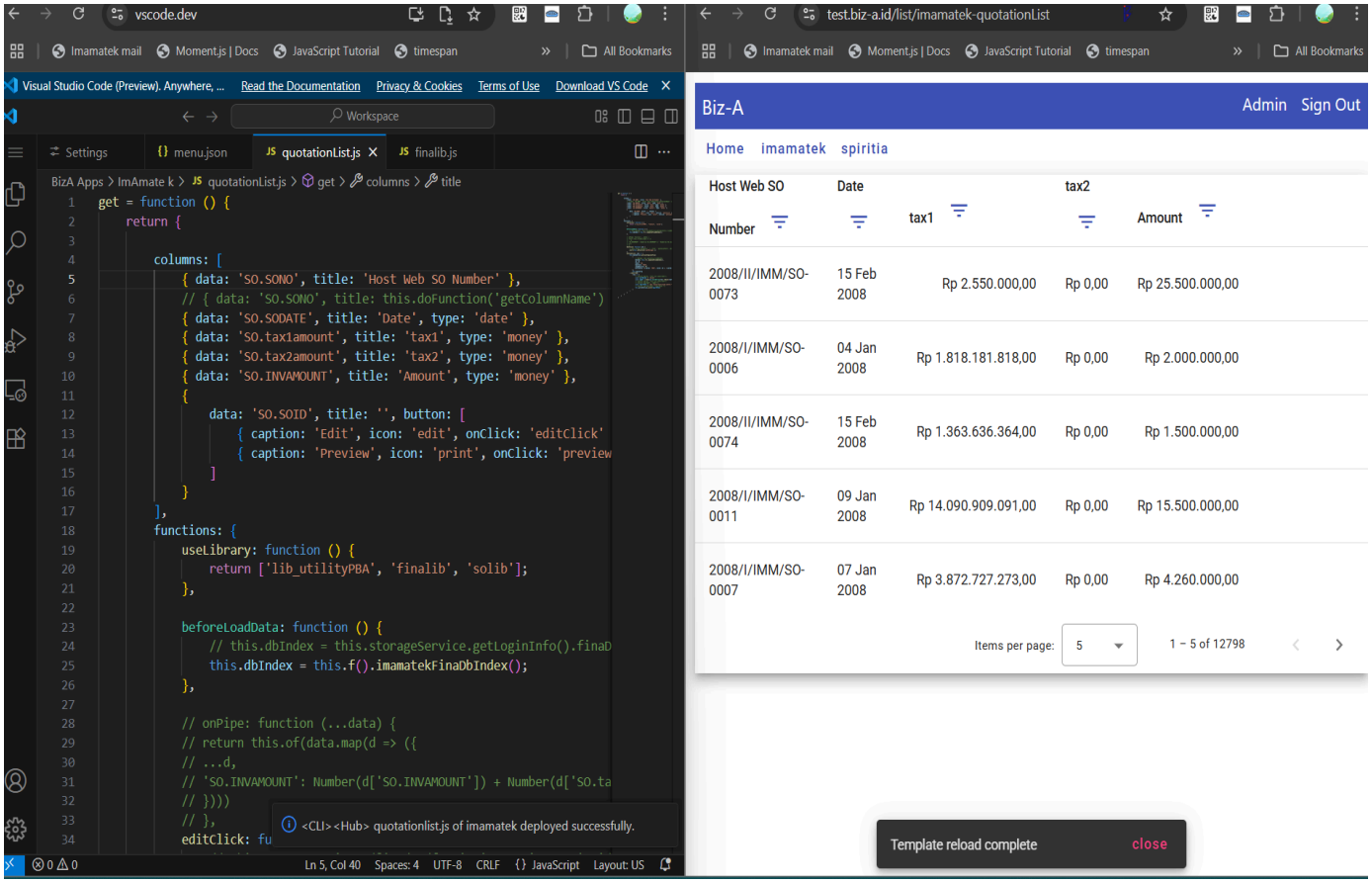
7. Buka salah satu file template untuk app Biz-A, misalkan : “menu.json”. Lakukan perubahan di file ini dan simpan. Setelah proses simpan akan terjadi proses pengiriman script file ini ke Biz-A Client yang sudah aktif (Point 4).

Contoh :

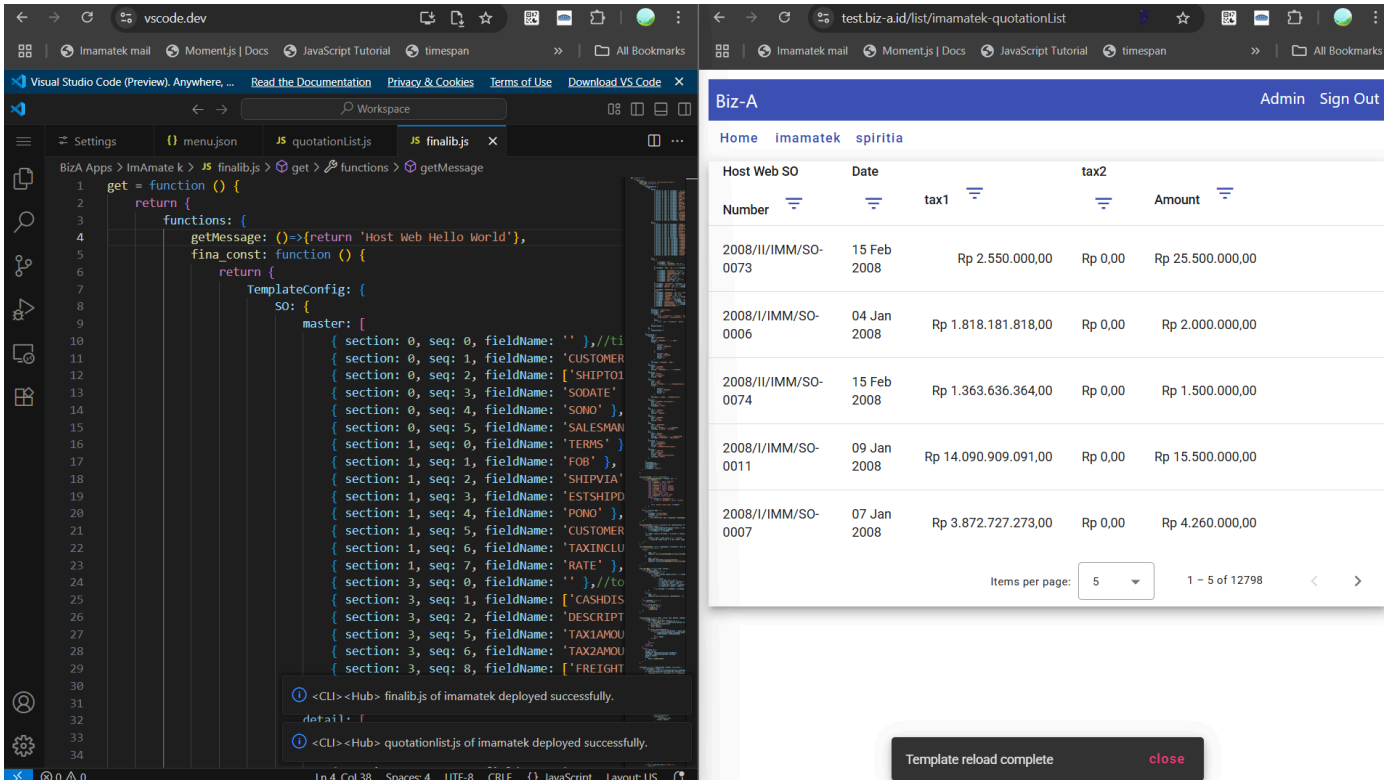
➔ Deployment menu.json (Ada perubahan file menu.json di salah satu apps Biz-A)



➔ Deployment Form\List (Ada perubahan file template di form / list yang sedang terbuka Biz-A)

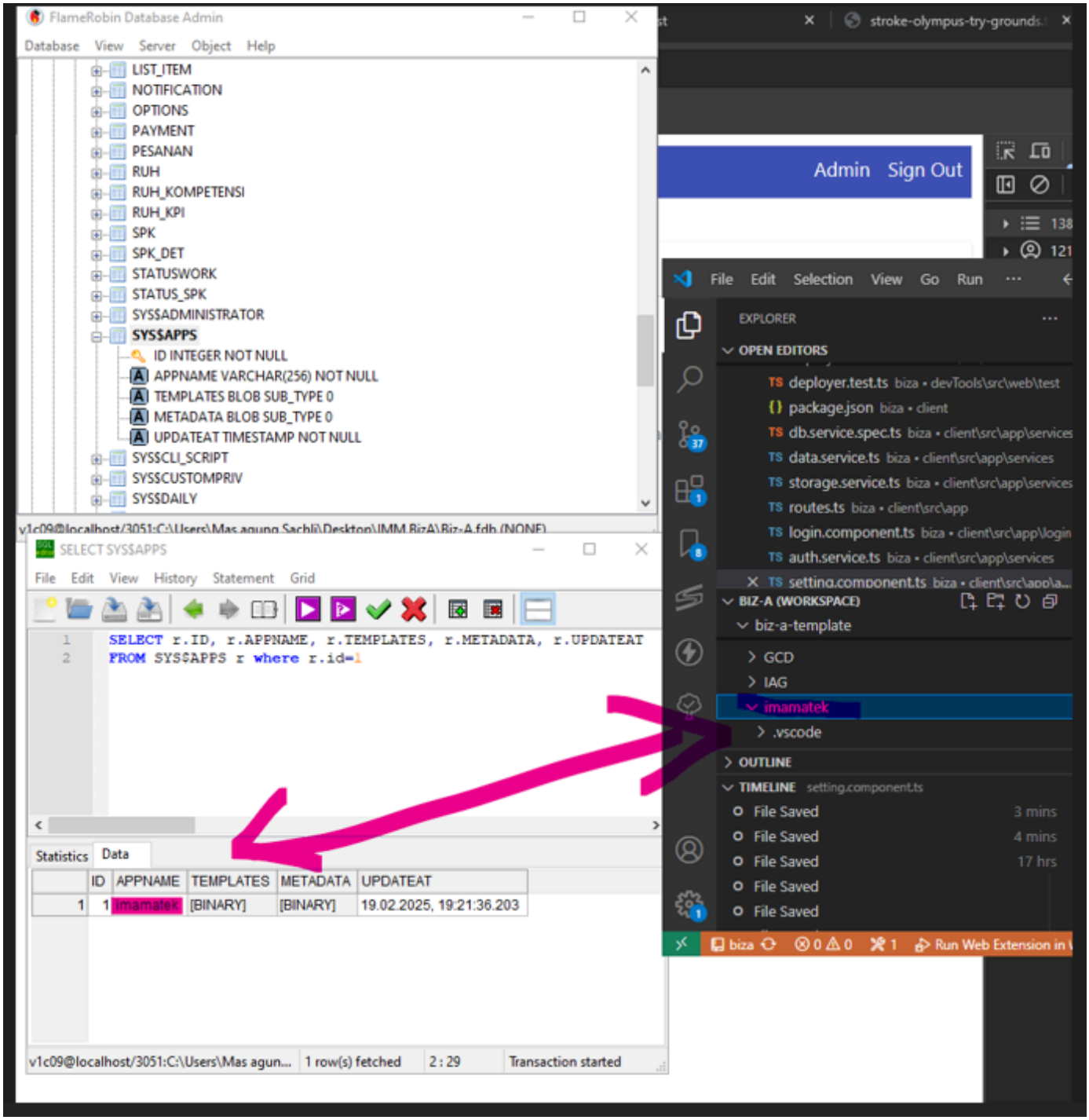


➔ Deployment Library (Ada perubahan file library yang dipakai di form / list yang sedang terbuka di Biz-A)



Note :

- Bila Biz-A Apps yang dibuat sudah selesai, maka tetap harus melakukan proses upload script melalui Biz-A CLI, agar bisa dipakai oleh end user. Dengan kata lain perubahan script yang ada hanya disimpan sementara di storage pada browser yang membuka Biz-A Client.
- Fitur ini bekerja secara realtime. Jadi Biz-A Client bisa ada di di jaringan manapun, dan VsCode nya bisa di jaringan manapun.
- Relasi antara Biz-A DevTools dan Biz-A CLient adalah many-to-many. Jadi bisa pakai lebih dari satu vsCode dan lebih dari satu Biz-A Client.
- Bila menggunakan VsCode Web ditambah bisa mengakses Repository, maka Anda bisa bekerja untuk membuat / merubah Script Biz-A dari IOT manapun asalkan IOT tersebut bisa membuka VsCode Web dan memiliki koneksi internet.
- Proses reloader pada Biz-A Client hannya akan melakukan reload apabila script yang mengalami perubahan tersebut merupakan bagian dari form / list yang sedang aktif di Biz-A Client.
- Syarat & kondisi untuk Biz-A DevTools mengirimkan script ke Biz-A Client :
    - Nama Folder dari file script yang sedang diedit sesuai dengan nama App yang ada di database BIz-A (table SYS$APPS.AppName)



- Terdapat perubahan script pada file tersebut.
- Minimal harus ada 1 buah Biz-A client yang login.
- Biz-A CLI harus aktif.
- Biz-A DevTools harus memiliki koneksi ke Biz-A CLI dan Biz-A Hub. Minimal satu koneksi harus berhasil.

Debugger

1. Aktifkan DevTools di Browser yang kalian pakai
2. Tambahkan kata ***debugger*** di dalam script template yang kalian mau debug
3. Upload script template tersebut ke Biz-A
4. Login Biz-A Client, dan buka form / list yang akan menggunakan script template tersebut.
5. DevTools browser akan langsung berhenti, dan silahkan dipakai semua fungsi DevTools Browser yang ada seperti, watcher, variable value, callStack, ataupun breakpoint event. Terdapat banyak hal yang bisa membantu kalian melakukan debugging.

Note : untuk cara memakai devTools di masing - masing browser, silahkan merujuk ke homepage yang sudah disediakan oleh pembuat browser tersebut. Contoh : devTools Chrome

Bab 8. Proxy Lokal & FINA Hub

Proxy Lokal

Apabila script di Biz-A perlu request ke sebuah API server namun terkendala dengan error CORS (Cross Origin Resource Sharing), maka solusi yang akan digunakan adalah menggunakan proxy.
Untuk API server yang mempunyai public IP, silakan menggunakan apiSvc.proxy yang dibahas di bab 6.

Keterbatasan apiSvc.proxy adalah ia hanya bisa mengakses API server dengan IP Public. Sedangkan untuk IP lokal tidak bisa.

Bila Biz-A client perlu mengakses API server di IP lokal, maka perlu menginstall proxy server di jaringan lokal.

Note: Gunakan biz-a-cli mulai versi 2.3.2 ke atas.

Aplikasi proxy lokal ini sudah ada di dalam biz-a-cli.
Cara meng-install biz-a-cli dapat dilihat di Bab 4 sub bab **Install biz-a-cli**.
Setelah biz-a-cli ter-install, dari cmd ketik **proxy**.
Port default yang akan digunakan adalah 3000. Apabila ingin mengganti port, gunakan argumen **-p**.

Contoh: **proxy -p 3001**.

Setiap request yang dilakukan harus menyertakan header: `{target:ip_api}` di mana ip_api diganti dengan IP API server.

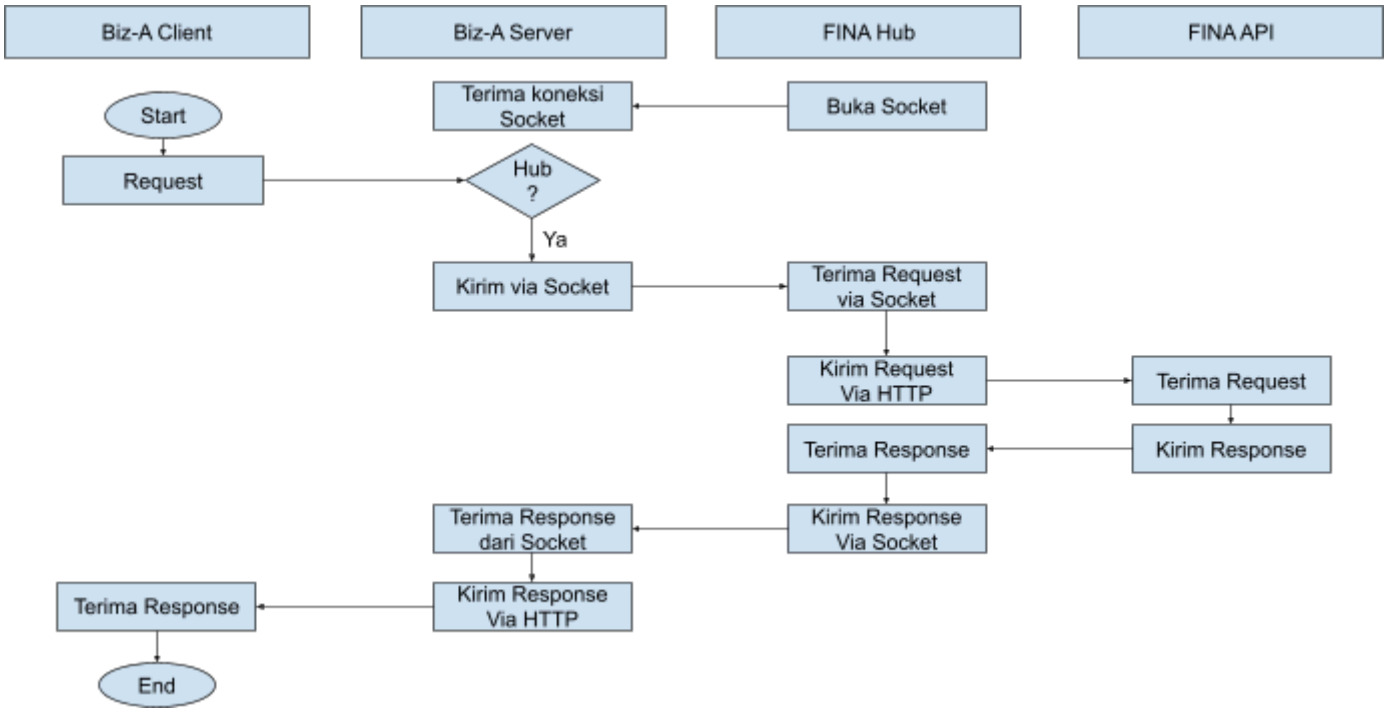
Contoh request POST menggunakan proxy:

```
let options = {headers:{target:'http://ip_api_server'}}
this.dataService.httpPostWithSpinner(
      'http://ip_proxy:83/scanlog/new?sn=61',
      {},
      this.options
).subscribe(data=>{
      if (!data.Result){
            alert('tidak ada data');
      } else {
            console.log(data.data);
      }
})
```

FINA Hub

Apabila server tempat FINA API berada tidak mempunyai IP Public / VPN, maka kita bisa menggunakan FINA Hub sebagai solusi alternatif.

Cara kerja FINA Hub adalah sebagai berikut:



Cara setup FINA Hub:

1. Install biz-a-cli.
    a. Bila sebelumnya sudah pernah install, pastikan versinya di atas 2.3. Cara cek versi: uploadbiza --version.

b. Bila masih versi lama, uninstall dulu dengan cara: npm uninstall biz-a-cli

2. FINA API ***HTTP*** => Ketik hub --server **[http_server]** --sub **[subdomain]** --hostname **[ip_API]** --port **[port_API]**
   a. [http_server] diganti dengan url Biz-A Server https://server.biz-a.id
   b. [subdomain] diganti dengan nama unik..
   c. [ip_API] diganti dengan IP lokal dari FINA API (IP saja, tanpa http). Bila hub dan FINA API berada di server yang sama, bisa gunakan **localhost**.
   d. [port_API] diganti dengan port yang digunakan oleh FINA API.

   Contoh:
   ```
   hub --server https://server.biz-a.id --sub imamatek --hostname localhost --port 212
   ```

3. FINA API ***HTTPS*** => Ketik hub --server **[http_server]** --sub **[subdomain]** --hostname **[ip_API]** --port **[port_API]** *--secure*

   a. [http_server] diganti dengan url Biz-A Server https://server.biz-a.id
   b. [subdomain] diganti dengan nama unik.
   c. [ip_API] diganti dengan FQDN (sesuai dengan ssl yang dipakai) dari FINA API (tanpa https).
   d. [port_API] diganti dengan port yang digunakan oleh FINA API.

   Contoh:
   ```
   hub --server https://server.biz-a.id --sub imamatek --hostname localhost --port 212 —-secure
   ```

4. FINA API ***Publish to Internet (--publish)***
   a. Secara default bernilai true
   b. Silahkan isikan dengan false, bila tidak diperlukan karena DirectHub mau diakses secara local dengan SSL sendiri

   Contoh:
   ```
   hub --server https://server.biz-a.id --sub imamatek --hostname localhost --port 212 —-publish false
   ```

5. FINA API ***Hub server url (--hub)***
   a. Secara default bernilai `https://biz-a-hub-ed77b066db87.herokuapp.com/`
   b. Silahkan isikan dengan url lain bila diperlukan

   Contoh:
   ```
   hub --server https://server.biz-a.id --sub imamatek --hostname localhost --port 212 —-hub https://my.BizA.hub
   ```

Cara mengetes apakah FINA Hub sudah berjalan dengan baik:

https://server.biz-a.id/hub/fina/rest/TMfgMethod/sql_data_to_json/select * from company/1?subdomain=[subdomain]

Ganti [subdomain] dengan subdomain yang tepat.

Untuk mengurangi pengetikan, silakan buat file dengan berekstensi .bat.
Caranya:

1. Buat sebuah file baru dengan nama finahub.bat (nama bebas, yang penting berekstensi bat).
2. Bila file tersebut dengan notepad (atau text editor apa pun).
3. Ketik: `hub -s https://server.biz-a.id --sub imamatek -h localhost -p 212` di dalam file tersebut.
4. Simpan file
5. Untuk menjalankannya tinggal ketik: finahub atau finahub.bat.

Untuk menjalankan FINA Hub setiap kali komputer menyala, tempatkan file bat tersebut di dalam start up.
Caranya:

1. Tekan windows key, lalu ketik **run** lalu tekan enter.
2. Ketik di dalam window run: **shell:startup** lalu tekan enter
3. Akan muncul file explorer. Klik kanan di dalam box-nya, pilih **New | Short cut**. Pilih file bat yang akan dijalankan.
4. Klik OK.

WatchDog FINA Hub

Bagian ini ditambahkan dikarenakan adanya hub yang mati dikarenakan adanya error yang tidak tertangani oleh kode sehingga dimunculkan oleh Node.
Saat ini diambil 2 cara, yaitu sisi **kode** dan sisi **worker**. Keduanya bisa dikombinasikan jika diperlukan.

Cara kode adalah dengan menambahkan :

```
process.on('uncaughtException', (err) => {  //debug
    console.log('Unhandled Exception:', err);
    logger.error('Unhandled Exception:', err) //(jika pakai winston)
});

process.on('unhandledRejection', (err) => {  //debug
    console.log('Unhandled Rejection:', err);
    logger.error('Unhandled Rejection:', err)  //(jika pakai winston)
});
```

pada file utama seperti index.js atau app.js atau server.js

Cara worker adalah dengan install **@socket.io/pm2** pada npm global.

- [ ] Ketik pada cmd: **npm install @socket.io/pm2 -g**
- [ ] Ganti folder ke folder bin. Contoh: cd “C:\Users\Operational\AppData\Roaming\npm\node_modules\biz-a-cli\bin”
- [ ] Jalankan: pm2 start <file yang akan dipanggil>--watch --ignore-watch="node_modules" –<set nama yang diinginkan> -- <isi dengan argument pada file terpanggil>
  Contoh: pm2 start ./hub.js --watch --ignore-watch="node_modules" –time --name="scy sp3000 d2" -- --server http://localhost:3000 --sub scy --hostname localhost --port 212 -d 2 --serverport 3010
- [ ] Untuk melihat list apa saja yang aktif bisa gunakan “pm2 list”
- [ ] Untuk mematikan atau menyalakan aplikasi yang dijaga, gunakan “pm2 start/stop <id atau nama aplikasi yang sudah diset>”
- [ ] Untuk membuang aplikasi yang dijaga, gunakan “pm2 delete <id atau nama aplikasi yang sudah diset>”
- [ ] Untuk memonitor aplikasi yang aktif dan juga melihat log yang dikerjakan, bisa menggunakan “pm2 monit”. Gunakan seperlunya karena kalau melihat percobaan pada IOS, agak memakan resource CPU.
- [ ] Untuk melihat semua log yang ada di dalam command prompt CLI hub yang berjalan, bisa melihat di C:\Users\<nama login>\.pm2\logs
- [ ] Untuk lebih lengkap, bisa melihat https://pm2.keymetrics.io

Bab 9. Aturan Pembuatan Database

Wajib

Tabel

1. Nama field Primary Key-nya harus ID dengan tipe integer not null.
2. Nama field Foreign Key Detail ke master harus [nama_tabel_master]_ID.
3. Tabel Detail harus mempunyai Primary Key ID sendiri.

Generator

1. Setiap tabel yang mempunyai ID wajib mempunyai generator sendiri.
2. Nama generatornya harus [nama_tabel]_GEN.

Trigger

1. Setiap tabel yang mempunyai field ID harus mempunyai trigger BEFORE INSERT
2. Code di before insert ini adalah untuk mengisi ID dengan mengambil nilai dari generator.
3. Code-nya adalah: `if (new.id is null) then new.id = gen_id(nama_gen, 1);`

Disarankan

1. Nama Foreign Key dimulai dengan nama_table_yg_dituju_ID
2. Nama tabel detail: Nama_tabel_master_DET.

Bab 10. Hak Akses

Pendahuluan

Detail requirement untuk fungsi hak akses di Biz-A dapat dilihat di sini.

Bab 11. Management by Exception & Callback

Manual book lebih rinci ada di sini.

Bab 12. Struktur Parameter

IApiConfig

```
IApiConfig = {
    dbIndex     : number,
    url         : string,
    methodClass?: string,
    useProxy?   : boolean,
    finaDbIndex?: string,
    subdomain?  : string,
    defaultForm?: string,
}
```

Bab 13. Contoh Code

Contoh Membuat Field menghilang dalam kondisi tertentu sesuai isi modelnya

```
  {
        className: "col-auto",
        type: "button",
        props: {
            color: "primary",
            variant: "raised",
            icon: "done",
            text: "Save",
            onClick: function () {
                this.f().saveButton();
            },
        },
        expressions: {
            hide: "model.view == true"
        }
```

```
    },
    {
        className: "col-2",
        key: "picothers",
        type: "input",
        props: {
            label: "PIC",
            required: true,
            placeholder: "Enter the PIC",
        },
        expressions: {
            hide: "model.pic != 0"
        }
    }
```

Contoh di detail-grid untuk hitung otomatis saat ada perubahan di qty ataupun Harga

```
get = function () {
  return {
    model: {},
    tableName: "",
    fields: [
      {
        type: "detail-grid",
        props: {
          tableName: 'pesanan',
          detailKey: 'pesanan_det',
          addBtn: {
            show: true,
            icon: 'add'
          },
          deleteBtn: {
            show: true,
            icon: 'delete',
            label: "Hapus",
            key: "delete"
          },
          columns: [
            { data: "id" },
            { data: "kode", title: "Kode Produk", readonly: false },
            { data: "qty", title: "Qty", type: "integer", readonly: false },
            { data: "nama_produk", title: "Nama Produk", readonly: false },
```

```
            { data: "price_unit", title: "Harga/Unit", type: "money", readonly: false },
            { data: "jumlah", title: "Jumlah", type: "money", readonly: true
}
          ],
          watchKey: "id",
          onCalc: [
            {
              watchKey: ["qty", "price_unit"],
              function: function (rowControls) {
                const qty = rowControls.get("qty")?.value
                const price_unit = rowControls.get("price_unit")?.value
                const hasil = qty * price_unit
                rowControls.get("amount").setValue(hasil);
                this.f().hitungTotal(); //function yang ada di functions
              }
            }
          ]
        },
      },
      {
        key: "subtotal",
        type: "input",
        props: {
          label: "Subtotal",
          placeholder: "Subtotal",
          disabled: true
        }
      },
      {
        key: "diskon",
        type: "input",
        props: {
          label: "Diskon",
          placeholder: "Diskon"
        }
      },
      {
        key: "total",
        type: "input",
        props: {
          label: "Total",
          placeholder: "Total",
          disabled: true
        }
      },
```

```
    ],
    functions: {
      onInit: function () {
        this.form.get("diskon").valueChanges.subscribe(e => {
          this.f().hitungTotal();
        })
      },
      hitungTotal: function () {
        //lakukan perhitungan
      }
    }
  }
}
```

Contoh Kirim Email di BIZA, setelah simpan ke database.

```
this.dataService
  .sendModel(this.model, this.apiConfig, "applicant")
  .pipe(
    this.map((e) => {
      if (e.success == "OK") {
        let script = email_user.map((dt_email) => {
          let findUser = user.find(
            (e) => e.userid == dt_email.usercreate
          );
          let script2 = /#username#/g.test(script_email)
            ? script_email.replace(/#username#/g, findUser.username)
            : script_email;

          const contentText = /#breakLine#/g.test(script2)
            ? script2.replace(/#breakLine#/g, `\n`)
            : script2;

          const contentHTML = /#breakLine#/g.test(script2)
            ? script2.replace(/#breakLine#/g, `<br>`)
            : script2;
          return {
            companyname: company,
            mailOptions: {
              text: contentText,
              html: contentHTML,
              subject: `Auto Generated email Biz-A (Permintaan Pembelian
baru)`,
              to: `${findUser?.username} <${dt_email.email}>`,
```

```
              from: "Biz-A <biz-a@imamatek.com>",
            },
          };
        });

        return script;
      } else {
        throw ("applicant data failed to save");
      }

    }),
    this.mergeMap((data) => {
      return this.from(data).pipe(
        this.concatMap((dt) =>
          this.appService.sendMail(this.appConfig, dt)
        ),
        this.toArray(),
        this.catchError((err) => {
          alert("Email ke gudang gagal terkirim!");
          return this.of({ error: err.error });
        }),
      );
    })
  )
  .subscribe(e => {
    alert("Data Saved Successfully!")
    this.router.navigate(["../list", "appealList"]);
  })
```

Contoh Untuk readonly field tipe money, select, & date

```
{
  key: 'total',
  type: 'money',
  props: {
    label: 'Total',
    disabled: true
  }
}
```

Contoh SQL membuat table, generator dan trigger sekaligus bisa menggunakan berikut.

Contoh berikut ini adalah contoh membuat table template

```
SET TERM ^ ;
CREATE TABLE TEMPLATE(
  ID INTEGER NOT NULL,
  FILENAME VARCHAR(256) NOT NULL,
  TEMPLATE BLOB SUB_TYPE 0,
  PRIMARY KEY (ID)
)^
CREATE GENERATOR TEMPLATE_GEN^
CREATE TRIGGER TEMPLATE_BI FOR TEMPLATE ACTIVE
BEFORE INSERT POSITION 0
AS BEGIN
  new.ID=GEN_ID(TEMPLATE_GEN,1);
END^
SET TERM ; ^
```

Petunjuk Khusus

1. Hooks tidak boleh didefinisikan langsung di dalam fields. Seluruh implementasi hook harus ditempatkan di dalam functions. Pisahkan antara logika dengan tampilan (fields)
2. Pada fields bertipe detail-grid, properti onCalc tidak memerlukan penggunaan bind(this).
3. Selain menggunakan this.model, kita juga dapat menyimpan data yang akan dikirim ke database ke dalam sebuah variabel berbentuk object. Catatannya, nama key dalam object tersebut harus sama dengan nama fields pada tabel. Misalnya, terdapat relasi master-detail sebagai berikut:

Tabel pesanan memiliki field: id, nama, subtotal, diskon, total
Tabel pesanan_det memiliki field: id, pesanan_id, nama_produk, qty, jumlah, price_unit, kode
Maka, jika disimpan dalam sebuah variabel, strukturnya akan menjadi seperti berikut:

```
get = function () {
  return {
    model: {},
    tableName: "",
    fields: [
      {
        key: "nama",
        type: "input",
        props: {
          label: "Nama",
          placeholder: "Nama",
        }
```

```
      },
      {
        type: "detail-grid",
        props: {
          tableName: 'pesanan',
          detailKey: 'pesanan_det',
          addBtn: {
            show: true,
            icon: 'add'
          },
          deleteBtn: {
            show: true,
            icon: 'delete',
            label: "Hapus",
            key: "delete"
          },
          columns: [
            { data: "id" }, //tidak perlu ditambahkan label untuk id
            { data: "kode", label: "Kode Produk", readonly: false },
            { data: "qty", label: "Qty", type: "integer", readonly: false },
            { data: "nama_produk", label: "Nama Produk", readonly: false },
            { data: "price_unit", label: "Harga/Unit", type: "money",
readonly: false },
            { data: "jumlah", label: "Jumlah", type: "money", readonly: true
}
          ],
          watchKey: "id",
          onCalc: [
            {
              watchKey: ["qty", "price_unit"],
              function: function (rowControls) {
                const qty = rowControls.get("qty")?.value
                const price_unit = rowControls.get("price_unit")?.value
                const hasil = qty * price_unit
                rowControls.get("amount").setValue(hasil);
                this.f().hitungTotal(); //function yang ada di functions
              }
            }
          ]
        },
      },
      {
        key: "subtotal",
        type: "input",
        props: {
```

```
          label: "Subtotal",
          placeholder: "Subtotal",
          disabled: true
        }
      },
      {
        key: "diskon",
        type: "input",
        props: {
          label: "Diskon",
          placeholder: "Diskon"
        }
      },
      {
        key: "total",
        type: "input",
        props: {
          label: "Total",
          placeholder: "Total",
          disabled: true
        }
      },
      {
        type: 'button',
        props: {
          text: 'Simpan Pesanan',
          color: 'primary',
          variant: 'raised',
          onClick: function ($event) {
            this.f().savePesanan();
          }
        }
      }
    ],
    functions: {
      onInit: function (id) {
        this.ID = id && id != "null" ? id : null
        this.form.get("diskon").valueChanges.subscribe(e => {
          this.f().hitungTotal();
        })
      },
      hitungTotal: function () {
        //lakukan perhitungan
        //untuk mendapatkan nilai dari detail-grid, gunakan
this.form.value.pesanan_det
```

```
      },
      savePesanan: function () {
        const dataKirim = {
          id: this.ID,
          nama: this.model.nama,
          subtotal: this.model.subtotal,
          diskon: this.model.diskon,
          total: this.model.total,
          pesanan_det: this.form.value.pesanan_det.map(e => ({
            id: e.id,
            //pesanan_id: this.ID, //tidak perlu dituliskan kolom
pesanan_id, jika kirim bersamaan dengan masternya
            kode: e.kode,
            qty: e.qty,
            nama_produk: e.nama_produk,
            price_unit: e.price_unit,
            jumlah: e.jumlah,
          }))
        }

        return this.dataService.sendModel(dataKirim, this.config.api,
'pesanan').subscribe(e => {
          //pengiriman tersebut bisa langsung menggunakan sendModel dan
dapat menyimpan di kedua table (maksimal hanya dua table) dengan syarat
field kedua dari table detail adalah nama_table_master_det (contohya
pesanan_det).
        })
      }
    }
  }
}
```

Contoh Cara Menyimpan File ke tabel

Untuk penyimpanan file tidak bisa menggunakan this.dataService.sendModel. File harus terlebih dahulu dikonversi ke base64, kemudian base64 tersebut dikonversi Kembali dari binary to ASCII, baru dikirim menggunakan this.dataService.updateImg(). Wajib dilakukan pengambilan id dengan genid karena akan digunakan untuk keyValue dalam simpan base64. Berikut untuk simpan dan lihat file:

```
get = function () {
  return {
    model: {},
    tableName: "", //Property tableName cukup dikosongkan (isi dengan string
kosong "") jika dalam property fields terdapat type input-file. Jadi, tidak
perlu diisi dengan nama table.
    fields: [
```

```
      {
        key: 'bukti_transfer',
        type: 'input-file',
        props: {
          label: 'Upload Bukti Transfer',
          filetype: '.jpg, .jpeg, .png, .pdf',
        },
        expressions: {
          hide: "model.id == null" //hide ketika this.model.id == null
(Ketika create baru)
        }
      },
      {
        type: "button",
        props: {
          color: "primary",
          variant: "raised",
          icon: "preview",
          onClick: function () {
            this.f().previewBtn("bukti_transfer", this.ID); //untuk melihat
isi file yang diinput
          },
        },
      },
      {
        template: "<br><br>"
      },
      {
        className: "auto-caps",
        key: "cust",
        type: "input",
        props: {
          label: "Perusahaan Customer",
          placeholder: "Masukan Perusahaan Customer",
          required: true
        },
      },
      {
        className: "auto-caps",
        key: "custname",
        type: "input",
        props: {
          label: "Nama Customer",
          placeholder: "Masukan Nama Customer",
          required: true
```

```
        },
      },
      {
        className: "auto-caps",
        key: "salesname",
        type: "input",
        props: {
          label: "Nama Sales",
          placeholder: "Masukan Nama Sales",
          required: true
        },
      },
      {
        className: "auto-caps",
        key: "project",
        type: "input",
        props: {
          label: "Project",
          placeholder: "Masukan Project",
          required: true
        },
      },
      {
        template: "<br><br>"
      },
      {
        className: "col-4",
        type: "button",
        props: {
          text: "Simpan",
          variant: "flat",
          color: "primary",
          onClick: function () {
            if (this.form.valid) {
              this.f().saveBtn(this.model)
            } else {
              alert("Anda perlu mengisi perusahan dan nama customer, nama
sales, dan project!")
            }
          },
        },
      },
    ],
    functions: {
      onInit: function (id) {
```

```
        this.USERID = this.storageService.getCurrentUser();
        this.IMG_BASE_64 = ""
        this.ID = id && id != "null" ? id : null
        this.model.id = this.ID
        this.model = structureClone(this.model)

        this.dbIndexBiza = this.config.api.dbIndex
        this.dbIndexFina = this.config.api.finaDbIndex
      },
      saveBtn: function (model) {
        let target = this.form.value?.bukti_transfer;
//this.form.get('bukti_transfer')?.value wajib pakai optional chaining
operator
        const reader = new FileReader();
        reader.readAsDataURL(target);
        reader.onload = (e) => {
          this.IMG_BASE_64 = e.target.result.split(",")[1]; //ambil data
base64 nya setelah koma
          this.IMG_TYPE = e.target.result.split(",")[0]; //ambil data file
type nya sebelum koma

        }

        if (!model.cust) throw new Error("Anda belum mengisi customer yang
dikunjungi!")
        let progressRef = this.loadingService.start("Simpan data",
"indeterminate")

        //cek jika ID ada maka tidak perlu genID
        return (this.ID ? this.of(this.ID) :
this.dataService.genId('VISITCLIENT_GEN', this.apiConfig)).pipe(
          this.map(id => {
            this.ID = id; //simpan id table visitclient ke variabel global

            let senddata = {
              id,
              userid: this.USERID,
              cust: model.cust ? model.cust?.toUpperCase() : null,
              custname: model.custname ? model.custname?.toUpperCase() :
null,
              salesname: model.salesname ? model.salesname?.toUpperCase() :
null,
              project: model.project ? model.project?.toUpperCase() : null,
              lat_in: model.lat_in,
              lon_in: model.lon_in,
```

```
            address_in: model.alamat,
            checkin_time: this.dayjs().format('YYYY-MM-DD HH:mm:ss'),
            arrival_date: this.dayjs().format("YYYY-MM-DD"),
          }
          this.model = structuredClone(this.model)
          return senddata
        }),
        this.mergeMap((senddata) => {
          return this.dataService.sendModel(senddata, this.apiConfig,
"visitclient")
        }),
        this.mergeMap(() =>
          this.dataService.genId('VISIT_IMG_GEN', this.apiConfig)
        ),
        this.mergeMap((id) => {
          this.ID_IMG = id; //simpan id table gambar ke variabel global
          let senddata = {
            id,
            visitclient_id: this.ID,
            img_type: this.IMG_TYPE
          }

          return this.dataService.sendModel(senddata, this.apiConfig,
"visit_img")
        }),
        this.mergeMap(() => {
          let obj = {
            dbIndex: this.dbIndexBiza,
            tableName: "visit_img",
            imgCol: "img",
            keyCol: "id",
            keyValue: this.ID_IMG,
            img: window.btoa(this.IMG_BASE_64),
          };

          return this.dataService.updateImg(this.apiConfig, obj);
        }),
        this.finalize(() => this.loadingService.stop(progressRef)),
      ).subscribe(e => {
        if (e.success == "OK") {
          alert("Berhasil Simpan Data Checkin!")
          this.loadingService.stop(progressRef)
          this.router.navigate(["./main"]);
        } else {
          this.loadingService.stop(progressRef)
```

```
            alert("Terdapat Error!")
          }
        })
      },
      previewBtn: function (key, id) {//fungsi untuk membuat preview file
        if (id) { //jika bukan berubah create baru
          this.f().getDoc(id, key) //ambil data dokumen di database
            .subscribe(e => {
              if (e.length > 0) {
                const hasil = e.map(d => this.f().formatListFile(d))
                const ref = this.dialog.open(
                  this.FormlyDialogComponent,
                  this.f().dialogConfigTampilan(
                    hasil.map(s => s.full)
                  )
                );
              } else {
                alert("Tidak terdapat file!")
              }
            })
        } else { //saat create baru
          const reader = new FileReader();
          let hasil = ""
          if (target) {
            reader.readAsDataURL(target);
            reader.onload = (e) => {
              hasil = this.f().formatListFile({ id: null, full:
e.target.result })
            }

            const ref = this.dialog.open(
              this.FormlyDialogComponent,
              this.f().dialogConfigTampilan(
                hasil.map(s => s.full)
              )
            );
          } else {
            alert("Tidak terdapat file!")
          }
        }
      },
      formatListFile: function (data) {
        return {
          id: data.id,
          full: `${data.img_type},${data.data}`
```

```
  };
},
getDoc: function (visitclient_id, jenis) {
  const params = {
    filter: [],
    columns: [
      { data: "visit_img.id", key: "id" },
      { data: "visit_img.visitclient_id", key: "visitclient_id" },
      { data: "visit_img.img_type", key: "img_type" },
      { data: "visit_img.img", key: "img" },
      { data: "visit_img.jenis", key: "jenis" },
    ],
  };

  if (visitclient_id) {
    params.filter.push({
      junction: "",
      column: "visit_img.visitclient_id",
      operator: "=",
      value1: visitclient_id,
    });
  }

  if (jenis) {
    params.filter.push({
      junction: "and",
      column: "visit_img.jenis",
      operator: "=",
      value1: `'${jenis}'`,
    });
  }


  return params;
},
dialogConfigTampilan: function (url) {
  const images =
    typeof url == "object" && url?.length > 0
      ? url.map((l) => ({ iframe: `${l}` }))
      : [{ iframe: `${url}` }];

  const config = new this.MatDialogConfig();
  config.width = `800px`;
  config.height = `600px`;
  config.data = {
```

```
          caption: `Documents`,
          captionOK: "OK",
          captionCancel: "Cancel",
          model: {},
          fields: [
            {
              className: "change-width",
              type: "swiper",
              props: {
                images,
                params: {
                  navigation: true,
                  pagination: true,
                  scrollbar: false,
                },
                attributes: {
                  style: `max-width: 800px; max-height: 600px;`,
                },
              },
            },
          ],
        };

        return config;
      }
    },
  };
};
```

10. Object yang dikirim ke database menggunakan sendModel, di field database harus ada juga property tersebut.

11. RxJS startWith belum ada di dalam komponen this (this.startWith). Harus menggunakan this.rx.get.startWith.
12. Property dbIndex dalam type lookup tidak dapat diisi dengan this.config.api.dbIndex di dalam fields. Sehingga bisa didefinisikan di dalam onInit dengan menuliskan sebagai berikut

```
get = function () {
  return {
    model: {},
    tableName: "",
    fields: [
      {
        key: "customerid",
        type: "lookup",
        props: {
```

```
          label: "Customer",
          placeholder: "Customer",
          lookupOptions: {
            table: "persondata", //ambil data dari database fina
            key: "id",
            display: "personname",
            dbIndex: 2 //wajib ditulis dengan angka. Tidak boleh jika value
dbIndex diisi dengan this.config.api.dbIndex
          }
        }
      }
    ],
    functions: {
      onInit: function (id) {
        this.ID = id && id != "null" ? id : null
        let lu_p = this.fields[0]._componentRefs[0].instance
//this.fields[0] adalah posisi lookup berada di dalam property fields.
        lu_p.props.lookupOptions = { ...lu_p.props.lookupOptions, dbIndex:
this.config.api.dbIndex }; //jika menggunakan db biza
      }
    }
  }
}
```

13. Jangan menggunakan async await untuk mengirim data ke database.
14. Jangan menggunakan typescript. Gunakan JavaScript.
15. Di dalam tipe selain detail-grid, tidak bisa mengakses 'this' pada property fields jadi pindahkan ke functions. Jangan gunakan onChange dalam semua field.
16. Tidak boleh menggunakan this.form.patchValue({ nomor_pesanan: id }) untuk meng-update form/model. Hanya gunakan this.form.get("nomor_pesanan").setValue(id) atau this.model.nomor_pesanan = id
17. Tidak boleh menggunakan field.formControl.valueChanges pada expressions di fields. Hanya gunakan valueChanges di onInit.
18. Contoh penambahan button create, edit dan delete saat kondisi atau status tertentu pada list.

```
get = function () {
  return {
    caption: 'Daftar Master Mitra',
    columns: [
      { title: 'Nama', data: 'MASTER_MITRA.NAMA' }, //wajib ada { data:
"...", title: "..." }
      { title: 'Nama PIC', data: 'MASTER_MITRA.pic_id.id.pic.nama' },//jika
ingin mengambil dari table lain
      {
        title: '',
```

```
        data: 'MASTER_MITRA.ID',
        button: [
          {
            caption: "Edit",
            icon: "edit",
            onClick: "editClicked",  //wajib hanya ditulis nama fungsinya
saja. tidak boleh ada code pada columns
          },
          {
            caption: 'Delete',
            icon: 'delete',
            color: 'warn',
            onClick: 'confirmDelete' //wajib hanya ditulis nama fungsinya
saja. tidak boleh ada code pada columns
          }
        ]
      }
    ],
    filter: [],
    functions: {
      editClicked: function (id) {
        // const data = JSON.stringify({ id, edit: true, view: false })
//untuk yang ingin penambahan status lainnya

        this.f().getStatus(id).subscribe(e => { //digunakan untuk mengecek
status, jika status tidak sesuai maka tidak bisa edit
          if (e.length > 0) {
            if (e[0].status_mitra == "Baru") {
              this.router.navigate(["../../form", "mitraForm", id]);
            } else {
              alert("Data tidak boleh di Edit!")
            }
          } else {
            alert("Data tidak ditemukan!")
          }
        })
      },
      confirmDelete: function (id) {
        if (confirm('Yakin ingin menghapus mitra ini?')) {
          this.f().getStatus(id).subscribe(e => { //digunakan untuk mengecek
status, jika status tidak sesuai maka tidak bisa dihapus
            if (e.length > 0) {
              if (e[0].status_mitra == "Baru") {
                this.dataService
                  .deleteModel(id, this.config.api, 'master_mitra')
```

```
                  .pipe(this.switchMap(() => this.loadData())) //fungsi
loadData adalah fungsi bawaan biza
                  .subscribe({
                    next: () => {
                      alert('Data berhasil dihapus');
                    },
                    error: (err) => {
                      alert('Gagal menghapus: ' + JSON.stringify(err))
                    }
                  });
              } else {
                alert("Data tidak boleh dihapus!")
              }
            } else {
              alert("Data tidak ditemukan!")
            }
          })

        }
      },
      getStatus: function (id) {
        const param = {
          filter: [{ junction: "", column: "mitra.id", operator: "=",
value1: id }],
          columns: [
            { data: "mitra.status_mitra", key: "status_mitra" }, //wajib {
data: "...", key: "..." }
          ]
        };

        return this.dataService.queryData(param, this.config.api, true,
true)
      }
    },
    buttons: [
      {
        caption: 'Create New',
        icon: 'add',
        color: 'primary',
        link: ['../../form', 'mitraform', null]
      }
    ]
  };
};
```

19. Jika saat edit tableName tidak diisi maka bisa melakukan sebagai berikut:

```
get = function () {
  return {
    model: {
      id: null,
      consultantname: null,
      consultantid: null,
      consultantaddress: null,
    },
    tableName: "",
    fields: [
      { key: "id" },
      {
        key: "consultantid",
        type: "input",
        props: {
          label: "ID",
          required: true
        }
      },
      {
        key: "consultantname",
        type: "input",
        props: {
          label: "Name",
          required: true
        }
      },
      {
        key: "consultantaddress",
        type: "input",
        props: {
          label: "Address",
          required: true
        }
      },
      {
        type: "detail-grid",
        props: {
          tableName: "",
          detailKey: "consultant_det",
          columns: [
            { key: "id" },
            { key: "jenis", label: "Jenis" },
          ],
```

```
        }
      },
      { template: "<br>" },
      {
        fieldGroupClassName: "row mb-3 justify-content-center",
        fieldGroup: [
          {
            className: "col-auto",
            type: "button",
            props: {
              color: "warn",
              variant: "raised",
              icon: "cancel",
              text: "Cancel",
              onClick: function () {
                this.router.navigate(["../list", "consultantList"]);
              },
            },
          },
          {
            className: "col-auto",
            type: "button",
            props: {
              color: "primary",
              variant: "raised",
              icon: "done",
              text: "Save",
              onClick: function () {
                if (this.form.valid) {
                  this.f().saveButton();
                } else {
                  alert("You need to fill in the required fields!")
                }
              },
            },
          },
        ]
      }
    ],
    functions: {
      onInit: function (id) {
        this.DBINDEXBIZA = this.config.api.dbIndex
        const param = JSON.parse(id) ? JSON.parse(id) : id //parse jika dari
list param yang dikirim JSON.stringify({id: id, edit: true})
        this.ID = typeof param == "object" ? param.id : param
```

```
        this.grid = this.fields.find(f => f.type ===
"detail-grid")._componentRefs[0].instance.grid;
        if (param?.edit) {
          this.fields[3].props.disabled = true; //jika type fields select,
date, lookup, money maka readonly diganti dengan disabled
          this.model = structuredClone(this.model)
        }


        if (this.ID) {
          this.f().editMode(this.ID)
        }
      },
      json: function (form, id) {
        return {
          id,
          consultantname: form?.consultantname ? model?.consultantname :
null,
          consultantid: form?.consultantid ? model?.consultantid : null,
          consultantaddress: form?.consultantaddress ?
model?.consultantaddress : null,
          consultant_det: form.consultant_det.map(e => ({
            id: e.id,
            jenis: e.jenis
          }))
        }
      },
      editMode: function (id) {
        let ref = this.loadingService.start(
          " Get Data ... ",
          "indeterminate"
        );


        return this.f().getConsultant(id, this.config.api).pipe(
          this.finalize(() => this.loadingService.stop(ref)),
          this.catchError((err) => {
            this.loadingService.stop(ref);
            // this.notificationService.showError(err)
            // throw err;
            return this.of(err)
          })
        ).subscribe(v => {
          this.DATAAWAL = structuredClone(v)
          this.form.get("consultantid").setValue(v[0]?.consultantid)
```

```
          this.form.get("consultantname").setValue(v[0]?.nama)
          this.form.get("consultantaddress").setValue(v[0]?.address)
          this.grid.assignData({ data: v.map(e => ({ id: e.det_id, jenis:
e.jenis })) }) //wajib menggunakan assignData untuk memasukan data ke table

          this.model = structuredClone(this.model)
        })
      },
      saveButton: function () {
        const kirim = this.f().json(this.form.value, this.ID)


        return this.dataService.sendModel(kirim, this.apiConfig,
"consultant")
          .pipe(
            this.map(res => {
              if (res.success != 'OK') throw new Error("Data cannot be
saved!")

              return res
            }),
          )
          .subscribe(e => {
            if (e?.success == "OK") {
              alert("Consultant data saved successfully")
              this.router.navigate(["../list", "consultantList"]);
            } else {
              alert("Consultant data failed to save")
            }
          })
      },
      getConsultant: function (id, config) {
        const params = {
          filter: [],
          columns: [
            { data: "consultant.id", key: "id" },
            { data: "consultant.consultantname", key: "nama" },
            {
              data: "consultant.consultantid",
              key: "consultantid",
            },
            {
              data: "consultant.consultantaddress",
              key: "address",
            },
```

```
            {
              data: "consultant.id.consultant_id.consultantdet.id",
              key: "det_id",
            },
            {
              data: "consultant.id.consultant_id.consultantdet.jenis",
//wajib ditulis { data: "...", key: "..." }
              key: "jenis",
            },
          ],
        };


        if (id) {
          params.filter.push({
            junction: "",
            column: "consultant.id",
            operator: "=",
            value1: id,
          });
        }


        return this.dataService.queryData(params, config, true, true)
//hasil querydata adalah langsung nilai-nilai yang diinginkan dalam bentuk
array of obj, contoh dari querydata tersebut balikannya adalah [{id:"1",
consultantid:"2", address: "abc", jenis: "a"}]
      },
    }
  }
}
```

20. Gunakan optional chaining operator (model?.nama atau this.form.get(“nama_key”)?.value), jika nilai belum tentu ada.
21. Tidak boleh ada Subscribe di dalam subscribe. Dikhawatirkan akan balapan.
22. Tidak boleh menuliskan dbIndex: this.config.api.dbIndex baik di fields atau di list nanti akan error unexpected end of input
23. Setiap assign ke model, wajib di refresh menggunakan this.model = structureClone(this.model)
24. Jangan menggunakan disabled: () => this.model.mode !== 'create' di fields, karena tidak bisa diubah menjadi false saat run time.
25. Tidak bisa di list menggunakan expressions: { hide: "row['PESANAN.STATUS_PESANAN'] !== 'Siap Kirim'" } untuk hide button.
26. Tidak ada method saat ambil gambar menggunakan this.dataService.getImg() atau /fina/rest/TOrmMethod/get_img.

27. Jika ingin mengambil nilai html dalam tag template bisa menggunakan sebagai berikut:

```
get = function () {
  return {
    model: {
      id: null,
      nomor_pesanan: null,
      nama_penerima: null,
      tanggal_pesan: null,
    },
    tableName: "",
    fields: [
      {
        key: "nomor_pesanan",
        type: "input",
        props: {
          label: "Nomor Pesanan",
          disabled: true
        }
      },
      {
        key: "nama_penerima",
        type: "input",
        props: {
          label: "Nama Penerima",
          disabled: true
        }
      },
      {
        key: "tanggal_pesan",
        type: "date",
        props: {
          label: "Tanggal Pesan",
          disabled: true
        }
      },
      {
        type: "read-only-table",
        props: {
          queryParam: {
            order: [{ column: 1, dir: "asc" }],
            filter: [
              { junction: '', column: 'PESANAN_DET.pesanan_id', operator:
'=', value1: '', value2: '' },
            ],
            columns: [
```

```
            { data: "PESANAN_DET.NAMA_PRODUK", title: "Nama Barang" },
            { data: "PESANAN_DET.QTY", title: "Qty", type: "number" }
          ]
        },
        watchKey: "id",
        filters: [{
          index: 0,
          valueIndex: 1,
          value: "'_A_'",
          replaceKey: '_A_',
        }]
      }
    },
    {
      key: "tanggal_terima",
      type: "date",
      props: {
        label: "Tanggal Terima",
      }
    },
    {
      template: '<div id="signature-container" style="border:1px solid
#ccc; height:150px; margin-top:10px;"></div>'
    },
    {
      type: "button",
      props: {
        text: "Simpan",
        color: "primary",
        variant: "raised",
        onClick: function () {
          this.f().saveKirim();
        }
      }
    }
  ],
  functions: {
    onInit: function (id) {
      if (id && id !== "null") {
        const param = {
          filter: [{ junction: "", column: "pesanan.id", operator: "=",
value1: id }],
          columns: [
            { data: "pesanan.id", key: "id" },
            { data: "pesanan.nomor_pesanan", key: "nomor_pesanan" },
```

```
            { data: "pesanan.nama", key: "nama_penerima" },
            { data: "pesanan.tanggal_order", key: "tanggal_pesan" },
          ]
        };

        this.dataService.queryData(param, this.config.api, true,
true).subscribe(res => {
          if (res.length > 0) {
            const row = res[0];
            this.mode.id = row.id
            this.form.get("id").setValue(row.id);
            this.form.get("nomor_pesanan").setValue(row.nomor_pesanan);
            this.form.get("nama_penerima").setValue(row.nama_penerima);
            this.form.get("tanggal_pesan").setValue(row.tanggal_pesan);

            this.model = structuredClone(this.model);
          }
        });
      }

      setTimeout(() => {
        this.f().initSignaturePad();
      }, 500);
    },

    initSignaturePad: function () {
      const templates = document.querySelectorAll("formly-template");
//wajib search ke formly-template
      const lastTemplate = templates[templates.length - 1];
      //const formly = document.querySelector("formly-template");
      const container = lastTemplate.querySelector("div:last-of-type");
//jangan mencari dengan ID atau pun CLASS jika dari formly-template. Karena
id ataupun class tidak terbentuk. Tidak boleh 'div:last-of-type' diganti
menjadi '#signature-container' atau '.signature-container'
      if (!container) return;

      const canvas = document.createElement("canvas");
      canvas.id = "signatureCanvas";
      canvas.width = container.clientWidth;
      canvas.height = 150;
      canvas.style.border = "none";
      container.innerHTML = "";
      container.appendChild(canvas);

      const ctx = canvas.getContext("2d");
```

```
let isDrawing = false;

const getPos = (e) => {
  const rect = canvas.getBoundingClientRect();
  return {
    x: (e.clientX || e.touches?.[0]?.clientX || 0) - rect.left,
    y: (e.clientY || e.touches?.[0]?.clientY || 0) - rect.top
  };
};

const startDrawing = (e) => {
  isDrawing = true;
  const pos = getPos(e);
  ctx.beginPath();
  ctx.moveTo(pos.x, pos.y);
  e.preventDefault();
};

const draw = (e) => {
  if (!isDrawing) return;
  const pos = getPos(e);
  ctx.lineWidth = 2;
  ctx.lineCap = "round";
  ctx.strokeStyle = "black";
  ctx.lineTo(pos.x, pos.y);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo(pos.x, pos.y);
  e.preventDefault();
};

const stopDrawing = () => {
  isDrawing = false;
  ctx.beginPath();
};

canvas.addEventListener("mousedown", startDrawing);
canvas.addEventListener("mousemove", draw);
canvas.addEventListener("mouseup", stopDrawing);
canvas.addEventListener("mouseout", stopDrawing);

canvas.addEventListener("touchstart", startDrawing);
canvas.addEventListener("touchmove", draw);
canvas.addEventListener("touchend", stopDrawing);
```

```
        this.canvas = canvas;
      },

      saveKirim: function () {
        const signatureData = this.canvas ?
this.canvas.toDataURL("image/png") : null;
        const kirim = {
          id: this.model.id,
          tanggal_terima: tanggal_terima,
          status_pesanan: "Pesanan Selesai"
        };
        const base64 = signatureData.split(",")[1]
        const tipe_data = signatureData.split(",")[0]
        const kirim_ttd = {
          id: null,
          pesanan_id: this.model.id,
          tipe_data
        }

        return this.dataService.sendModel(kirim, this.config.api, "pesanan")
          .pipe(
            this.mergeMap(res =>
              this.dataService.sendModel(kirim_ttd, this.config.api,
"tandatangan")
            ),
            this.mergeMap(() => {
              let obj = {
                dbIndex: this.dbIndexBiza,
                tableName: "tandatangan",
                imgCol: "img",
                keyCol: "pesanan_id",
                keyValue: this.model.id,
                img: window.btoa(base64),
              };

              return this.dataService.updateImg(this.apiConfig, obj);
            }),
          ).subscribe(res => { //cukup satu subscribe aja
            if (res.success === "OK") {
              alert("Data pengiriman berhasil disimpan");
              this.router.navigate(["../list", "pesananlist"]);
            } else {
              alert("Gagal menyimpan data");
            }
          });
```

```
      }
    }
  };
};
```

Note: jangan menggunakan getElementById untuk mencari element dalam template.

28.Tidak ada condition: function (row) { ... } dalam key di button pada list.

```
{
  title: "",
  data: "PESANAN.ID",
  button: [
    {
      icon: "visibility",
      color: "primary",
      caption: "Cek Pesanan",
      condition: function (row) { //tidak ada key tersebut di button
        const status = row["PESANAN.STATUS_PESANAN"];
        return status === "Order Baru" || status === "Telah Direview";
      },
      onClick: function (id) {
        this.router.navigate(["../../form", "pesananform", JSON.stringify({
id: id, status: "Cek Pesanan" })]);
      }
    },
  ]
}
```

29. Contoh jika ID form sudah pasti hanya 1 saja.

```
get = function () {
  return {
    model: {
      id: null,
      email_admin: null
    },
    tableName: "options",
    fields: [
      {
        key: "id"
      },
      {
        key: "email_admin",
        type: "input",
```

```
        props: {
          label: "Email Admin",
          required: true,
          type: "email"
        }
      },
      {
        type: "button",
        props: {
          text: "Simpan",
          color: "primary",
          variant: "raised",
          onClick: function () {
            this.f().save();
          }
        }
      }
    ],
    functions: {
      save: function () {
        const data = { ...this.model, id: 1 };
        this.dataService
          .sendModel(data, this.config.api, "options")
          .subscribe(() => {
            alert("Konfigurasi berhasil disimpan");
          });
      }
    }
  };
};
```

30. Tidak terdapat props.hidden dalam biza.

```
{
  key: "mode",
  type: "input",
  props: {
    hidden: true //tidak ada
  }
}
```

Jika ingin field hidden bisa diubah menjadi

```
{ key: "mode" }
```

31. Gunakan boostrap pada className masing-masing fields agar lebih bagus tampilannya.
32. Variable global di biza dapat menggunakan this.NAMA_VARIABEL. this.NAMA_VARIABEL ini dapat digunakan dalam semua functions.

33. Saat assign data ke label, agar lebih rapih di perbaiki penulisannya seperti berikut ini.

```
get = function () {
  return {
    model: {
      id: null,
      nama: null,
      no_telp: null,
      alamat: null,
      mode: null
    },
    tableName: "",
    fields: [
      { key: "id" },
      { key: "mode" },

      // Data Pelanggan
      { template: "<h5>Data Pelanggan</h5>" },
      {
        className: "col-6",
        key: "nama",
        type: "label",
        props: { label: "Nama" }
      },
      {
        className: "col-6",
        key: "no_telp",
        type: "label",
        props: { label: "No Telp" }
      },
      {
        className: "col-12",
        key: "alamat",
        type: "label",
        props: { label: "Alamat" }
      },
      {
        className: "col-12",
        key: "mitra",
        type: "label",
        props: { label: "Mitra" }
      },
      { template: "<br>" },

      // Detail Grid Produk
      { template: "<h5>Detail Produk</h5>" },
```

```
      {
        type: "detail-grid",
        props: {
          tableName: "pesanan",
          detailKey: "pesanan_det",
          columns: [
            { key: "id" },
            { key: "kode", label: "Kode", readonly: true },
            { key: "nama_produk", label: "Nama Produk", readonly: true },
            { key: "qty", label: "Qty", type: "number", readonly: true },
            { key: "price_unit", label: "Price/Unit", type: "money",
readonly: true },
            { key: "jumlah", label: "Jumlah", type: "money", readonly: true
}
          ],
          readonly: true
        }
      },
      { template: "<br>" },

      // Tombol Preview File DP (hanya muncul di mode Cek Pesanan)
      {
        type: "button",
        props: {
          text: "Lihat Bukti DP",
          icon: "preview",
          color: "primary",
          onClick: function () {
            this.f().previewFile('dp');
          }
        },
        expressions: {
          hide: "model.mode !== 'Cek Pesanan'"
        }
      },

      // Tombol Preview File Pelunasan (hanya muncul di mode Cek Pelunasan)
      {
        type: "button",
        props: {
          text: "Lihat Bukti Pelunasan",
          icon: "preview",
          color: "primary",
          onClick: function () {
            this.f().previewFile('pelunasan');
```

```
          }
        },
        expressions: {
          hide: "model.mode !== 'Cek Pelunasan'"
        }
      },
    ],
    functions: {
      onInit: function (id) {
        if (id && id !== "null") {
          const param = JSON.parse(id);
          this.ID = param.id;
          this.model.mode = param.status || param.mode;
          this.model = structuredClone(this.model);

          this.f().loadDataPesanan(this.ID);
        }

        // Inisialisasi grid
        this.grid = this.fields.find(f => f.type ===
"detail-grid")._componentRefs[0].instance.grid;
      },

      loadDataPesanan: function (id) {
        const param = {
          filter: [{ junction: "", column: "pesanan.id", operator: "=",
value1: id }],
          columns: [
            { data: "pesanan.id", key: "id" },
            { data: "pesanan.nama", key: "nama" },
            { data: "pesanan.no_telp", key: "no_telp" },
            { data: "pesanan.alamat", key: "alamat" },
            { data: "pesanan.mitra_id.id.master_mitra.nama", key: "mitra" },
          ]
        };

        this.dataService.queryData(param, this.config.api, true, true)
          .subscribe(data => {
            if (data.length > 0) {
              const row = data[0];
              function capitalize(str) {
                return str.charAt(0).toUpperCase() +
str.slice(1).toLowerCase();
              }
```

```
            Object.keys(row).forEach(key => {
              if (row[key] !== undefined && row[key] !== null) {
                this.model[key] = ` ${capitalize(key)}: ${row[key]}\u00A0
`;  //awal dan akhir dikasih spasi agar tidak menempel tulisannya
                this.form.get(key)?.setValue(` ${capitalize(key)}:
${row[key]}\u00A0`); //awal dan akhir dikasih spasi agar tidak menempel
tulisannya
              }
            });

            this.model = structuredClone(this.model);
            this.f().loadDetailPesanan(id);
          }
        });
    },

    loadDetailPesanan: function (id) {
      const param = {
        filter: [{ junction: "", column: "pesanan_det.pesanan_id",
operator: "=", value1: id }],
        columns: [
          { data: "pesanan_det.id", key: "id" },
          { data: "pesanan_det.kode", key: "kode" },
          { data: "pesanan_det.nama_produk", key: "nama_produk" },
          { data: "pesanan_det.qty", key: "qty" },
          { data: "pesanan_det.price_unit", key: "price_unit" },
          { data: "pesanan_det.jumlah", key: "jumlah" }
        ]
      };

      this.dataService.queryData(param, this.config.api, true, true)
        .subscribe(data => {
          this.grid.assignData({ data: data });
        });
    },

    previewFile: function (jenis) {
      const param = {
        filter: [
          { junction: "", column: "file_pesanan.pesanan_id", operator:
"=", value1: this.ID },
          { junction: "and", column: "file_pesanan.jenis", operator: "=",
value1: `'${jenis}'` }
        ],
        columns: [
```

```
            { data: "file_pesanan.id", key: "id" },
            { data: "file_pesanan.file_data", key: "file_data" },
            { data: "file_pesanan.tipe_file", key: "tipe_file" }
          ]
        };

        this.dataService.queryData(param, this.config.api, true, true)
          .subscribe(data => {
            if (data.length > 0) {
              const images = data.map(item => ({
                img: `${item.tipe_file},${item.file_data}`
              }));

              const config = new this.MatDialogConfig();
              config.width = "800px";
              config.height = "600px";
              config.data = {
                caption: `Bukti ${jenis === 'dp' ? 'DP' : 'Pelunasan'}`,
                captionOK: "Tutup",
                captionCancel: "",
                model: {},
                fields: [
                  {
                    type: "swiper",
                    props: {
                      images: images,
                      params: {
                        navigation: true,
                        pagination: true,
                        scrollbar: false
                      }
                    }
                  }
                ]
              };

              this.dialog.open(this.FormlyDialogComponent, config);
            } else {
              alert(`Tidak ada bukti ${jenis === 'dp' ? 'DP' : 'pelunasan'}
ditemukan`);
            }
          });
      },
    }
  };
```

```
};
```

34. Contoh untuk membuat params untuk query data. Berikut ini adalah SQL nya

```
SELECT r.ID route_id, r.FGCODE itemno_output, p.ID process_id,
p.OUTPUTQUANTITY, p.ITEMUNIT unit_output,
    prm.ITEMNO itemno_material, prm.QUANTITY qty_material, prm.ITEMUNIT
unit_material
FROM ROUTE r
join PROCESS p on r.ID = p.ROUTE_ID
join PROCESSRAWMATERIAL prm on prm.PROCESSID = p.ID
where p.WORKCENTER_ID = 139
```

Berikut adalah hasil Params untuk query data:

```
const param = {
  length: -1, // mengambil semua data
  columns: [
    { data: "ROUTE.ID", key: "route_id" },
    { data: "ROUTE.FGCODE", key: "itemno_output" },
    { data: "ROUTE.ID.ROUTE_ID.PROCESS.ID", key: "process_id" },
    { data: "A.OUTPUTQUANTITY", key: "outputquantity" },
    { data: "A.ITEMUNIT", key: "unit_output" },
    { data: "A.ID.PROCESSID.PROCESSRAWMATERIAL.ITEMNO", key:
"itemno_material" },
    { data: "B.QUANTITY", key: "qty_material" },
    { data: "B.ITEMUNIT", key: "unit_material" }
  ],
  filter: [
    {
      junction: "",
      column: "A.WORKCENTER_ID",
      operator: "=",
      value1: 139
    }
  ],
  order: [
    { column: 1, dir: "asc" } // mengurutkan berdasarkan Route ID
  ]
};
```